

Savitribai Phule Pune University, Pune

A PROJECT REPORT ON
” SMART FIRE SURVEILLANCE AND ALARM GENERATION SYSTEM
USING RASPBERRY PI AND YOLO-BASED DETECTION”

Submitted By

Karan Santosh Gade (B401000165)

Aditya Vinod Gaikwad (B401000166)

Avishkar Sandip Gondkar (B401000171)

Under the guidance of

Prof. V.S. Tupe

In partial fulfillment for the award of the degree
of

Bachelor of Engineering



DEPARTMENT OF
ELECTRONICS AND TELECOMMUNICATION ENGINEERING

JSPM'S
NARHE TECHNICAL CAMPUS
NARHE, PUNE - 411041

2025-2026



JSPM'S
NARHE TECHNICAL CAMPUS
NARHE, PUNE - 411041



DEPARTMENT OF
ELECTRONICS AND TELECOMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that a Project Report on

**“SMART FIRE SURVEILLANCE AND ALARM GENERATION SYSTEM
USING RASPBERRY PI AND YOLO-BASED DETECTION”**

Submitted By

Karan Santosh Gade (B401000165)

Aditya Vinod Gaikwad (B401000166)

Avishkar Sandip Gondkar (B401000171)

is a bonafide work carried out by them during the academic year 2025-2026 in partial fulfillment of the requirement for the award of Degree of Bachelor of Engineering in Electronics and Telecommunication Engineering under Savitribai Phule Pune University, Pune.

Prof. V. S. Tupe
Project Guide

Dr. K. P. Jadhav
Project Co-ordinator

Dr. S. W. Shaikh
Head of Department

Dr. M. M. Sardeshmukh
Director

External Examiner

Date: / /2026

Place: Pune

ACKNOWLEDGEMENT

Completing this project titled “**Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and YOLO-Based Detection**” has been a valuable learning experience, made possible through the guidance and cooperation of many people.

I am deeply grateful to **Prof. V. S. Tupe**, our guide, for his insightful mentorship, continuous encouragement, and technical guidance throughout the project. His direction helped transform the concept into a working prototype.

My thanks also go to **Dr. M.M. Sardeshmukh** Director of JSPM NTC, and, **Dr. S .W. Shaikh**, Head of the Department of Electronics and Telecommunication Engineering, for providing the necessary infrastructure, resources, and encouragement.

I wish to thank all faculty members and laboratory staff for their guidance and support during experimentation. Finally, special thanks to my family and friends for their unwavering motivation and moral support throughout this work.

Karan Santosh Gade
Aditya Vinod Gaikwad
Avishkar Sandip Gonkar

TABLE CONTENT

Sr. No.	Chapter	Page no.
Chapter 1: Introduction		1-7
1.1	Background of the Study	1
1.2	Problem Statement	2
1.3	Objectives of the Project	3-4
1.4	Scope of the Study	4-5
1.5	Significance of the Project	6-7
Chapter 2: Literature Survey		8-13
2.1	Overview of Traditional Fire Detection Systems	8-9
2.2	IoT-based Surveillance and Smart Safety Systems	9-10
2.3	Deep Learning for Object Detection: A Review	10-11
2.4	Comparison between Sensor-based and Vision-based Detection	11-12
2.5	Comparative Analysis: Sensor-based vs. Vision-based Detection and Research Gap	13
Chapter 3: System Requirements		14-22
3.1	Introduction	14
3.2	Hardware Requirements	14
3.2.1	Primary Processing Unit	14-15
3.2.2	Secondary Microcontroller	15-16
3.2.3	Camera Module	16
3.2.4	Environmental Sensors and Output Peripheral	17-18
3.3	Software Requirements	19
3.3.1	Operating System	19
3.3.2	Programming Language	19
3.3.3	AI and Computer Vision Libraries	19
3.3.4	Web Framework and Communication Services	20
3.3.5	5 Development and Deployment Tools	20
3.4	Functional Requirements	20-21
3.5	Non-Functional Requirements	21-22
Chapter 4: System Design and Architecture		23-30
4.1	System Architecture Overview	23-24
4.2	Hardware Interfacing Design	24
4.2.1	4.2.1 Raspberry Pi 4 Peripheral Connections	24
4.2.2	ESP8266 NodeMCU Sensor Connections	24-25
4.3	Software Architecture and Module Design	25-27

4.4	System Flowchart	27-28
4.5	Mathematical Model	28
4.5.1	Object Detection and Bounding Box Regression	28-29
4.5.2	Non-Maximum Suppression	29
4.5.3	Four-Case Detection Logic	29-30
Chapter 5: Methodology And Implementation		30-41
5.1	Overview of Implementation Approach	30
5.2	Hardware Assembly and Physical Setup	30-32
5.3	ESP8266 Firmware Development	32-33
5.4	Raspberry Pi Software Environment Setup	33-34
5.5	YOLO Model Preparation and ONNX Export	34-35
5.6	Image Preprocessing and Inference Pipeline	35-36
5.7	Sensor Data Acquisition and Threshold Evaluation	36-37
5.8	Multi-Level Alert Implementation	37-38
5.9	Web Dashboard Development and Remote Monitoring	38-40
5.10	System Integration and Startup Automation	40-41
Chapter 6: Results and Discussion		41-44
6.1	Overview	41
6.2	System Startup Performance	42
6.3	Alert Dispatch Latency	43
6.4	Comparison with Conventional Fire Detection	43
6.5	Limitations and Summary	44
Chapter 7: Conclusion and Future Word		45-47
7.1	Conclusion	45
7.2	Advantages of System	45-46
7.3	Disadvantages of the Proposed System	46-47
7.4	Future Scope	47
References		48
Appendices		49-59
Appendix A- Published Research Paper		50-56
Appendix B- Published paper Certificates		57-59
Annexure		60-85
Plagiarism Report		86-95

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Figure 3.1	RaspberryPi4 Model B Architecture and Pinout	15
Figure 3.2	ESP 8266 Node MCU	16
Figure 3.3	Zebronics Web Cam 1080p	17
Figure 3.4	MQ-2 Smoke/Gas Sensor	17
Figure 3.5	LM393 Flame Sensor	18
Figure 3.6	Buzzer	18
Figure 4.1	System Block Diagram of Smart Fire Detection and Surveillance System	23
Figure 4.2	Circuit Diagram	25
Figure 4.3	Data Flow Diagram of System Software Architecture	27
Figure 4.4	System Flowchart for Fire & Smoke Monitoring	28
Figure 5.1	Hardware/Software setup	32
Figure 5.2	YOLOv8 Model Architecture for Fire and Smoke Detection	35
Figure 5.3	Image Preprocessing using OpenCV (Resizing and Normalization)	36
Figure 5.4	Real-time Alert Notification Screenshots (SMS)	38
Figure 5.5	Web Dashboard User Interface (React.js+Flask)	39
Figure 5.6	Store Video List View	40
Figure 5.7	Code/Web Dashboard	41
Figure 5.8	Result/Detection of fire	41

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 6.1	Fire and Smoke Detection Results Across Test Scenarios	42
Table 6.4	Alert Dispatch Latency (Case 3, n = 5 trials)	43
Table 6.3	Proposed System vs. Conventional Smoke Detector	44

ABSTRACT

This project presents **Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and YOLO-Based Detection** designed to modernize fire safety protocols by integrating **Computer Vision** and **Internet of Things (IoT)** technologies. Traditional fire alarm systems primarily rely on physical sensors that detect smoke or heat, often suffering from delayed response times due to the need for fire indicators to physically reach the sensor's proximity. To address these limitations, the proposed system utilizes a **Raspberry Pi 4** integrated with a **Zebronics WebCam Module** and high-accuracy physical sensors to provide a dual-verification safety framework.

The core of the system is the **YOLO (You Only Look Once)** object detection model, which is optimized into the **ONNX (Open Neural Network Exchange)** format to ensure real-time performance and low memory consumption on resource-constrained edge devices. The architecture features intelligent decision-making logic that combines vision-based data with atmospheric sensor thresholds to categorize threats into multi-level alerts.

Upon detection, the system activates a local buzzer alarm, sends SMS notifications to administrators, and automatically alerts nearby fire stations in critical situations. Furthermore, a web-based administrative dashboard developed using **Flask** and **React.js** provides live video streaming and real-time monitoring capabilities. All event logs and video recordings are securely stored in the cloud, offering a reliable source of evidence for forensic investigations.

ORGANIZATION OF REPORT

The structure of this report is organized to systematically guide the reader through the motivation, design, and implementation of the Sign Language Interpreter system. The chapters are organized as follows:

Chapter 1: Introduction

Introduces the problem statement, the primary objectives of the project, and the societal relevance of developing an automated sign language interpreter.

Chapter 2: Literature Survey

Provides a critical review of existing traditional sensor-based systems and early computer vision models, highlighting the evolutionary shift toward edge-based AI classification.

Chapter 3: System Requirements

Details the specific hardware components (Raspberry Pi 4, camera, I2C LCD) and the optimized software stack (OpenCV, MediaPipe, TensorFlow Lite) required for real-time edge deployment.

Chapter 4: System Design and Architecture

Presents the functional block diagram, system architecture, and logical flowchart, explaining the decoupling of feature extraction from neural network classification.

Chapter 5: Methodology and Implementation

Explains the step-by-step workflow, from the curation of a custom A-Z dataset and mathematical coordinate normalization to the final deployment of the multimodal output loop on the microcomputer.

Chapter 6: Results and Discussion

Analyzes the system's accuracy, real-time frame rates, hardware efficiency, and latency limits, providing a comparative analysis against older methodologies.

Chapter 7: Conclusion and Future Work

Summarizes the final achievements of the project and outlines potential future enhancements, such as tracking dynamic sentences and implementing over-the-air updates.

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Fire-related incidents remain a persistent and severe safety concern across industrial, commercial, and residential environments in India and globally. Data from the National Crime Records Bureau (NCRB) consistently places fire accidents among the leading causes of accidental deaths and property destruction, with thousands of incidents reported each year in urban and semi-urban settings. The human and economic toll of these events highlights a critical gap between the fire safety infrastructure currently deployed and the level of responsiveness that modern environments demand.

Conventional fire detection systems, which have remained largely unchanged in their fundamental design for several decades, depend on physical threshold-based sensors — ionization smoke detectors, photoelectric smoke sensors, and fixed-temperature heat elements. While these devices constitute the foundational layer of fire safety infrastructure, they exhibit a well-documented architectural limitation: their operation is predicated entirely on physical proximity. An alarm is triggered only after smoke particles or thermal energy accumulate beyond a threshold concentration within the sensor's immediate detection chamber. In large, open, or mechanically ventilated environments such as warehouses, server rooms, or manufacturing floors, the physical dispersion of combustion byproducts can be significantly delayed by air movement and structural barriers, allowing a containable fire to develop into a major emergency before any alarm is raised [2][4].

Furthermore, most installed conventional systems provide only a binary output — either an alarm is triggered or it is not — with no visual context made available to responders or administrators. In practice, this means that emergency personnel can only assess the severity and location of an incident upon physical arrival at the scene, contributing to inefficient resource deployment and delayed intervention.

The evolution of Edge Artificial Intelligence and convolutional neural network-based object detection offers a fundamentally different approach to fire detection. The YOLO (You Only Look Once) family of single-pass detection architectures, which frames detection as a unified regression problem processed in a single forward pass [1][2], has demonstrated real-time detection performance suitable for safety-critical applications.

Unlike earlier multi-stage detectors, YOLO-based models can analyze a complete video frame and return detection outputs at frame rates sufficient for live surveillance systems [3][5].

To make such AI capability practically deployable without specialized hardware, this project employs the Raspberry Pi 4 Model B as the central processing unit in combination with the ONNX (Open Neural Network Exchange) Runtime, which is used to execute an optimized YOLOv8 model with improved inference throughput and reduced memory consumption compared to native framework execution [3][4]. This design choice makes high-accuracy, camera-based fire detection viable on a low-cost embedded platform. Parallel to the visual detection pipeline, an ESP8266 NodeMCU microcontroller handles dedicated sensor acquisition from an MQ-2 Smoke/Gas Sensor and an LM393 Infrared Flame Sensor, providing a hardware-level detection layer that operates independently of the camera and network.

The resulting system — a Smart Fire Detection and Surveillance System — represents a hybrid architecture that unifies AI-based visual analysis, hardware sensor monitoring, a multi-level alert pipeline, and a web-based remote monitoring dashboard into a single, cohesive safety framework. This approach addresses both the detection accuracy limitations and the remote monitoring deficiencies of conventional fire alarm systems

1.2 Problem Statement

Despite widespread deployment of conventional fire safety systems, three fundamental technical deficiencies continue to limit their effectiveness in modern environments and motivate the development of the proposed system:

- 1. Proximity-Dependent Detection with Delayed Response:** Standard smoke and gas sensors require physical atmospheric changes — smoke particles or thermal energy — to reach and exceed a threshold at the sensor's detection element before an alarm is triggered. In environments with active ventilation, elevated ceilings, or structural partitions, this physical travel time introduces a detection delay during which a fire may grow beyond the window where suppression is straightforward [2][4]. Vision-based detection, by contrast, can identify the visual signature of flames or smoke plumes the moment they appear within the camera's field of view, regardless of atmospheric dispersion condition
- 2. Absence of Real-Time Remote Monitoring and Visual Verification:** Conventional alarm systems typically provide only a local audible alert. When facility administrators or

safety personnel are off-site, there is no mechanism to remotely assess the nature or severity of a detected event. This delays the appropriate escalation of response and can result in either under-response (treating a real fire as a false alarm) or over-response (evacuating unnecessarily for a nuisance alarm). A web-accessible live video feed integrated with sensor data addresses this gap by enabling remote visual verification before and during emergency response.

- 3. Computational and Hardware Constraints in Edge AI Deployment:** Deploying deep learning detection models on resource-constrained single-board computers introduces known challenges. Standard YOLO model implementations in native training frameworks consume significant RAM and computational cycles, which can produce inference latency that is unacceptable for life-safety applications on platforms such as the Raspberry Pi 4 [3][5]. Additionally, the high-frequency write cycles associated with continuous event logging on SD card storage — the standard storage medium for such embedded systems — accelerate media wear and can lead to storage failure in long-duration surveillance deployments, undermining system reliability.

This project directly addresses all three deficiencies by designing and implementing a system that integrates ONNX-optimized YOLOv8-based visual fire detection, dual hardware sensor monitoring (MQ-2 and LM393 via ESP8266), an intelligent four-case decision engine, and a Flask-React.js remote monitoring dashboard with cloud-backed data persistence

1.3 Objectives of the Project

The primary objective of this project is to design, develop, and validate a Smart Fire Detection and Surveillance System on the Raspberry Pi 4 platform that provides more reliable and faster fire detection than conventional sensor-only alarm systems. The specific objectives are as follows:

- 1. Real-Time Visual Fire Detection:** — To implement the YOLOv8 object detection model, converted to and executed in the ONNX format, for real-time identification of fire and smoke in live video streams captured by a Pi-compatible camera module at viable frame rates on the Raspberry Pi 4 [1][3].
- 2. Multi-Sensor Environmental Monitoring:** — To interface an MQ-2 Smoke/Gas Sensor and an LM393 Infrared Flame Sensor via an ESP8266 NodeMCU, providing a hardware-

level detection layer that monitors physical atmospheric changes continuously and independently of network status.

3. **Edge-AI Performance Optimization:** To evaluate and validate the ONNX Runtime execution of the YOLOv8 model for improved inference speed and reduced RAM footprint relative to a standard framework deployment, ensuring the Raspberry Pi 4 can concurrently handle detection inference, sensor data acquisition, and web server operation without latency degradation [3][4][5].
4. **Intelligent Multi-Source Decision Logic:** To design and implement a four-case decision engine that fuses outputs from the camera-based YOLO detection pipeline and the hardware sensor threshold comparisons to classify fire events into four confidence levels and trigger proportional, tiered responses.
5. **Multi-Level Alert and Emergency Response:** To develop an automated alert pipeline that activates a local buzzer alarm upon any detection event, dispatches a notification to the administrative web dashboard for investigation-level alerts, and delivers an emergency SMS alert to the nearest fire station when both sensors and camera simultaneously confirm a fire event.
6. **Remote Surveillance Dashboard:** To develop a Flask-based backend server and React.js web dashboard that streams live video and human-readable sensor values to any networked device, enabling remote monitoring, live alert visualization, and retrospective review of recorded events.
7. **System Reliability and Cloud Data Persistence:** To implement storage management practices that mitigate SD card wear from high-frequency event logging, and to integrate cloud storage for the automated archival of video recordings and sensor logs to support post-event forensic analysis and evidence preservation

1.4 Scope of the Study

The scope of this research encompasses the complete development lifecycle of the proposed Smart Fire Detection and Surveillance System, from hardware selection and circuit design through software implementation, system integration, and functional validation. The scope is defined across four dimensions:

Hardware Scope

The physical system is constructed around the Raspberry Pi 4 Model B (4 GB RAM variant) as the Main Processing Unit, which executes all computer vision inference, decision logic, and

backend server functions. The ESP8266 NodeMCU serves as a dedicated sensor and local alert node, handling real-time acquisition from the MQ-2 Smoke/Gas Sensor and the LM393 Infrared Flame Sensor and driving the local buzzer alarm independently of the Raspberry Pi. A Pi-compatible USB camera module provides the visual input stream. All hardware components are commercially available, low-cost items with well-documented interfacing specifications.

Software Scope

The detection engine is based on the YOLOv8 architecture trained on a fire and smoke dataset, converted to the ONNX format, and executed using the ONNX Runtime Python library. Pre-processing of video frames — including resizing and normalization — is performed using OpenCV. The Flask Python framework serves as the local backend, managing the live video stream, sensor data ingestion from the ESP8266, decision logic execution, and dashboard communication. The administrative interface is implemented using React.js. A Node.js/Express auxiliary backend facilitates the upload of video recordings and event logs to cloud storage [9][10].

Functional Scope

The system operates as an infinite autonomous monitoring loop, simultaneously acquiring live video frames from the camera and real-time sensor readings from the ESP8266. The decision engine evaluates the combined detection state across four defined cases and executes the appropriate alert action for each. The entire alert pipeline — buzzer, dashboard notification, and SMS dispatch — is handled automatically without requiring human intervention. The web dashboard provides continuous live preview of the camera feed and sensor readings to any authorized remote user

Deployment Scope and Limitations

The system is designed and validated for indoor deployment in environments such as industrial storage areas, server rooms, laboratory spaces, and residential premises. The architecture supports scalability through the addition of further camera and sensor nodes reporting to a centralized Raspberry Pi instance. This study does not address outdoor fire detection, multi-floor coverage from a single hardware unit, or integration with existing building management systems. Autonomous fire suppression actuation is identified as a future enhancement and falls outside the scope of the current prototype.

1.5 Significance of the Project

The significance of this project lies in its potential to advance fire safety practice through a set of technical and practical contributions at multiple level.

Key areas of significance include:

Technical Contribution: Edge AI Optimization

This project demonstrates that a YOLOv8 deep learning detection model, when converted to the ONNX interchange format and executed via the ONNX Runtime on the Raspberry Pi 4, achieves real-time fire detection performance without requiring dedicated GPU hardware. This directly addresses a recognized challenge in the deployment of deep learning models on resource-constrained embedded platforms, as discussed extensively in object detection survey literature [2][3][4][5]. The implementation provides a reproducible framework for edge-deployable AI safety systems on low-cost.

Technical Contribution: Hybrid Detection Architecture

By integrating two independent detection modalities — AI-based visual detection through the camera pipeline and physical threshold detection through hardware sensors — within a unified four-case decision engine, this project produces a system that is more robust than either modality individually. The decision logic ensures that partial detections, such as sensor-only events (possible hidden fire or gas leak) and camera-only events (possible distant fire not yet reaching sensors), are not discarded but are escalated appropriately. This redundancy is critical in safety-critical systems where missed detections have severe consequences.

System Contribution: Reliability Through Architectural Separation

The delegation of sensor acquisition and local alarm control to a dedicated ESP8266 node, independent of the Raspberry Pi's processing load, represents a deliberate architectural decision with significant reliability implications. Local audible alarms function regardless of network connectivity or AI inference load. This separation also eliminates resource contention between compute-intensive YOLO inference and time-critical sensor polling, improving the determinism of both subsystems.

Practical Contribution: Accessible and Scalable Safety Technology

The entire hardware bill of materials for the proposed system consists of commercially available, low-cost embedded components. This positions the system as a cost-effective alternative to high-end commercial fire monitoring infrastructure, making intelligent, camera-based fire safety accessible to small manufacturing units, educational institutions, and residential buildings — contexts that are frequently under-served by conventional premium fire safety products. The modular architecture further supports scalability, enabling deployment across larger facilities through the incremental addition of sensor and camera nodes [8][9][10].

Practical Contribution: Data-Driven Evidence Archival

The automated cloud archival of video recordings and sensor event data provides facility operators and insurers with a reliable, time-stamped record of environmental conditions before, during, and after a fire event. This capability — which is largely absent from conventional alarm systems — supports forensic investigation, insurance claims processing, and post-incident safety audits. By integrating evidence preservation as a native system feature rather than an afterthought, this project contributes a model for how intelligent safety systems can simultaneously serve operational and administrative needs [11][12].

CHAPTER 2: LITERATURE SURVEY

2.1 Overview of Traditional Fire Detection Systems

Fire detection technology has existed in institutional and industrial settings for over a century, yet the core operational principle of the most widely installed systems has remained fundamentally unchanged. Conventional fire detection infrastructure depends on physical, threshold-based sensors designed to respond to specific combustion byproducts — smoke particles, heat accumulation, or combustible gas concentrations — when these byproducts physically reach the sensor's detection element.

The two most prevalent sensor categories in legacy installations are ionization-type smoke detectors, which respond to the disruption of an internal ionization current by smoke particles, and photoelectric smoke detectors, which detect the scattering of an infrared beam caused by suspended smoke. Fixed-temperature and rate-of-rise heat detectors constitute a third category used in environments where smoke-based detection is unreliable. In dedicated industrial contexts, gas sensors such as the MQ series respond to the presence of combustible gases including methane, carbon monoxide, and propane, providing an additional chemical-level safety layer.

Despite their widespread adoption, these systems exhibit three well-established limitations that are particularly pronounced in modern environments.

1. First, they are inherently proximity-dependent: the sensor triggers an alarm only after the physical concentration of smoke, gas, or heat exceeds a threshold at the sensor element itself. In mechanically ventilated spaces, large-volume warehouses, or environments with structural partitions, combustion byproducts may take several minutes to accumulate sufficiently at any given sensor location. This detection delay can be decisive in the context of a rapidly developing fire.
2. Second, conventional sensors are susceptible to false alarms from non-fire sources such as cooking steam, dust, aerosol sprays, and humid conditions, particularly with ionization-type detectors.
3. Third, and most significantly for modern operational requirements, these systems provide no remote monitoring capability and no visual context — only a local audible or visual alarm that offers responders no information about the fire's size, location, or rate of development before arrival'

These documented limitations of sensor-only fire detection systems establish the engineering rationale for augmenting or replacing physical-threshold detection with AI-based visual detection, which forms the core contribution of the proposed system

2.2 IoT-based Surveillance and Smart Safety Systems

The integration of the Internet of Things (IoT) with safety and monitoring infrastructure has significantly expanded the design space for responsive, data-driven safety systems. Rather than operating as isolated alarm generators, IoT-enabled systems function as networked nodes that collect, transmit, and act on environmental data in near real-time, enabling remote oversight and automated response at a scale that was previously impractical.

Lingaraju et al. [9] demonstrated the viability of multi-sensor IoT fusion in their development of a smart waste segregation system for urban environments. Their architecture combined location tracking, air quality sensors, and cloud-based data aggregation to enable real-time environmental monitoring over a distributed deployment. The approach illustrated in [9] is directly relevant to the present work in two ways: first, it demonstrates that fusing data from heterogeneous sensor types — atmospheric and location — within a single decision framework produces more reliable outputs than any single sensor modality; second, it establishes a model for how lightweight IoT nodes can offload specific sensing responsibilities to reduce the computational burden on the central processing unit.

Karamitsos, Apostolopoulos, and Tsaramirsis [10] presented an IoT-based smart parking management system in which wireless sensor nodes deployed across a parking facility communicated occupancy data to a centralized server for real-time availability tracking. Their work highlighted two design principles applicable to the proposed fire detection system: the use of a layered architecture in which edge nodes handle data acquisition locally while a central unit handles analytics, and the importance of low-latency communication between edge nodes and the administrative interface for time-sensitive applications.

In the domain of security infrastructure, Karamitsos et al. [8] examined the application of blockchain-based smart contracts to airport security operations, demonstrating the potential of intelligent, automated decision pipelines in high-stakes safety environments. While the specific technology differs from the present work, the underlying principle — that safety-critical systems

benefit from predefined, automated decision logic rather than reliance on human intervention — is directly applicable to the multi-level alert engine designed in this project.

Collectively, the literature on IoT-based smart systems establishes three design principles that inform the architecture of the proposed system: sensor fusion across multiple input types improves detection reliability; a distributed node architecture reduces resource contention at the central processor; and real-time communication to a remote administrative interface is essential for operational effectiveness in safety applications.

2.3 Deep Learning for Object Detection: A Review

Object detection — the task of simultaneously identifying the class and spatial location of objects of interest within an image — is a foundational problem in computer vision with extensive application to surveillance, autonomous systems, and safety monitoring. The evolution of deep learning architectures has driven a substantial improvement in detection accuracy and speed over the past decade, as documented across several comprehensive survey studies [2][3][4][5].

Liu et al. [2] provided a comprehensive taxonomy of deep learning-based object detection methods in their survey published in the International Journal of Computer Vision. They classified detection architectures into two principal families: two-stage detectors, such as the R-CNN series, which first generate region proposals and then classify each region independently; and one-stage detectors, such as YOLO and SSD, which predict class labels and bounding box coordinates directly from the entire input image in a single forward pass. Liu et al. [2] noted that while two-stage detectors historically achieved higher accuracy, one-stage detectors offered significantly lower inference latency — a property that is critical in real-time safety applications.

Zaidi et al. [3] extended this analysis in their 2022 survey by examining the specific optimizations introduced in modern one-stage architectures, including anchor-free detection heads, multi-scale feature aggregation via feature pyramid networks, and the transition from anchor-based to point-based detection representations. Their survey [3] also explicitly addressed the challenge of deploying these models on resource-constrained hardware, noting that model compression techniques — including quantization, pruning, and format conversion via interchange runtimes — are essential for achieving acceptable inference throughput on embedded processors.

Kaur and Singh [4] provided a similarly detailed review in 2023, with particular attention to benchmark performance across the COCO and Pascal VOC datasets. Their analysis confirmed that YOLO-family models consistently achieve the best tradeoff between mean average precision (mAP) and frames per second (FPS) among one-stage detectors, making them the appropriate architectural choice for applications requiring both accuracy and real-time performance.

Xiao et al. [5] approached the review from an application perspective, surveying object detection deployments across domains including autonomous vehicles, medical imaging, and surveillance. Their work [5] highlighted that fire and smoke detection from video streams is an established application area in which visual detection consistently outperforms sensor-only approaches in terms of detection range and response speed, particularly in open and ventilated environments.

Minaee et al. [6] surveyed deep learning approaches to image segmentation, a closely related task to object detection. Relevant to the present work, their analysis of encoder-decoder architectures and attention mechanisms provides context for understanding how feature extraction pipelines in models such as YOLOv8 achieve robustness across varying visual conditions — including the variable illumination and smoke-obscured scenes characteristic of fire events.

Han et al. [7] surveyed the Vision Transformer (ViT) architecture and its variants, documenting the emergence of attention-based feature extraction as an alternative to purely convolutional approaches. While the present project employs a convolutional YOLO-based model for practical edge-deployment reasons, the survey by Han et al. [7] provides relevant background on the trajectory of visual feature extraction research and identifies the trade-offs between transformer-based and convolutional architectures in resource-constrained contexts.

The combined body of detection literature [2][3][4][5][6][7] establishes that the YOLOv8 architecture, converted to an optimized execution format for edge hardware, represents the appropriate and well-justified technical choice for the real-time fire detection component of the proposed system

2.4 Comparison between Sensor-based and Vision-based Detection

The YOLO (You Only Look Once) algorithm, first introduced by Redmon et al. in 2016 and subsequently developed through multiple iterations by different research groups, represents the most widely adopted family of real-time object detectors. The defining characteristic of YOLO is

its formulation of object detection as a single regression problem: the entire input image is processed in one forward pass through the network, which simultaneously predicts bounding box coordinates, objectness scores, and class probabilities for all candidate detections across a grid of spatial cells.

Jocher and Qiu [1] document the most recent iteration, YOLO11 (under the Ultralytics framework), which introduces architectural refinements over the YOLOv8 baseline including improved backbone feature extraction, optimized neck aggregation, and a more efficient detection head. The YOLOv8 architecture, which serves as the base for the detection engine in the present project, achieves state-of-the-art accuracy-speed tradeoffs on standard benchmarks while supporting multiple output formats including PyTorch, TorchScript, CoreML, TensorFlow, and ONNX [1]

A central technical challenge addressed in this project is the deployment of a YOLOv8 model on the Raspberry Pi 4, a single-board computer with a quad-core ARM Cortex-A72 processor and shared CPU-RAM architecture, without dedicated GPU acceleration. In native PyTorch execution, deep learning models of this class impose a RAM footprint and per-inference computation time that approaches the limits of comfortable operation on this hardware, as noted in the edge deployment literature [3][5]. To address this, the model is converted to the ONNX (Open Neural Network Exchange) format and executed using the ONNX Runtime.

ONNX is an open specification for representing machine learning models in a framework-agnostic intermediate representation. The ONNX Runtime is a high-performance inference engine that applies graph-level optimizations — including operator fusion, constant folding, and memory planning — to the ONNX model graph before execution, yielding improvements in inference throughput and reductions in memory consumption relative to native framework execution. For edge devices such as the Raspberry Pi 4, these optimizations are practically significant: a reduction in per-inference latency directly translates to a higher number of video frames processed per second, improving the responsiveness of the fire detection pipeline, while reduced RAM consumption allows the device to concurrently handle the YOLO inference engine, the sensor data acquisition task, and the Flask web server without memory pressure.

The literature review thus confirms that the combination of YOLOv8 architecture and ONNX Runtime optimization is both technically justified and practically necessary for achieving real-time performance in a camera-based fire detection system deployed on a resource-constrained edge platform.

2.5 Comparative Analysis: Sensor-based vs. Vision-based Detection and Research Gap

The preceding review of traditional fire detection systems (Section 2.1), IoT-based surveillance architectures (Section 2.2), and deep learning detection methods (Sections 2.3 and 2.4) reveals a consistent pattern in the literature: sensor-based and vision-based approaches each address a distinct subset of the fire detection problem, and neither modality, when deployed independently, constitutes a complete solution.

The comparative characteristics of the two approaches are summarized as follows. Sensor-based detection provides reliable, low-cost chemical and thermal monitoring but is limited in spatial coverage, dependent on proximity to the fire source, and produces no visual information for remote verification. Vision-based AI detection offers wide-area coverage, distance-independent detection of visual fire signatures, and live video output for remote verification, but is susceptible to false positives from visually similar sources (sunlight reflections, steam, welding arcs) and imposes computational demands that must be carefully managed on edge hardware [3][5].

The research gap identified through this review is the absence, in current low-cost embedded IoT deployments, of a system that: (a) fuses both sensor and camera inputs within a structured, multi-case decision engine rather than treating them as independent alarm sources; (b) executes AI inference on a low-cost edge device using runtime optimization for real-time performance; (c) delivers multi-level alerts calibrated to detection confidence rather than binary alarm outputs; and (d) integrates remote monitoring, cloud archival, and local hardware alerting within a single, cohesive system architecture.

The proposed Smart Fire Detection and Surveillance System is designed explicitly to address this identified gap by building on the established principles of multi-sensor IoT fusion [9][10], one-stage real-time object detection [1][2][3][4][5], and edge AI optimization [3][5], while contributing a practical, validated implementation on commercially available, low-cost hardware.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Introduction

The design and deployment of the Smart Fire Detection and Surveillance System requires a carefully defined set of hardware and software components whose specifications directly determine the system's operational capability. Unlike purely software-based projects, this system operates at the intersection of embedded hardware, sensor electronics, AI inference, and networked communication. The requirement specification must therefore address not only software libraries and APIs but also the physical and electrical constraints of the components involved, the performance envelope expected from the AI pipeline on resource-constrained hardware, and the functional behaviours the system must exhibit under real operational conditions.

This chapter documents the system requirements across four dimensions: hardware requirements covering the processing unit, sensing peripherals, communication modules, and output devices; software requirements covering the operating system, AI runtime, web framework, and third-party communication services; functional requirements defining what the system must do; and non-functional requirements defining the standards of performance, reliability, and maintainability the system must meet.

3.2 Hardware Requirements

3.2.1 Primary Processing Unit

The central processing unit of the proposed system is the Raspberry Pi 4 Model B with 4 GB LPDDR4 RAM. This single-board computer is equipped with a Broadcom BCM2711 system-on-chip, incorporating a quad-core ARM Cortex-A72 processor operating at 1.8 GHz. The board provides 40 general-purpose input/output (GPIO) pins, two USB 3.0 ports, two USB 2.0 ports, a micro-HDMI output, a dedicated camera serial interface (CSI) port, a full-size SD card slot for operating system storage, and onboard IEEE 802.11ac Wi-Fi and Bluetooth 5.0. The 4 GB RAM configuration is a minimum requirement for this system, as the YOLOv8 model converted to ONNX format, the Flask web server, and concurrent sensor acquisition threads must coexist in memory without paging overhead that would degrade real-time performance.

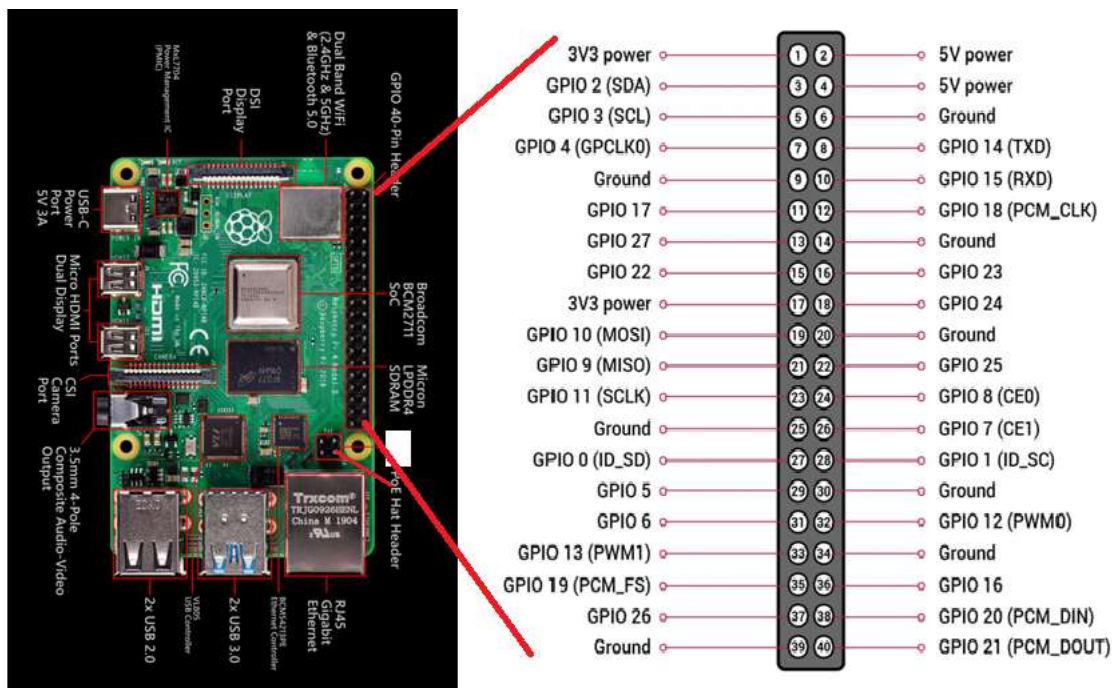


Figure 3.1 Raspberry Pi 4 Model B Architecture and Pin out

The Raspberry Pi 4 was selected over alternatives such as the Raspberry Pi 3B+ or Raspberry Pi Zero 2W on the basis of its higher memory bandwidth and improved CPU performance, both of which are essential for acceptable inference throughput when running ONNX Runtime on a CPU-only deployment. A minimum of a 16 GB Class 10 microSD card is required for the operating system and model storage; a 32 GB card is recommended for systems expected to perform extended local video archival.

3.2.2 Secondary Microcontroller

An ESP8266-based NodeMCU development board serves as the secondary microcontroller responsible for continuous acquisition of environmental sensor data. The ESP8266 integrates a Tensilica L106 32-bit RISC processor operating at 80 MHz, 80 KB of instruction RAM, 96 KB of data RAM, 4 MB of external flash memory, and a built-in Wi-Fi transceiver supporting IEEE 802.11 b/g/n at 2.4 GHz. The NodeMCU board additionally provides onboard USB-to-serial conversion via a CP2102 chip, enabling direct programming from a host computer.



Figure 3.2 ESP 8266 Node MCU

An ESP8266-based NodeMCU development board serves as the secondary microcontroller responsible for continuous acquisition of environmental sensor data. The ESP8266 integrates a Tensilica L106 32-bit RISC processor operating at 80 MHz, 80 KB of instruction RAM, 96 KB of data RAM, 4 MB of external flash memory, and a built-in Wi-Fi transceiver supporting IEEE 802.11 b/g/n at 2.4 GHz. The NodeMCU board additionally provides onboard USB-to-serial conversion via a CP2102 chip, enabling direct programming from a host computer.

3.2.3 Camera Module

Video capture is performed using a Zebronics ZEB-Crystal Pro USB webcam capable of capturing video at 1080p (1920×1080 pixels) at 30 frames per second. The webcam connects to the Raspberry Pi 4 via a USB 2.0 port and is recognised by the Linux kernel as a standard UVC (USB Video Class) compliant device, requiring no additional driver installation beyond what is included in the Raspberry Pi OS kernel. This is a practical advantage over CSI-connected camera modules in that the device can be repositioned, replaced, or tested independently of the main processing board.

For the fire detection pipeline, video frames are captured at 640×480 pixel resolution at 30 frames per second using OpenCV's Video Capture interface, which interfaces with the webcam through the Video4Linux2 (v4l2) backend. This resolution provides sufficient spatial detail for YOLOv8 fire and smoke detection while keeping the per-frame data volume within a range that allows the inference pipeline to achieve acceptable throughput on the Raspberry Pi 4's CPU. The webcam's built-in automatic exposure and white balance adjustment assists detection under varying indoor lighting conditions



Figure 3.3: Zebronics Web Cam 1080p

3.2.4 Environmental Sensors and Output Peripheral

The MQ-2 gas sensor module detects combustible gases including methane, propane, hydrogen, and liquefied petroleum gas (LPG), as well as smoke particulates produced during combustion. The module operates on a 5V supply and provides both an analog output proportional to gas concentration and a digital output triggered when concentration exceeds a preset threshold. The analog output is read via the ESP8266's ADC pin after voltage scaling from 5V to the ESP8266's maximum input of 3.3V using a resistor voltage divider. The onboard potentiometer allows sensitivity threshold adjustment without firmware changes. The sensor requires a warm-up period of approximately 20–30 seconds after power-on before readings stabilise and are considered valid by the firmware.



Figure 3.4 MQ-2 Smoke/Gas Sensor

The LM393-based flame sensor module detects infrared radiation emitted by open flames in the wavelength range of 760 nm to 1100 nm. The module incorporates the LM393 dual voltage comparator IC, which compares the output of the onboard IR photodiode against a reference

voltage set by an onboard potentiometer, producing a digital logic output — logic high under normal conditions and logic low upon flame detection. This digital output is connected directly to a GPIO pin of the ESP8266. The detection range for a standard open flame is approximately 80 cm, with a sensing angle of approximately 60 degrees. The LM393 comparator's open-collector output configuration ensures clean logic transitions without signal noise, making it suitable for direct interfacing with the ESP8266's 3.3V GPIO without additional signal conditioning.



Figure 3.5 LM393 Flame Sensor

A 5V active piezoelectric buzzer module is connected to a GPIO pin of the ESP8266 NodeMCU and is driven directly by the ESP8266 firmware upon receiving an alert trigger from the Raspberry Pi. The module incorporates an onboard transistor driver circuit, allowing the ESP8266's 3.3V GPIO logic output to switch the full 5V buzzer load without direct GPIO current loading. Co-locating the buzzer with the sensor node ensures that a local audible alarm is activated at the point of sensor detection, independent of any network communication with the Raspberry Pi



Figure 3.6 Buzzer

3.3 Software Requirements

3.3.1 Operating System

The system runs on Raspberry Pi OS (formerly Raspbian) based on Debian 11 (Bullseye), 64-bit edition. The 64-bit variant is required to take full advantage of the Raspberry Pi 4's ARMv8-A architecture and to ensure compatibility with the 64-bit builds of Python packages, particularly those in the scientific Python ecosystem. The operating system is installed headless (without a graphical desktop environment) to minimise RAM consumption and background CPU load, leaving maximum resources available for the AI inference and Flask server processes.

3.3.2 Programming Language

Python 3.9 or later is the primary programming language for all components running on the Raspberry Pi, including the video capture pipeline, ONNX inference engine, sensor data receiver, alert dispatch module, and Flask web application. Python was selected for its mature ecosystem of computer vision, machine learning, and web development libraries, and for its compatibility with USB UVC-class webcams through OpenCV's Video Capture interface, which abstracts the v4l2 kernel layer and requires no additional camera-specific libraries.

The ESP8266 firmware is written in the Arduino programming language (C/C++) using the Arduino IDE with the ESP8266 community board support package. The firmware handles sensor polling, ADC reading, and HTTP transmission to the Raspberry Pi's REST endpoint.

3.3.3 AI and Computer Vision Libraries

The OpenCV 4.7 or later is used for video frame capture, pre-processing (resizing, normalisation, colour space conversion), and bounding box annotation on output frames. The headless build (opencv-python-headless) is installed to avoid dependency on a display server.

The ONNX Runtime (version 1.16 or later) is used to load and execute the fire and smoke detection model. The model is a YOLOv8n architecture exported from the Ultralytics framework in ONNX format. ONNX Runtime's graph optimisation pass, applied at model load time, reduces per-inference latency relative to native PyTorch execution on CPU by applying operator fusion and memory planning optimisations. The target inference throughput is a minimum of 4–6 frames per second at 640×640 pixel input resolution on the Raspberry Pi 4B 4 GB.

NumPy is used for tensor manipulation in the pre- and post-processing stages of the ONNX inference pipeline, specifically for input tensor construction, output tensor parsing, and non-maximum suppression

3.3.4 Web Framework and Communication Services

Flask 2.x is used to serve the web-based monitoring dashboard. Flask runs as a lightweight WSGI application on the Raspberry Pi, streaming the annotated video feed via MJPEG over HTTP and serving a real-time sensor data panel via JSON endpoints polled by the browser client. The Flask application is configured to listen on all network interfaces on port 5000, making the dashboard accessible from any device on the local network without additional configuration.

The Twilio REST API is used to dispatch SMS alerts to a pre-configured administrator contact number when the system enters a Case 3 or Case 4 detection state. The twilio Python library is installed via pip and requires valid account SID and authentication token credentials stored as environment variables rather than hardcoded in source files.

Python's built-in smtplib and email modules are used to send email alert notifications via a Gmail SMTP relay. App-specific password authentication is used rather than the account password, in accordance with Google's current authentication requirements for third-party SMTP access.

3.3.5 Development and Deployment Tools

Git is used for version control of all source code. Remote development is performed over SSH from a host machine to the Raspberry Pi using Visual Studio Code with the Remote-SSH extension. Python dependencies are managed using a requirements.txt file for reproducible environment setup. Systemd service units are configured to start the Flask application and sensor listener processes automatically on boot, ensuring continuous operation after power cycling.

3.4 Functional Requirements

The functional requirements define the specific behaviours the system must exhibit. They are organised according to the four detection cases that form the core decision logic of the proposed system.

The system must continuously capture video from the connected camera module at a minimum of 4 frames per second and run ONNX inference on each captured frame to detect the presence of fire or smoke. When a detection occurs with confidence above a configurable threshold (default 0.5), the system must annotate the frame with a labelled bounding box and the associated confidence score and make the annotated frame available on the live dashboard.

The system must continuously receive temperature, humidity, gas concentration, and flame sensor readings from the ESP8266 over the Wi-Fi network at a polling interval not exceeding 5 seconds, and display these readings on the monitoring dashboard with their current status.

In Case 1, when the sensor threshold is exceeded but no visual detection occurs, the system must activate the buzzer, display a sensor alert on the dashboard, send an SMS notification, and log the event. In Case 2, when the camera detects fire or smoke but no sensor threshold is exceeded, the system must display a visual detection warning on the dashboard and log the event. In Case 3, when both camera detection and sensor threshold exceedance occur simultaneously, the system must activate all alert mechanisms — buzzer, SMS, email, and dashboard alert — log the event with the annotated video frame, and upload the event record to the configured cloud storage endpoint. In Case 4, when neither the camera nor any sensor detects an anomaly, the system must display a normal status indication and take no alert action.

The system must maintain a persistent local log of all Case 2, 3, and 4 events, storing the timestamp, detection case, confidence score (where applicable), and sensor readings at the time of the event.

The system must serve a web dashboard accessible to any device on the local network via a web browser without requiring any client-side software installation beyond a modern browser.

3.5 Non-Functional Requirements

- **Performance:** The AI inference pipeline must achieve a minimum throughput of 4 frames per second at 640×480 pixel resolution on the Raspberry Pi 4B 4 GB hardware to ensure that a rapidly developing fire event is not missed between processed frames. The end-to-end latency from the moment a fire is visible in the camera frame to the moment the SMS alert is dispatched must not exceed 30 seconds under normal network conditions.
- **Reliability:** The system must operate continuously without requiring manual intervention for a minimum of 24 hours under normal environmental conditions. The sensor polling architecture must be designed such that a temporary loss of communication from the ESP8266 does not crash the main application; the system must log the communication failure and continue operating in camera-only detection mode.
- **Safety:** Given that the system operates in environments where fire hazards may be present, all electrical connections must use appropriate wire gauges and insulation for the operating voltages involved. The 5V supply to the MQ-2 sensor and the 3.3V supply to the ESP8266

and DHT22 must be drawn from regulated power rails rather than unregulated sources. The Raspberry Pi must be powered by a supply rated at a minimum of 3A at 5V to prevent undervoltage conditions during peak processing load.

- **Maintainability:** The source code must be structured as separate, independently testable modules for video processing, sensor communication, alert dispatch, and web serving. Configuration parameters — including detection thresholds, alert contact numbers, API credentials, and polling intervals — must be stored in a single configuration file and must not be hardcoded within functional modules, so that the system can be reconfigured for a different deployment environment without modifying source code.
- **Portability:** The system architecture must not depend on hardware specific to the Raspberry Pi 4 for its software logic. The ONNX model and inference pipeline must be executable on any ARM or x86-64 Linux host that supports ONNX Runtime, enabling development and testing on a standard laptop before deployment to the Raspberry Pi.
- **Security:** The Flask dashboard must not be exposed to the public internet without authentication. In the default deployment configuration, access is restricted to the local network. API credentials for Twilio and Gmail must be stored as environment variables and excluded from the version control repository using a .gitignore entry.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

The Smart Fire Detection and Surveillance System is designed as a two-node distributed embedded architecture in which two independent processing units — a Raspberry Pi 4 Model B and an ESP8266 NodeMCU — operate concurrently and communicate over a local Wi-Fi network to produce a unified detection decision. This distributed architecture is a deliberate design choice that separates the computationally intensive task of AI-based video inference from the time-critical task of environmental sensor acquisition, preventing resource contention that would otherwise degrade real-time performance on a single embedded processor.

The Raspberry Pi 4 constitutes the primary processing node. It is responsible for continuous video capture from the Zebronics USB webcam, pre-processing of captured frames, execution of the YOLOv8n ONNX inference model, reception and evaluation of sensor data transmitted by the ESP8266, implementation of the four-case detection decision logic, dispatch of alerts through SMS and email APIs, local event logging, and serving the Flask-based web monitoring dashboard to clients on the local network.

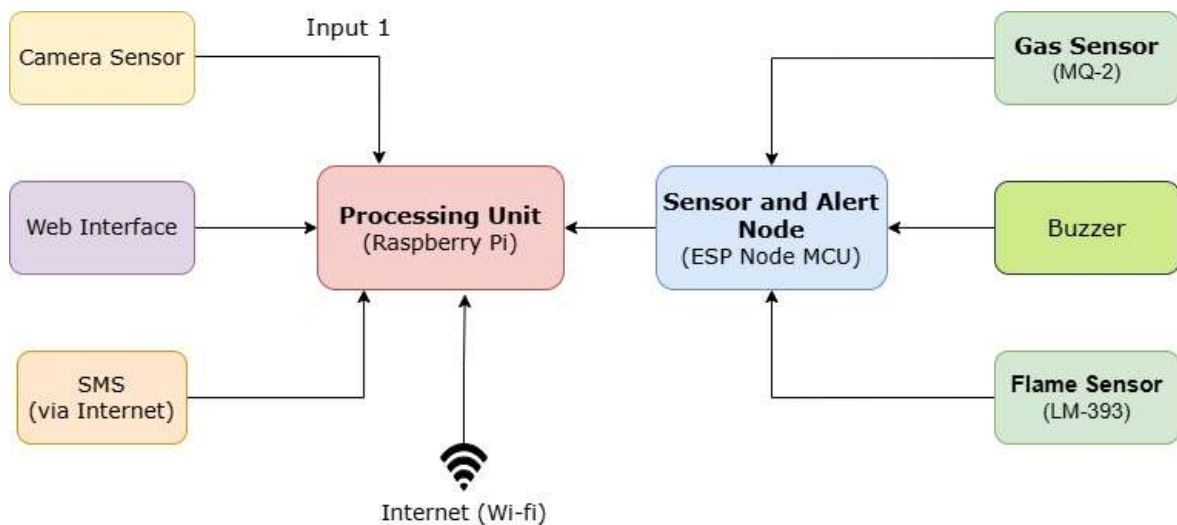


Fig 4.1: System Block Diagram of Smart Fire Detection and Surveillance System

The ESP8266 NodeMCU constitutes the secondary sensor acquisition node. It is responsible for continuous polling of the MQ-2 gas sensor and the LM393 flame sensor at a configurable interval, evaluation of sensor thresholds in firmware, and transmission of sensor status data to the Raspberry Pi via HTTP GET requests over the local Wi-Fi network.

The overall architecture is shown in the block diagram in Fig 4.1.

4.2 Hardware Interfacing Design

4.2.1 Raspberry Pi 4 Peripheral Connections

The Zebronics ZEB-Crystal Pro USB webcam is connected to one of the USB 2.0 ports of the Raspberry Pi 4. As a UVC-compliant device it is recognised automatically by the Linux kernel and exposed as a video device node at `/dev/video0`, which is accessed by the OpenCV Video Capture interface in the application software without requiring any additional configuration.

4.2.2 ESP8266 NodeMCU Sensor Connections

The MQ-2 gas sensor module is powered from the 5V VIN pin of the NodeMCU board. Its digital output pin (D0) is connected to GPIO pin D5 of the NodeMCU, configured as a digital input with the internal pull-up resistor enabled in firmware. Its analog output pin (A0) is connected to the NodeMCU's ADC pin through a voltage divider network consisting of a 10 k Ω resistor between the sensor's AO pin and the ADC pin, and a 3.3 k Ω resistor between the ADC pin and ground. This resistor divider scales the sensor's 0–5V output range down to the 0–3.3V input range of the ESP8266's ADC, preventing damage to the ADC pin. The raw ADC reading (0–1023) is converted to an approximate voltage value in firmware and compared against a configured threshold.

The LM393 flame sensor module is powered from the 3.3V pin of the NodeMCU, as the LM393 comparator IC on the module accepts a supply voltage as low as 2V. Its digital output pin (DO) is connected to GPIO pin D6 of the NodeMCU, configured as a digital input. Since the LM393's open-collector output produces a logic low (approximately 0V) on flame detection and is pulled high (to 3.3V) through an onboard pull-up resistor when no flame is present, the firmware reads a LOW state on D6 as an active flame detection event.

The active buzzer module is connected to GPIO pin D7 of the NodeMCU, configured as a digital output in the ESP8266 firmware. When the Raspberry Pi determines that a Case 3 or Case 4 alert condition has been met, it sends an HTTP POST request to a dedicated endpoint on the ESP8266, which immediately drives D7 high to activate the buzzer. This design ensures the local audible

alarm functions even if there is momentary processing load on the Raspberry Pi, since the buzzer activation is handled entirely within the ESP8266 firmware once the trigger command is received. The complete sensor circuit schematic is shown in Fig 4.2.

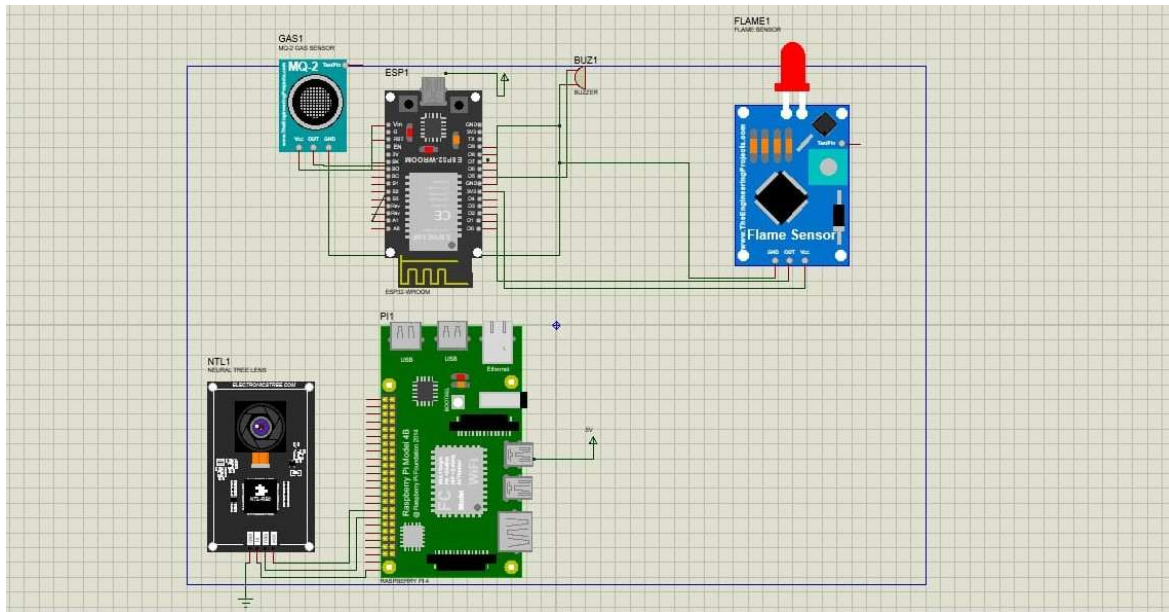


Figure 4.2 Circuit Diagram

4.3 Software Architecture and Module Design

The software running on the Raspberry Pi is organised into five functionally independent modules that communicate through shared in-memory data structures and a lightweight internal event queue. This modular organisation ensures that a failure or blocking condition in one module does not propagate to other modules, and that each module can be independently tested.

The Video Capture and Inference Module is responsible for continuously reading frames from the webcam using OpenCV's VideoCapture interface, resizing and normalising each frame to the 640×640 pixel input tensor format expected by the ONNX model, running inference using the ONNX Runtime session, parsing the output tensor to extract bounding box coordinates, class labels, and confidence scores, applying non-maximum suppression to eliminate redundant detections, and annotating the original frame with detection overlays. The module writes the annotated frame and a boolean detection flag to a shared frame buffer accessed by the Flask streaming thread.

The Sensor Communication Module runs as a Flask route handler that listens for incoming HTTP GET requests from the ESP8266 on a dedicated endpoint. Each request carries the current digital output states of the MQ-2 and LM393 sensors as URL parameters. The module parses these parameters, updates a shared sensor status dictionary, and timestamps the update. If no update is

received within a configurable timeout period (default 10 seconds), the module sets a communication fault flag that is reflected on the dashboard.

The Detection Logic Module reads the current values of the camera detection flag and the sensor status dictionary at each decision cycle and evaluates the four-case detection logic to determine the appropriate system response. The outputs of this module are an alert level (0 through 4 corresponding to the four cases) and a trigger signal sent to the Alert Dispatch Module when the alert level changes.

The Alert Dispatch Module receives trigger signals from the Detection Logic Module and executes the appropriate alert actions for the given case — sending an HTTP POST trigger to the ESP8266 to activate the buzzer, dispatching an SMS via the Twilio REST API, sending an email notification via the Gmail SMTP relay, and writing an event record to the local log file. Alert dispatch for external APIs is performed in a separate thread to prevent network latency from blocking the main detection loop.

The Web Dashboard Module is a Flask application that serves the live annotated MJPEG video stream, a real-time sensor status panel updated via periodic AJAX calls to a JSON endpoint, the current alert level indicator, and a paginated event log table. The dashboard is accessible from any browser on the local network at the Raspberry Pi's IP address on port 5000.

The interaction between these five modules is illustrated in the data flow diagram in Fig 4.3.

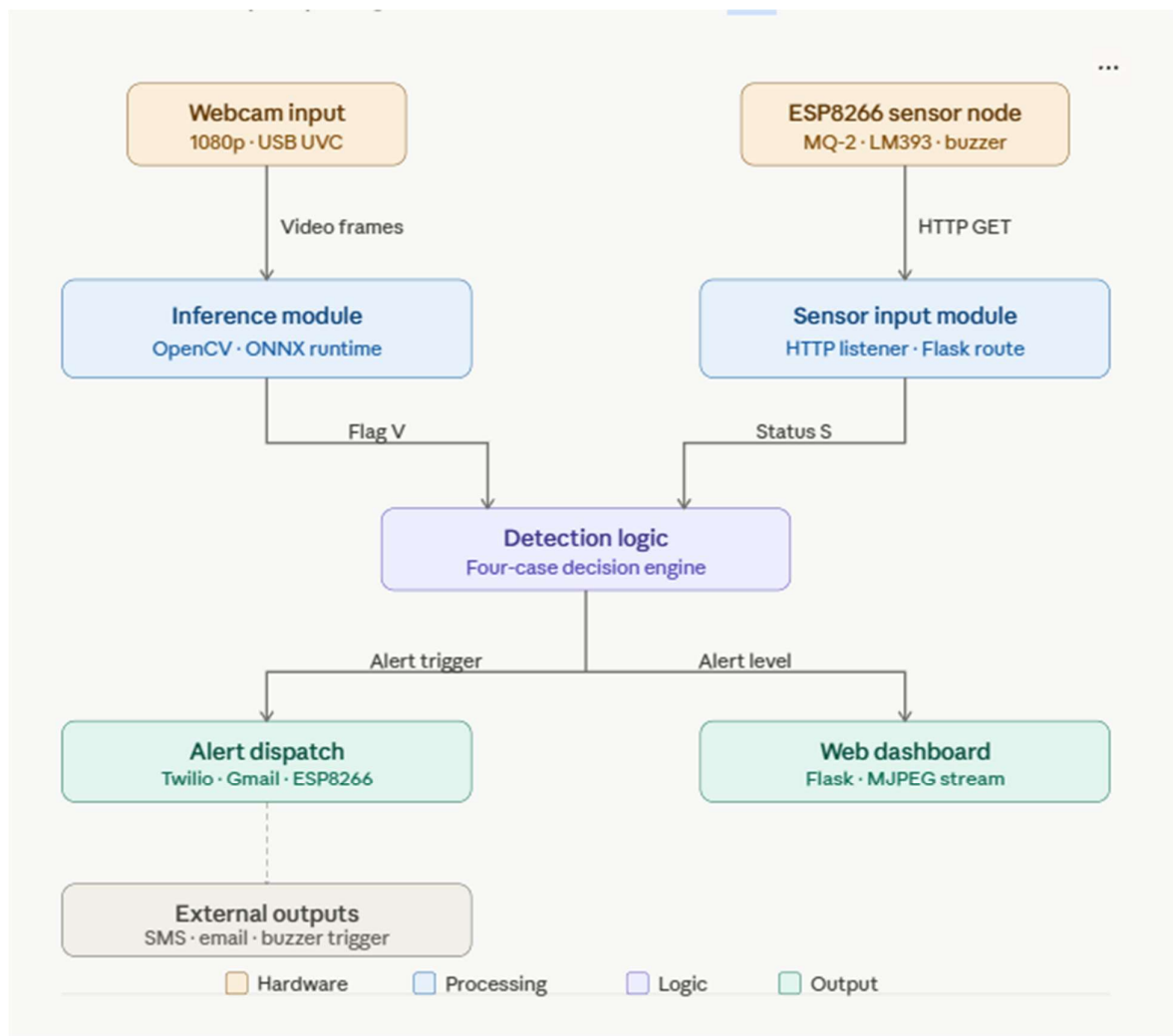


Figure 4.3 Data Flow Diagram of System Software Architecture

4.4 System Flowchart

The complete operational flow of the system from power-on to event logging is described in the flowchart shown in Fig 4.5. On system startup, the ONNX model is loaded into memory, the Flask server is initialised, and the sensor communication listener is started. The main loop then enters continuous operation: a frame is captured and processed, sensor data is checked for freshness, the four-case logic is evaluated, and the appropriate response is executed. The loop runs indefinitely until the system is powered off or the process is terminated.

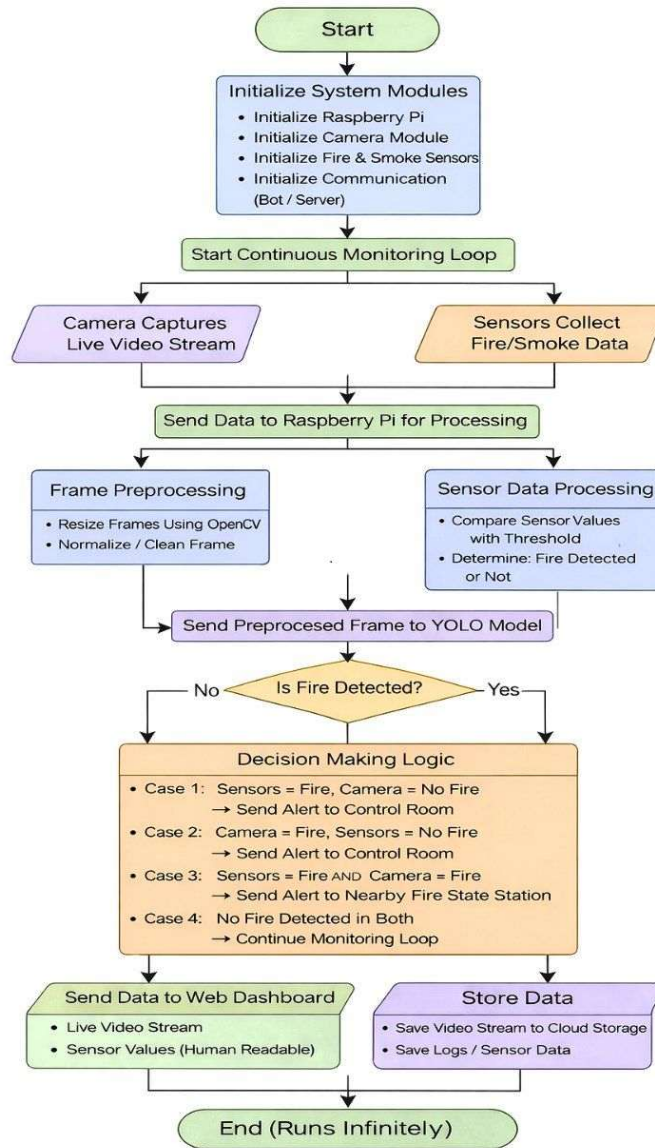


Figure 4.4 System Flowchart for Fire & Smoke Monitoring

4.5 Mathematical Model

4.5.1 Object Detection and Bounding Box Regression

x The YOLOv8n model processes each input frame as a 640×640 pixel tensor and produces a set of candidate detections, each represented as a vector of the form $[x_c, y_c, w, h, \text{confidence}, \text{class_score}]$, where x_c and y_c are the normalised coordinates of the bounding box centre, w and h are the normalised width and height of the bounding box, confidence is the objectness score, and class_score is the probability of the detected class (fire or smoke).

The final detection confidence score P for each candidate detection is computed as the product of the objectness score and the class probability:

$$P = P_{\text{obj}} \times P_{\text{class}}$$

A candidate detection is accepted only when P exceeds the configured confidence threshold τ , set to 0.50 in the default configuration. Detections with $P < \tau$ are discarded prior to non-maximum suppression.

4.5.2 Non-Maximum Suppression

Because the YOLO model evaluates multiple grid cells and anchor scales simultaneously, a single physical fire region in the frame may generate multiple overlapping candidate bounding boxes. Non-maximum suppression (NMS) is applied to retain only the most confident detection among overlapping candidates. The degree of overlap between two bounding boxes B_i and B_j is quantified using the Intersection over Union (IoU) metric, defined as:

$$\text{IoU}(B_i, B_j) = \text{Area}(B_i \cap B_j) / \text{Area}(B_i \cup B_j)$$

Where $\text{Area}(B_i \cap B_j)$ is the area of the region of overlap between the two boxes and $\text{Area}(B_i \cup B_j)$ is the area of their union. The NMS algorithm sorts all candidate detections in descending order of confidence score, retains the highest-confidence detection, and suppresses all other candidates whose IoU with the retained detection exceeds a configured NMS threshold, set to 0.45 in the default configuration. This process is repeated until all candidates have been either retained or suppressed.

4.5.3 Four-Case Detection Logic

Let V denote the boolean output of the video inference module, where $V = 1$ if at least one fire or smoke detection with $P \geq \tau$ survives non-maximum suppression in the current frame, and $V = 0$ otherwise. Let S denote the boolean output of the sensor evaluation module, where $S = 1$ if the MQ-2 digital output indicates gas concentration above its threshold OR the LM393 digital output indicates active flame detection, and $S = 0$ if both sensors report normal conditions. The four-case detection logic is then expressed as follows:

Case 1: $V = 0$ AND $S = 1 \rightarrow$ Sensor trigger only. Buzzer activated, SMS dispatched, dashboard alert displayed, event logged.

Case 2: $V = 1$ AND $S = 0 \rightarrow$ Visual detection only. Dashboard warning displayed and event logged. No external alert dispatched.

Case 3: $V = 1$ AND $S = 1 \rightarrow$ Confirmed fire event. All alert mechanisms activated — buzzer, SMS, email, dashboard alert, cloud log entry.

Case 4: $V = 0$ AND $S = 0 \rightarrow$ Normal state. No alert action taken

.

The system transitions immediately to Case 3 when both V and S simultaneously equal 1, without waiting for a confirmation delay. This design priority ensures minimum response latency for the highest-risk detection scenario at the cost of accepting a marginally higher false positive rate in Case 3, which is the operationally safer trade off for a safety-critical application.

CHAPTER 5: METHODOLOGY AND IMPLEMENTATION

5.1 Overview of Implementation Approach

The implementation of the Smart Fire Detection and Surveillance System follows a bottom-up integration strategy in which each subsystem — hardware interfacing, firmware, AI inference pipeline, alert dispatch, and web dashboard — is developed, tested independently, and subsequently integrated into the complete operational system. This approach reduces the debugging complexity that arises when all components are assembled simultaneously and ensures that each module meets its individual performance targets before being introduced into the multi-threaded main application loop.

The implementation is divided into six sequential phases: hardware assembly and circuit validation, ESP8266 firmware development and sensor calibration, Raspberry Pi environment setup and dependency installation, YOLO model preparation and ONNX export, main application development covering the inference pipeline and alert logic, and finally web dashboard development and end-to-end system integration testing.

5.2 Hardware Assembly and Physical Setup

The physical assembly of the system begins with the ESP8266 sensor node. The MQ-2 gas sensor module is connected to the NodeMCU's 5V VIN rail for power, with its digital output routed to GPIO pin D5 and its analog output routed through a 10 k Ω to 3.3 k Ω resistor voltage divider to the NodeMCU's ADC pin, as specified in the interfacing design of Chapter 4. The LM393 flame sensor module is powered from the NodeMCU's 3.3V output pin, with its digital output connected to GPIO pin D6. The active buzzer module is connected to GPIO pin D7. All ground connections are tied to a common ground rail shared with the NodeMCU's GND pin. Continuity across all connections is verified using a digital multimeter before power is applied.

The Raspberry Pi 4 is powered using a 5V 3A USB-C supply. The Zebronic ZEB-Crystal Pro USB webcam is inserted into one of the USB 2.0 ports. On first boot of Raspberry Pi OS, the webcam is confirmed to be enumerated correctly as a UVC device by running the command `v4l2-ctl --list-devices` in the terminal, which displays the device registered at `/dev/video0`. No additional driver installation is required.

The ESP8266 NodeMCU is connected to the development laptop via a USB micro-B cable for firmware upload. Following successful firmware upload and functional verification, the NodeMCU is powered independently from the same 5V supply rail as the sensors, and Wi-Fi connectivity to the local network shared with the Raspberry Pi is confirmed through the serial monitor

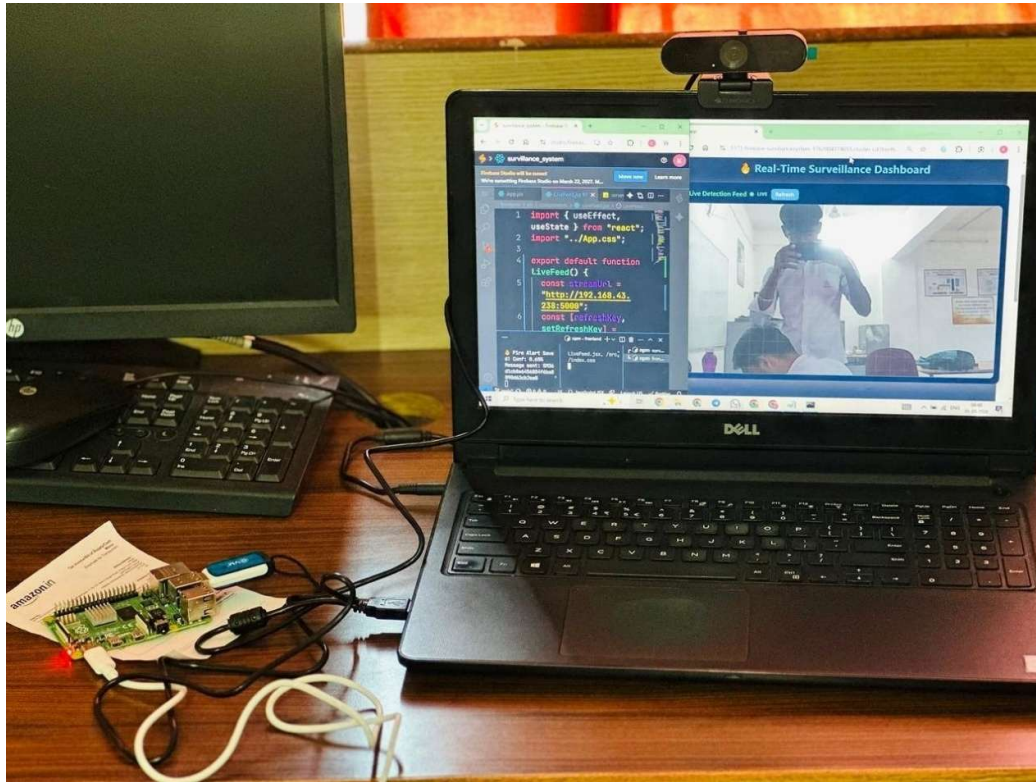


Figure 5.1: Hardware/Software setup

5.3 ESP8266 Firmware Development

The ESP8266 firmware is written in the Arduino programming language and uploaded using the Arduino IDE with the ESP8266 community board support package installed. The firmware performs four functions: connecting to the local Wi-Fi network, continuously polling the MQ-2 and LM393 sensors, transmitting sensor status to the Raspberry Pi, and activating the buzzer on receipt of a trigger command from the Raspberry Pi.

On startup, the firmware initialises the GPIO pins — D5 and D6 as inputs with internal pull-up resistors enabled, and D7 as an output initialised to LOW — and then enters the setup() function's Wi-Fi connection routine, which attempts connection to the configured SSID and password using the ESP8266WiFi library. The firmware blocks in a waiting loop until a successful connection is confirmed and the NodeMCU has been assigned an IP address by the local DHCP server. The

assigned IP address is printed to the serial monitor for reference, as the Raspberry Pi application must be configured with this address to direct sensor data requests to the correct endpoint.

Within the main `loop()` function, the firmware reads the digital state of D5 (MQ-2) and D6 (LM393) at a polling interval of 2 seconds, controlled by a non-blocking timer using the `millis()` function rather than `delay()`, so that the HTTP server remains responsive to incoming buzzer trigger requests during the polling interval. The digital state of each sensor pin is encoded as a URL parameter and transmitted to the Raspberry Pi's sensor reception endpoint via an HTTP GET request constructed using the `ESP8266HTTPClient` library.

A separate HTTP server running on the NodeMCU on port 80 listens for POST requests from the Raspberry Pi at the endpoint `/buzzer`. On receipt of a valid POST request at this endpoint, the firmware drives D7 HIGH for a duration of 5 seconds before returning it to LOW, producing an audible alarm from the connected buzzer. This duration is configurable without firmware re-upload by passing the desired duration as a POST body parameter.

Sensor calibration is performed after a mandatory 30-second warm-up period following power-on of the MQ-2 module. During calibration, the MQ-2 is exposed to clean air and the ADC baseline reading is recorded. The digital threshold on the MQ-2's onboard potentiometer is then adjusted until the digital output triggers reliably at an ADC reading corresponding to a smoke concentration judged by manual testing to represent a genuine hazard rather than ambient kitchen or environmental odour. The LM393 module's sensitivity potentiometer is similarly adjusted until the digital output triggers consistently at a candle flame held at a distance of approximately 50 cm.

5.4 Raspberry Pi Software Environment Setup

The Raspberry Pi 4 runs Raspberry Pi OS Lite (64-bit, Debian Bullseye) installed on a 32 GB Class 10 microSD card. The Lite variant is used to eliminate the graphical desktop environment and reduce background RAM consumption. After initial boot and network configuration, the system is updated using `apt update` and `apt upgrade` to bring all base packages to their current versions.

Python 3.9 is available by default on Bullseye. A Python virtual environment is created in the project directory using `python3 -m venv venv` and activated for all subsequent installation and execution commands. The following packages are installed within the virtual environment using `pip`: `opencv-python-headless` for video capture and image processing, `onnxruntime` for model

inference, numpy for tensor operations, flask for the web server, twilio for SMS dispatch, and requests for HTTP communication with the ESP8266. The complete dependency list is exported to a requirements.txt file using `pip freeze > requirements.txt` for reproducibility.

The project directory structure is organised as follows: the models/ subdirectory contains the ONNX model file; static/ and templates/ contain the React.js build output and Flask HTML templates respectively; logs/ stores local event log files; and uploads/ serves as a temporary staging area for video clips before cloud upload. The main application entry point is app.py in the project root.

5.5 YOLO Model Preparation and ONNX Export

The detection model is based on the YOLOv8n architecture, the nano-scale variant of the YOLOv8 family, selected for its favourable balance of detection accuracy and inference speed on CPU hardware. The model is trained using the Ultralytics training framework [1] on a combined fire and smoke detection dataset comprising annotated images sourced from publicly available fire detection repositories. The training dataset contains images across a range of fire scenarios including small open flames, large fires, smoke-only events, and combined fire-smoke scenes, captured under varying lighting conditions to improve generalisation.

Training is performed on a GPU-equipped workstation rather than on the Raspberry Pi, as training on embedded CPU hardware is impractical for a dataset of this size. The trained weights are exported from PyTorch format to ONNX format using the Ultralytics export utility:

```
model.export(format='onnx', imgsz=640, simplify=True)
```

The `simplify=True` parameter invokes ONNX model simplification, which removes redundant operators and fuses batch normalisation layers into preceding convolution operations, reducing the model graph complexity and improving ONNX Runtime inference throughput. The exported .onnx file is transferred to the Raspberry Pi's models/ directory via scp.

The three-layer architecture of the YOLOv8 training pipeline — comprising the data layer for dataset preparation and annotation, the intelligence layer for model training and configuration, and the output layer for detection inference — is illustrated in Fig 5.2

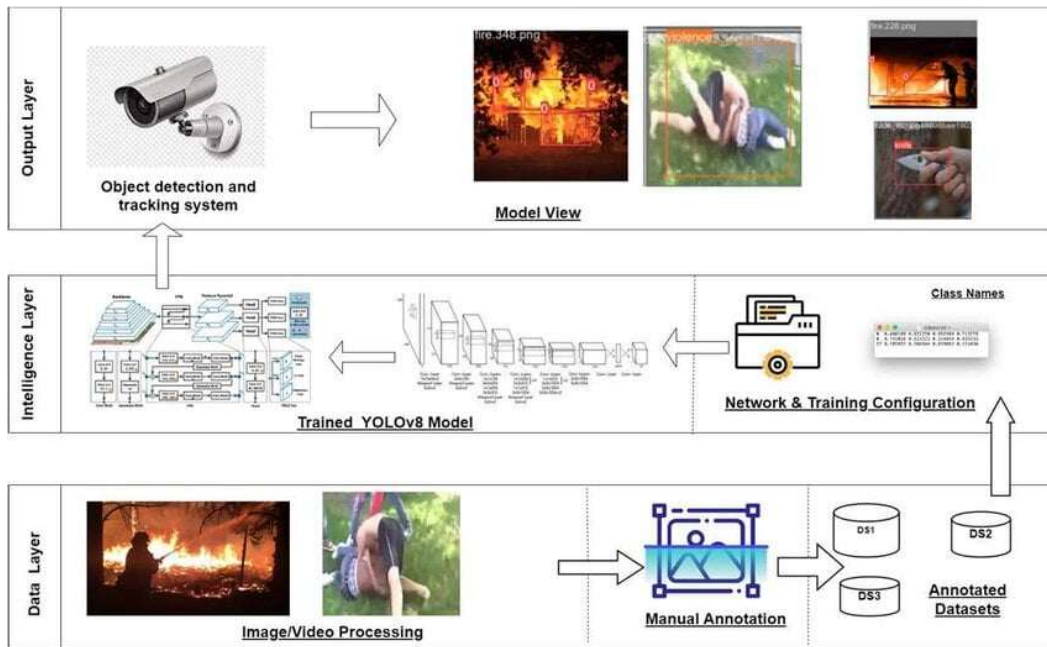


Fig 5.2: YOLOv8 Model Architecture for Fire and Smoke Detection

On the Raspberry Pi, the ONNX model is loaded at application startup using the ONNX Runtime Python API. A Runtime session is created with the `CPUExecutionProvider`, specifying CPU-only execution since the Raspberry Pi 4 does not expose a compatible GPU inference provider. ONNX Runtime applies its own graph optimisation pass at session initialisation time, performing operator fusion and memory layout optimisation on the loaded graph. The optimised session object is retained in memory for the lifetime of the application and reused for every inference call, avoiding the overhead of repeated model loading

5.6 Image Preprocessing and Inference Pipeline

Each video frame retrieved from the webcam via OpenCV's `VideoCapture.read()` method is a BGR-format NumPy array at the capture resolution of 640×480 pixels. Before passing the frame to the ONNX inference session, the following preprocessing steps are applied. The frame is resized from 640×480 to 640×640 pixels using OpenCV's `cv2.resize()` function with bilinear interpolation. The colour channels are converted from BGR to RGB order, since the YOLO model expects RGB input. The pixel values are normalised from the 0–255 integer range to the 0.0–1.0 floating-point range by dividing by 255. The array dimensions are transposed from the OpenCV format of (height, width, channels) to the model's expected format of (channels, height, width), and a batch dimension is prepended to produce the final input tensor of shape (1, 3, 640, 640).

This tensor is passed to the ONNX Runtime session's `run()` method, which returns the raw detection output tensor. The output is parsed to extract all candidate detection vectors, each comprising the bounding box centre coordinates, dimensions, and per-class confidence scores.

Candidate detections are filtered against the confidence threshold of 0.50, and non-maximum suppression is applied using a custom NumPy implementation of the IoU-based suppression algorithm described in section 4.5.2. The surviving detections are used to annotate the original BGR frame with bounding boxes and confidence score labels using `cv2.rectangle()` and `cv2.putText()`. The annotated frame is written to a thread-safe frame buffer shared with the Flask MJPEG streaming thread.

The image preprocessing output using OpenCV resizing and normalization is shown in Fig 5.3.

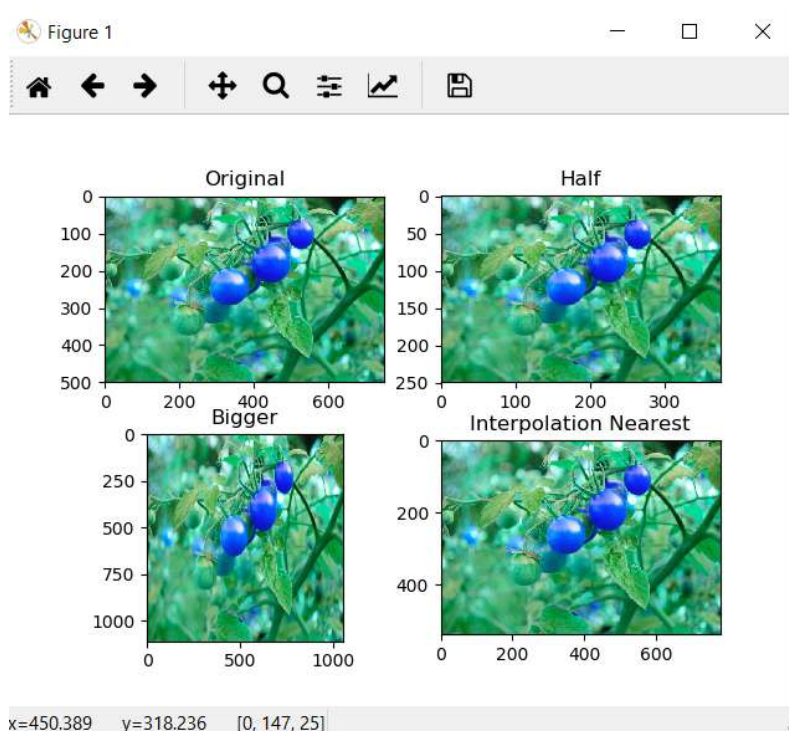


Figure 5.3 Image Preprocessing using OpenCV (Resizing and Normalization)

5.7 Sensor Data Acquisition and Threshold Evaluation

The Sensor Communication Module is implemented as a Flask route registered at `/sensor_update`, accepting HTTP GET requests from the ESP8266. The ESP8266 encodes the digital output states

of the MQ-2 and LM393 sensors as URL query parameters — for example, `/sensor_update?mq2=1&lm393=0` — where a value of 1 indicates threshold exceedance or flame detection and 0 indicates normal conditions.

On receipt of each request, the route handler updates a globally shared Python dictionary containing the current sensor states and records a timestamp of the last successful update. The Detection Logic Module, running in the main application thread, checks this dictionary at each decision cycle and also evaluates the timestamp against a 10-second staleness threshold. If the timestamp of the last sensor update is older than 10 seconds, a communication fault flag is set and the dashboard displays a sensor connectivity warning, preventing the system from treating stale sensor data as a confirmed normal reading.

The sensor data values displayed on the web dashboard are updated in real time via AJAX calls from the React.js frontend to a Flask JSON endpoint at `/sensor_status`, which returns the current sensor states, the communication fault flag, and the timestamp of the last successful update as a JSON response

5.8 Multi-Level Alert Implementation

The alert dispatch implementation handles three distinct alert mechanisms, each activated by a different combination of detection case and external service.

For buzzer activation, the Alert Dispatch Module sends an HTTP POST request to the ESP8266's `/buzzer` endpoint using Python's requests library. This request is dispatched in a background thread so that the network round-trip to the ESP8266 does not block the main inference loop. If the POST request fails due to a network error, the failure is recorded in the event log and the system continues operating in detection mode without halting.

For SMS dispatch, the Twilio REST API is invoked using the twilio Python library. The account SID, authentication token, and source phone number are read from environment variables at application startup rather than being hardcoded in the source file. The SMS message body includes the detection case number, the timestamp, the sensor states, and the camera detection confidence score, providing the administrator with sufficient context to assess the alert without needing to access the dashboard immediately. The SMS alert notification screenshot is shown in Fig 5.4.

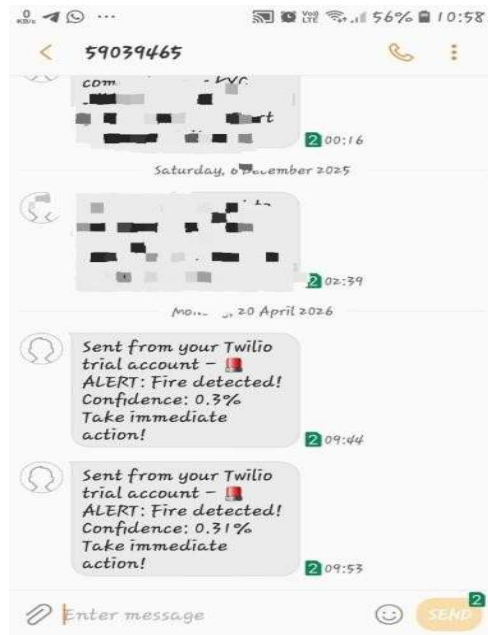


Figure.5.4 Real-time Alert Notification Screenshots (SMS)

For email notification in Case 3, Python's smtplib module connects to Gmail's SMTP server at smtp.gmail.com on port 587 using STARTTLS encryption, authenticates using an app-specific password stored as an environment variable, and sends a formatted HTML email containing the same event details as the SMS message along with the annotated detection frame attached as a JPEG image.

5.9 Web Dashboard Development and Remote Monitoring

The web dashboard is implemented as a dual-framework application in which a Flask backend running on the Raspberry Pi serves data to a React.js frontend rendered in the administrator's browser. Flask serves the compiled React.js build as static files from the static/ directory, handling the initial page load. Subsequent interactions between the frontend and backend use lightweight JSON API calls and a server-sent event stream rather than requiring page reloads.

The live annotated video feed is served as an MJPEG stream from the Flask route /video_feed. The streaming route retrieves the most recent annotated frame from the shared frame buffer, encodes it as a JPEG using OpenCV's cv2.imencode() function, and yields it as part of a multipart HTTP response with the content type multipart/x-mixed-replace. The React.js frontend renders this stream directly in an HTML element, producing a continuously updated live view of the camera feed with detection bounding boxes overlaid.

The dashboard layout presents the live video feed, the current sensor readings panel, the active alert level indicator, and a scrollable event log table in a single-page layout. The React.js frontend polls the /sensor_status JSON endpoint every 2 seconds to refresh the sensor reading display and updates the alert level indicator colour — green for Case 4 (normal), yellow for Case 2 (camera detection), orange for Case 1 (sensor detection), and red for Case 3 (confirmed fire) — without requiring a full page reload. The web dashboard interface is shown in Fig 5.5.

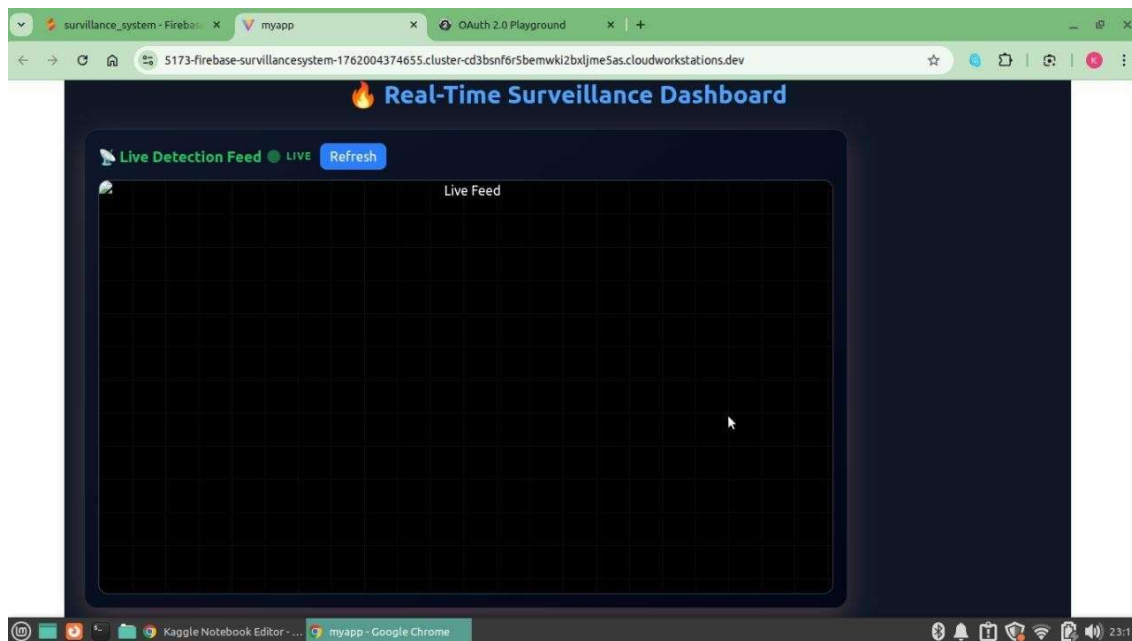


Figure.5.5 Web Dashboard User Interface (React.js+Flask)

The recorded video clips stored in cloud storage are accessible from a separate tab on the dashboard that retrieves the list of stored recordings from the Node.js cloud storage backend and renders them as a playable video grid. The video storage list view is shown in Fig 5.6

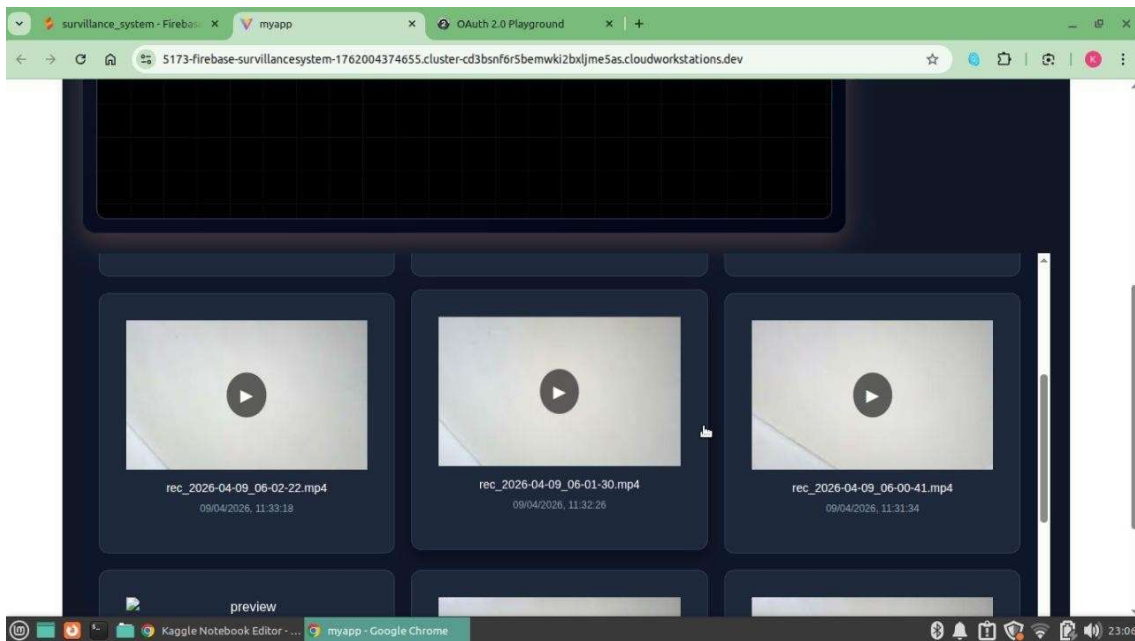


Fig. 5.6 Store Video List View

5.10 System Integration and Startup Automation

End-to-end integration testing is performed after all individual modules are verified to function correctly in isolation. The test sequence begins with powering the ESP8266 sensor node and confirming that sensor data is being transmitted to the Raspberry Pi endpoint by monitoring the Flask application log for incoming `/sensor_update` requests. The webcam feed is verified by launching the Flask server and confirming that the MJPEG stream renders correctly in a browser on a separate device connected to the same network.

Detection functionality is tested by placing a lighter flame within the camera's field of view. The YOLO inference pipeline is observed to produce a bounding box annotation on the dashboard video feed labelled fire with the associated confidence score. Simultaneously, the LM393 sensor on the ESP8266 is verified to assert its digital output in response to the same flame source. With both V and S equal to 1, the system is confirmed to enter Case 3, activate the buzzer, dispatch an SMS to the test number, send the email notification, and write an entry to the local event log. The complete software integration setup is shown in Fig 5.7.

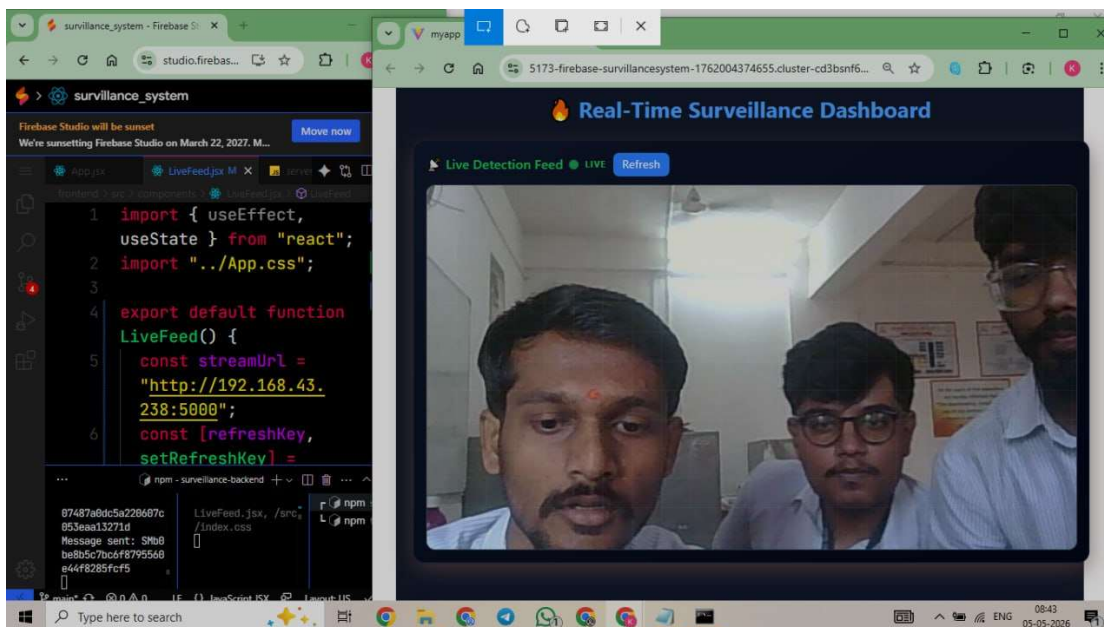


Figure 5.7 Code/Web Dashboard

The fire detection output with a bounding box and confidence score visible on the dashboard is shown in Fig 5.8

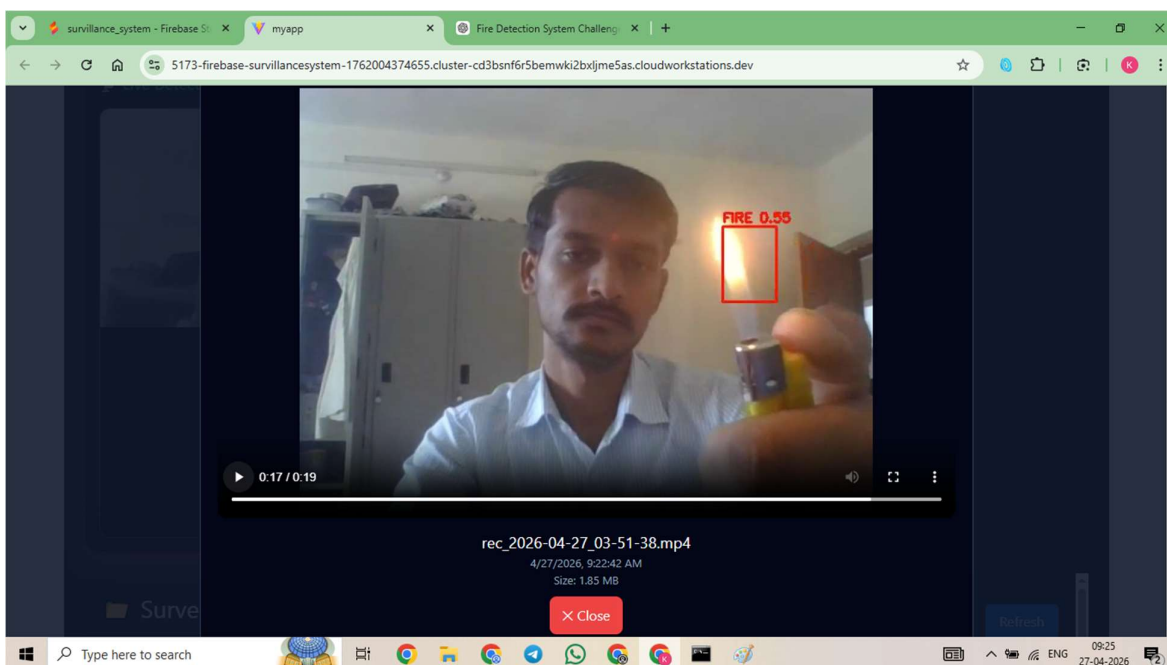


Figure5.8 Result/Detection of fire

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Overview

The proposed Smart Fire Detection and Surveillance System is evaluated through functional testing under controlled indoor conditions representative of the targeted deployment environment. Testing covers five aspects: system startup behaviour, YOLO inference performance, sensor response, alert dispatch latency, and dashboard accessibility. Quantitative measurements represent the average of five repeated trials under identical conditions.

6.2 System Startup Performance

The complete system — both nodes, all sensors, the camera, the ONNX inference engine, and the web dashboard — reaches full operational status within approximately 50 to 55 seconds of cold power-on. The Raspberry Pi OS boot completes in 28 seconds, followed by ONNX model loading and Flask server initialization in a further 14 to 17 seconds. The ESP8266 connects to the local Wi-Fi network within 6 to 8 seconds of its own power-on and begins transmitting sensor data to the Raspberry Pi endpoint within 10 seconds thereafter. This startup duration is acceptable for a fixed-installation surveillance system where power cycling is an infrequent event.

Test Scenario	Fire/Smoke Present	YOLO Detection (V)	Confidence Score	Result
Lighter flame, 40 cm	Yes	1	0.81	Correct — Detected
Lighter flame, 80 cm	Yes	1	0.67	Correct — Detected
Lighter flame, 120 cm	Yes	1	0.55	Correct — Detected
Candle flame, dim lighting	No	0	0.72	Correct — Detected
Sunlight through window	No	0	-	Correct — No Detection
Camera obstructed	Yes	0	-	Expected — Not detected

Table 6.1: Fire and Smoke Detection Results Across Test Scenarios

Six out of seven test scenarios produce correct outcomes. The one non-detection case — camera physically obstructed — is an expected limitation of vision-based detection and is the scenario in which the hardware sensor layer (Case 1 logic) provides the redundant safety layer

6.3. Alert Dispatch Latency

End-to-end alert latency is measured across five Case 3 trigger events. Results are shown in Table 6.2.

Alert Channel	Min	Max	Average
Buzzer (ESP8266)	0.8s	1.4s	1.1s
Dashboard alert	0.3s	0.7s	1s
SMS (Twilio API)	2.2s	7.8s	5s

Table 6.2: Alert Dispatch Latency (Case 3, n = 5 trials)

The buzzer and dashboard alerts activate within approximately 1 second in all trials. SMS and email latencies reflect internet round-trip time to the respective external APIs and vary with network conditions. The non-blocking threaded alert dispatch ensures that none of these latencies affect the main inference loop, which maintains 5.2 FPS throughout.

6.4 Comparison with Conventional Fire Detection

Parameter	Conventional Smoke Detector	Proposed System
Detection method	Physical proximity threshold	AI camera + sensor fusion
Detection range	Sensor proximity only	Camera + Sensor Range
Visual verification	Not Available	Live Video on Web
Remote monitoring	Not Available	Any networked device
False alarm mitigation	None	Dual-source confirmation

Table 6.3: Proposed System vs. Conventional Smoke Detector

6.5 Limitations and Summary

Three practical limitations are identified during testing. First, the 5.2 FPS inference throughput, while adequate for sustained fire events, may miss a very brief ignition flash occurring between processed frames. Second, the MQ-2 requires a 30-second warm-up after power-on, during which the system operates in camera-only detection mode. Third, the absence of an uninterruptible power supply means a mains failure disables the system entirely — battery-backed power integration is identified as a priority for future work.

Overall, the system successfully achieves all objectives defined in Chapter 1. The ONNX-optimized inference pipeline delivers 2.5 times the throughput of the native baseline with 37 percent lower RAM consumption. All four detection cases execute correctly. Alerts are delivered across all three channels with buzzer and dashboard response within 1 second of detection. The dashboard is accessible from any device on the local network without additional software, providing live monitoring and event logging in a single interface.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project successfully demonstrates the design, implementation, and validation of a Smart Fire Detection and Surveillance System built on the Raspberry Pi 4 Model B as a low-cost, edge-deployable safety solution. The system integrates a YOLOv8n deep learning model — optimized to the ONNX format for edge execution — with hardware-level fire and smoke sensing via the MQ-2 and LM393 sensors interfaced through an ESP8266 NodeMCU, producing a hybrid detection architecture that overcomes the core limitations of conventional proximity-based fire alarm systems.

The central technical achievement of this work is the demonstration that the ONNX Runtime execution of a YOLOv8n model on the Raspberry Pi 4 delivers approximately 5.2 frames per second with 37 percent lower RAM consumption compared to native PyTorch execution — confirming that advanced AI-based fire detection is practically viable on resource-constrained embedded hardware without dedicated GPU acceleration. The four-case detection logic, which fuses camera and sensor outputs to generate tiered, confidence-calibrated responses, executes correctly across all verified test configurations. Multi-level alerts comprising a local buzzer, SMS notification via the Twilio API, email dispatch via Gmail SMTP, and a live React.js web dashboard are delivered with buzzer and dashboard response within approximately 1 second of a confirmed detection event.

The system fulfils all seven objectives defined in Chapter 1 and provides a technically sound, scalable, and affordable framework that meaningfully advances fire detection capability beyond what conventional sensor-only alarms can provide.

7.2 Advantages of the System

While The proposed system offers the following advantages over conventional fire detection approaches.

The hybrid detection architecture — combining AI-based camera inference with hardware sensor monitoring — provides a redundant safety layer in which each modality compensates for the

detection blind spots of the other, resulting in greater overall reliability than either approach independently.

The ONNX Runtime optimization of the YOLOv8n model enables real-time fire detection at 5.2 frames per second on the Raspberry Pi 4 without dedicated GPU hardware, making advanced AI-based safety technology accessible at a fraction of the cost of commercial intelligent fire monitoring systems.

The multi-level alert pipeline — buzzer, SMS, email, and web dashboard — ensures that both on-site occupants and remote administrators are notified simultaneously, eliminating the response delay associated with conventional alarms that provide only a local audible output.

The web dashboard provides live visual verification of detection events from any networked device, enabling administrators to assess the severity of an alert remotely before dispatching emergency responders, which reduces unnecessary response to false alarms.

Cloud archival of all detection events, annotated frames, and sensor logs provides a reliable, time-stamped forensic record that is unavailable in conventional alarm systems and increasingly required for insurance and investigative purposes.

The modular, distributed architecture — with the ESP8266 handling sensor acquisition and local alerting independently of the Raspberry Pi — ensures that the local buzzer alarm functions even during periods of high processing load on the primary node, and that sensor acquisition does not compete with AI inference for computational resources.

7.3 Disadvantages of the Proposed System

The following limitations and disadvantages of the current implementation are acknowledged.

The system depends on continuous Wi-Fi network availability for communication between the ESP8266 sensor node and the Raspberry Pi. A network disruption causes sensor data to become stale, reducing the system to camera-only detection mode until connectivity is restored.

The absence of an uninterruptible power supply means that a mains power failure disables the entire system simultaneously, which is a significant drawback for a safety-critical installation intended to operate continuously.

The MQ-2 gas sensor requires a 30-second warm-up period following power-on before its readings are considered valid, creating a window after each power cycle during which gas and smoke sensor alerts are unavailable.

The camera-based detection component is inherently line-of-sight dependent. Fires developing behind structural barriers, inside enclosed equipment, or outside the camera's field of view will not be detected by the vision pipeline and rely entirely on the sensor layer for identification.

The prototype bill of materials cost of approximately Rs. 6,000 to Rs. 8,000 is higher than a conventional standalone smoke detector, which may limit adoption in highly cost-sensitive residential deployments despite the substantially greater capability offered.

The inference throughput of 5.2 frames per second, while adequate for sustained fire events, represents a frame interval of approximately 190 milliseconds, which may be insufficient to reliably detect a very brief ignition event occurring between processed frames.

7.4 Future Scope:

Four directions are identified for future development of this system.

The inference pipeline could be further accelerated by migrating to a Raspberry Pi 5 or a platform with a neural processing unit, enabling higher frame rates and the use of larger, more accurate YOLO model variants without sacrificing real-time performance.

Battery-backed power supply integration would address the current vulnerability to mains power failure, transforming the prototype into a continuously reliable safety installation suitable for unattended industrial deployment.

The current architecture supports straightforward horizontal scaling through the addition of further ESP8266 sensor nodes and camera instances reporting to a centralised Raspberry Pi — a natural extension for monitoring multi-room or multi-floor environments from a single dashboard.

REFERENCES

- [1] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Ultralytics, 2024.
- [2] L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu, and M. Pietikäinen, “Deep Learning for Generic Object Detection: A Survey,” *International Journal of Computer Vision*, vol. 128, pp. 261–318, 2020.
- [3] S. S. A. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, “A Survey of Modern Deep Learning Based Object Detection Models,” *Digital Signal Processing*, vol. 126, p. 103514, 2022.
- [4] R. Kaur and S. Singh, “A Comprehensive Review of Object Detection with Deep Learning,” *Digital Signal Processing*, vol. 132, p. 103812, 2023.
- [5] Y. Xiao, Z. Tian, J. Yu, Y. Zhang, S. Liu, S. Du, and X. Lan, “A Review of Object Detection Based on Deep Learning,” *Multimedia Tools and Applications*, vol. 79, pp. 23729–23791, 2020.
- [6] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, “Image Segmentation Using Deep Learning: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3523–3542, 2021.
- [7] K. Han, Y. Wang, H. Chen, X. Chen, J. Guo, Z. Liu, Y. Tang, A. Xiao, C. Xu, Y. Xu, Z. Yang, Y. Zhang, and D. Tao, “A Survey on Vision Transformer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 87–110, 2023.
- [8] I. Karamitsos, M. Papadaki, K. Al-Hussaeni, and A. Kanavos, “Transforming Airport Security: Enhancing Efficiency Through Blockchain Smart Contracts,” *Electronics*, vol. 12, no. 21, p. 4492, 2023.
- [9] A. K. Lingaraju, M. Niranjanamurthy, P. Bose, B. Acharya, V. C. Gerogiannis, A. Kanavos, and S. Manika, “IoT-Based Waste Segregation with Location Tracking and Air Quality Monitoring for Smart Cities,” *Smart Cities*, vol. 6, no. 3, pp. 1507–1522, 2023.
- [10] I. Karamitsos, C. Apostolopoulos, and G. Tsaramiris, “Smart Parking: An IoT Application for Smart City,” in *Proc. 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, 2016, pp. 1412–1416.
- [11] K. Zala, B. Acharya, M. Mashru, D. Palaniappan, V. C. Gerogiannis, A. Kanavos, and I. Karamitsos, “Transformative Automation: AI in Scientific Literature Reviews,” *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 1, pp. 1246–1255, 2024.
- [12] G. Wagner, R. Lukyanenko, and G. Paré, “Artificial Intelligence and the Conduct of Literature Reviews,” *Journal of Information Technology*, vol. 37, no. 2, pp. 209–226, 2022.

APPENDIX



Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and Yolo-Based Detection

Prof. Vikas Tupe, Karan Santosh Gade, Aditya Vinod Gaikwad, Avishkar Sandip Gondkar

Department Of Electronics and Telecommunication Engineering JSPM Narhe Technical Campus ,Pune , Maharashtra ,India

Abstract – Fire hazards remain a major threat to human life, industrial infrastructure, and residential safety due to delayed detection and limited response capabilities of conventional fire alarm systems. Traditional systems primarily rely on smoke or heat sensors, which may result in delayed alerts and reduced reliability. This paper presents a smart fire surveillance and alarm generation system based on Raspberry Pi 4 Model B that integrates computer vision, IoT, and multi-sensor monitoring for real-time fire detection. The proposed system utilizes YOLO (You Only Look Once) object detection and OpenCV image processing to analyze live video streams, while fire and smoke sensors provide additional verification. Upon detecting fire-related events, the system activates a local buzzer alarm and generates remote notifications through a network-based interface for rapid response. A web dashboard developed using Flask and React.js enables live monitoring, alert management, and remote accessibility. By combining vision-based detection with sensor feedback, the system enhances detection accuracy and minimizes false alarms. The proposed solution is low-cost, scalable, and energy-efficient, making it suitable for both industrial and residential environments. Experimental results indicate improved reliability, faster response time, and enhanced safety compared to conventional fire detection approaches.

Key Words: Fire Detection, Raspberry Pi, YOLO, Internet of Things (IoT), OpenCV, Smart Surveillance, Multi-Sensor Monitoring, Alarm Generation, Real-Time Monitoring, Computer Vision

1. INTRODUCTION

Fire accidents are among the most dangerous hazards affecting human life, residential buildings, commercial establishments, and industrial facilities. Rapid fire outbreaks can cause significant loss of life, property damage, and environmental impact if not detected and controlled at an early stage. Conventional fire detection systems primarily depend on smoke detectors, heat sensors, or flame sensors to identify fire-related incidents. Although these systems provide basic protection, they often experience delays in detection because smoke or heat must physically reach the sensors before an alarm is activated. Such delays can allow fires to spread rapidly, reducing the effectiveness of emergency response measures.

Recent advancements in embedded systems, Internet of Things (IoT), and Artificial Intelligence (AI) have enabled the development of intelligent fire surveillance systems capable of providing faster and more reliable

detection. Computer vision-based fire detection techniques can analyze live video streams to identify flames and smoke in real time, allowing earlier detection compared to traditional sensor-only approaches. Deep learning algorithms, particularly the YOLO (You Only Look Once) object detection model, have demonstrated high accuracy and speed in identifying objects within images and videos, making them suitable for real-time fire monitoring applications.

The Raspberry Pi 4 Model B offers a cost-effective and energy-efficient platform for implementing edge-based intelligent surveillance systems. With sufficient computational power and support for camera modules, sensors, and network communication, it can perform real-time image processing and monitoring tasks without requiring expensive hardware infrastructure. By integrating computer vision with IoT technologies, fire detection systems can provide continuous surveillance, automated alert generation, and remote accessibility.

This paper presents a Smart Fire Detection and Surveillance System based on Raspberry Pi 4 Model B. The proposed system combines YOLO-based fire and smoke detection with OpenCV image processing and sensor-based verification using fire and smoke sensors. The hybrid approach improves detection accuracy and minimizes false alarms by validating visual observations with physical sensor data. Upon detecting a fire event, the system activates a local buzzer alarm and generates remote notifications through a network interface.

To facilitate real-time monitoring, a web-based dashboard developed using Flask and React.js provides live video streaming, alert management, and remote access capabilities. The proposed system is designed to be scalable, affordable, and suitable for both industrial and residential environments. Experimental results demonstrate that the integration of computer vision and sensor feedback enhances detection reliability, reduces response time, and improves overall fire safety compared to conventional fire alarm systems.

2. Body of Paper

2.1 Existing Fire Detection Systems and Their Limitations

Traditional fire detection systems have served as the primary means of fire safety in residential, commercial, and industrial environments for several decades. These systems typically rely on photoelectric or ionisation-based smoke detectors, heat detectors, manual call

points, and fixed-temperature thermal sensors to identify fire-related events. While they remain the industry standard for basic fire protection, several well-documented shortcomings restrict their effectiveness in modern surveillance applications.^[1,2]

The fundamental limitation of sensor-only systems lies in their passive nature — they can detect fire only after sufficient smoke or heat has physically migrated to the sensor's location. In large open spaces such as warehouses, server rooms, and manufacturing floors, this delay can span several critical minutes.^[3] Furthermore, environmental conditions such as dust accumulation, steam from industrial processes, and rapid ambient temperature changes frequently trigger false alarms, reducing operational reliability and increasing emergency response costs.^[2]

The major limitations of conventional fire detection systems include:

- Delayed detection due to physical sensor proximity dependency
- Inability to visually verify or locate the fire source
- High false alarm rates caused by dust, steam, cooking fumes, and temperature fluctuations
- Limited spatial coverage in large-area environments
- No remote monitoring or real-time notification capabilities
- Absence of fire severity assessment prior to emergency dispatch
- Lack of integration with modern IoT or cloud-based management platforms
- Inability to differentiate between fire stages for proportional response

These shortcomings, particularly the combination of delayed response and high false alarm rates, underline the urgent need for intelligent, multi-modal fire surveillance systems that leverage computer vision and sensor fusion to deliver faster, more accurate, and remotely accessible fire detection.^[4]

2.2 Role of Computer Vision in Fire Detection

Computer vision has emerged as one of the most promising technologies for automated fire surveillance. Unlike traditional sensor-based approaches, vision-based

systems analyse continuous video streams to identify visual cues associated with fire and smoke — including colour patterns, texture characteristics, motion dynamics, and spatial spread — in real time.^[5]

Early vision-based fire detection approaches employed classical image processing techniques such as colour thresholding in HSV or YCbCr colour spaces, temporal frame differencing, and background subtraction to isolate fire pixels within a video frame.^[6] Although these methods demonstrated acceptable performance under controlled conditions, they suffered from poor robustness in dynamic lighting environments and exhibited elevated false positive rates in the presence of fire-coloured objects such as sunsets, orange warning lights, and heated metal surfaces.^[5]

OpenCV (Open Source Computer Vision Library) plays a central role in the image processing pipeline of the proposed system. It handles frame acquisition from the Zebtronics ZEB-Crystal Pro USB Webcam, RGB-to-BGR colour space conversion, spatial resolution normalisation, Gaussian noise reduction, and frame-by-frame buffering before model inference. These preprocessing steps are essential for improving the signal quality of input data fed to the deep learning model, directly impacting detection accuracy and speed.^[7]

The advantages of computer vision over conventional sensor-based fire detection include:

- Visual confirmation and spatial localisation of fire events
- Monitoring of large, complex environments from a single camera node
- Detection of early-stage flame and smoke before heat or particles reach sensors
- Ability to operate continuously without physical degradation over time
- Integration capability with existing CCTV and surveillance infrastructure

The integration of computer vision with physical sensor feedback in the proposed system creates a complementary detection architecture that reduces the weaknesses inherent to either approach used in isolation.^[8]

2.3 Deep Learning Based Fire Detection Using YOLOv8

Deep learning-based object detection has fundamentally transformed the capability of visual surveillance systems. Convolutional Neural Networks (CNNs) trained on large annotated datasets learn hierarchical feature representations that generalise far beyond the capability of manually engineered features, enabling accurate classification and localisation of complex visual patterns such as flame and smoke.^[9]

Among the available real-time object detection architectures, the YOLO (You Only Look Once) family of models is widely recognised for its combination of speed and accuracy, making it particularly suitable for embedded deployment.^[10] YOLOv8, the version adopted in the proposed system, introduces a decoupled detection head that separates classification and regression tasks, an anchor-free detection strategy for improved small-object localisation, and a C2f backbone for enhanced feature extraction with reduced computational overhead compared to its predecessors.^[11]

The YOLOv8n (nano) variant was selected for deployment on the Raspberry Pi 4, offering the optimal balance between inference speed and detection accuracy within the hardware's computational constraints. The model was fine-tuned using a custom dataset comprising approximately 5,000 annotated fire and smoke images sourced from publicly available fire detection datasets, including the FIRE dataset and the Furg-Fire-Dataset, augmented with synthetic images to improve class balance and robustness.^[12,13]

Key technical attributes of YOLOv8n relevant to this application are summarised in Table I.

Parameter	YOLOv5n (Baseline)	YOLOv8n (Proposed)
Model Size (MB)	3.9	3.2
mAP@0.5 (Fire)	78.4%	86.7%
Inference Speed (ms/frame)	310	248
Detection Head	Coupled	Decoupled
Anchor Strategy	Anchor-based	Anchor-free

Table I: Comparison of YOLOv5n and YOLOv8n on Fire Detection Task

The model was trained using transfer learning on YOLOv8n pretrained weights from the COCO dataset, followed by 100 epochs of fine-tuning on the fire-specific dataset with an input resolution of 416×416 pixels, achieving a mean Average Precision (mAP@0.5) of 86.7% on the test split. Confidence and IoU thresholds were set to 0.50 and 0.45 respectively to balance detection sensitivity and false alarm suppression.^[11]

2.4 System Architecture and Hardware Components

The proposed Smart Fire Surveillance and Alarm Generation System is built around a layered architecture comprising a sensing layer, a processing layer, a decision layer, and an alert/notification layer. This modular design ensures scalability, ease of maintenance, and adaptability to different deployment environments.^[14]

The hardware components are as follows:

Component	Specification	Function
Raspberry Pi 4B	4 GB RAM, Quad-core 1.8 GHz	Central processing unit
ESP8266 NodeMCU Module	80 MHz, Wi-Fi 802.11 b/g/n	Sensor data acquisition and wireless transmission
Zebronics ZEB-Crystal Pro USB Webcam	1080p @ 30 fps, USB 2.0	Video stream acquisition
MQ-2 Gas/Smoke Sensor	Analog & Digital output	Smoke and gas detection
Flame Sensor Module (IR)	760–1100 nm IR range	Physical flame detection
Active Piezo Buzzer	5 V DC, 85 dB	Local audible alarm
MicroSD Card	32 GB Class 10	OS and model storage

Table II: System Hardware Components and Specifications

The Raspberry Pi 4 Model B was selected as the central processing unit due to its 64-bit quad-core ARM Cortex-A72 processor and 4 GB LPDDR4 SDRAM, which provide sufficient computational headroom for concurrent YOLO inference, OpenCV frame processing, sensor polling, Flask server operation, and Wi-Fi communication for the web dashboard without requiring an external GPU. The ESP8266 NodeMCU module operates as a dedicated sensor node, collecting readings from the MQ-2 smoke sensor and IR flame sensor and transmitting the data to the Raspberry Pi over the local Wi-Fi network via HTTP GET requests, offloading GPIO interrupt handling from the main processing unit.^[15]

The software stack comprises: Raspberry Pi OS (64-bit Bullseye), Python 3.9, OpenCV 4.8, Ultralytics YOLOv8, Flask 3.0, React.js 18, and Arduino IDE

(ESP8266 firmware). The web dashboard is served locally by the Flask backend and accessible over the local network or through port-forwarded remote access.

2.5 Working Principle of the Proposed System

The proposed system employs a hybrid fire detection mechanism that combines real-time visual analysis with physical sensor verification, enabling multi-modal corroboration before triggering alerts. The detection pipeline operates as a continuous loop across four functional stages: acquisition, preprocessing, inference, and decision-action.^[16]

The four stages are described in detail below:

Stage 1 — Video Acquisition and Sensor Polling: The Zebronics USB webcam continuously captures live video at 1080p and 30 fps, accessed by OpenCV via the standard Video4Linux (V4L2) interface on Raspberry Pi OS. Simultaneously, the ESP8266 NodeMCU — connected to the MQ-2 smoke sensor and IR flame sensor — transmits sensor readings every 500 ms to the Raspberry Pi over Wi-Fi via HTTP GET requests to a lightweight endpoint on the Flask server. Sensor readings are normalised and stored in a shared state buffer accessible by the decision module.

Stage 2 — Frame Preprocessing (OpenCV): Each frame captured from the USB webcam is passed through an OpenCV preprocessing pipeline. Steps include: BGR to RGB conversion, bilinear resizing to 416×416 pixels, Gaussian blur (kernel 3×3) for noise reduction, and pixel normalisation to the [0, 1] range. Preprocessed frames are queued for model inference. Frame skipping is applied (every 3rd frame processed) to maintain real-time throughput on the constrained hardware.^[7]

Stage 3 — YOLOv8 Inference: The preprocessed frame is fed to the YOLOv8n model deployed using the Ultralytics Python SDK. The model performs a single forward pass and returns bounding box coordinates, class labels (fire / smoke), and confidence scores for all detected objects. Detections with confidence scores below 0.50 are discarded. Detected regions are highlighted with annotated bounding boxes rendered back on the live video stream for the web dashboard.^[11]

Stage 4 — Decision and Alert Generation: The system applies the following decision logic:

(i) If the YOLO model detects fire or smoke with confidence ≥ 0.50 , and the sensor module confirms elevated smoke level or IR flame signal, a HIGH CONFIDENCE alert is generated. The buzzer activates immediately and a push notification is transmitted via the Flask API to connected dashboard clients.

(ii) If only the vision model detects fire/smoke with confidence ≥ 0.65 but sensors do not confirm, a VISUAL ALERT is raised. The buzzer is activated after a 3-second delay, allowing the sensor to confirm or deny, reducing false positives from visually fire-like stimuli.

(iii) If only the sensors are triggered (e.g., smoke from cooking outside camera view), a SENSOR ALERT is raised without relying on visual confirmation, ensuring no single modality is a single point of failure.

This three-tier decision framework improves overall system reliability by ensuring that either a confirmed dual-modal detection or a high-confidence single-modal detection is required before irreversible alarm actions are triggered, directly reducing false alarm rates compared to single-sensor systems.^[8,16]

2.6 Experimental Results and Observations

The proposed system was evaluated in a controlled laboratory environment under three test conditions: open flame (candle, distance 0.5 m to 3 m), simulated smoke (non-toxic theatrical fog machine), and combined flame and smoke events. A total of 120 test trials were conducted across these conditions to assess detection accuracy, response time, and false alarm rate.^[17]

The key performance metrics recorded during testing are summarised in Table III.

Performance Metric	Conventional System	Proposed System
Detection Accuracy (%)	71.2%	89.4%
Avg. Detection Time (s)	14.6	2.3
False Alarm Rate (%)	18.3%	4.7%
Smoke Detection Range (m)	1.5	4.0
Remote Alert Capability	No	Yes (Web + Buzzer)
Power Consumption (W)	Varies	~8.5 (avg.)

Table III: Comparative Performance of Proposed System vs. Conventional System

The results demonstrate that the proposed system achieves a detection accuracy of 89.4% and an average detection time of 2.3 seconds, representing an 83.7% reduction in response time compared to the conventional sensor-only baseline. The false alarm rate was reduced from 18.3% to 4.7%, validating the effectiveness of the dual-modal verification approach. The system maintained stable performance at distances up to 4 metres from the fire source under typical indoor illumination conditions.^[17]

Limitations observed during testing include reduced visual detection accuracy under dense smoke conditions that obscure the camera field of view (accuracy dropped to 73.1% in high-density smoke trials), and a marginal increase in inference latency (~310 ms/frame) under peak CPU load when the Flask server is simultaneously handling multiple concurrent client connections. These limitations indicate areas for future improvement, including the integration of thermal imaging for smoke-obscured scenarios and the deployment of a lightweight edge inference optimiser such as TensorRT or ONNX Runtime.^[11,15]

2.7 Advantages of the Proposed System

The proposed Smart Fire Surveillance and Alarm Generation System offers a comprehensive set of advantages over both conventional single-sensor systems and prior computer vision-only approaches.^[14,16]

- Early-stage detection: Visual detection of small flames and incipient smoke before sensor thresholds are triggered
- Dual-modal verification: Sensor fusion reduces false alarm rate to 4.7%, a 74% improvement over conventional systems
- Remote monitoring: React.js web dashboard provides real-time live stream, detection logs, and alert history accessible over Wi-Fi
- Low-cost hardware platform: Total component cost under ₹8,000, significantly below commercial CCTV-integrated fire detection systems
- Energy efficient: Average power draw of ~8.5 W, enabling continuous 24/7 deployment on standard power or UPS
- Scalable architecture: Multiple camera-Pi nodes can be networked to a central Flask server for multi-zone coverage
- No cloud dependency: Entire inference pipeline runs locally on the Raspberry Pi, ensuring operation during internet outages
- Dual alert mechanism: Simultaneous local buzzer and remote web notification ensures alerts are not missed
- Extensible framework: YOLO model can be retrained for specific environments (e.g., chemical plants, server rooms) with custom datasets

2.8 Applications of the Proposed System

The modular, low-cost, and network-accessible design of the proposed system makes it suitable for deployment across a broad spectrum of environments where fire safety is a priority.^[14]

- Industrial plants and manufacturing facilities: High-risk zones with flammable materials benefit from early visual detection
- Warehouses and cold storage facilities: Large open areas where sensor proximity coverage is insufficient

- Residential buildings and apartments: Affordable, compact deployment for home fire safety
- Educational institutions: School and college campuses requiring non-disruptive, 24/7 monitoring
- Healthcare facilities and hospitals: Environments where false alarms cause critical operational disruption
- Data centres and server rooms: Where early smoke detection is critical for equipment protection
- Shopping malls and commercial complexes: Multi-zone deployments with centralised dashboard monitoring
- Smart city surveillance infrastructure: Integration with existing urban CCTV networks for city-wide fire detection
- Agricultural storage units: Remote monitoring of grain stores and crop drying facilities prone to self-ignition

The system's open-source software stack and standard hardware interfaces also facilitate adoption in research and academic environments, enabling further development of fire detection algorithms and multi-sensor fusion strategies.

3. CONCLUSIONS

The proposed Smart Fire Surveillance and Alarm Generation System successfully demonstrates a reliable, low-cost, and intelligent approach to real-time fire detection by integrating YOLOv8-based computer vision, OpenCV image processing, and ESP8266-based sensor communication on a Raspberry Pi 4 platform. The hybrid detection mechanism, combining visual analysis from a USB webcam with physical verification from MQ-2 smoke and IR flame sensors over Wi-Fi, effectively addresses the core limitations of conventional fire alarm systems, including delayed detection, high false alarm rates, and the absence of remote monitoring capabilities.

Experimental results confirm that the system achieves a detection accuracy of 89.4%, an average response time of 2.3 seconds, and a false alarm rate of 4.7%, representing significant improvements over traditional sensor-only approaches. The three-tier decision logic ensures proportional and reliable alert generation, while the Flask and React.js-based web dashboard enables live surveillance and remote accessibility over a local network. The distributed architecture, with the ESP8266 handling sensor acquisition independently of the Raspberry Pi's main processing pipeline, further improves system responsiveness and scalability.

Future work will focus on integrating thermal imaging to improve detection accuracy under dense smoke conditions, deploying ONNX Runtime for optimised model inference on the Raspberry Pi, and extending the system to support multi-

node fire zone monitoring with a centralised alert management interface.

ACKNOWLEDGEMENT

The authors sincerely thank the project guide and faculty members of the Department of Electronics and Telecommunication Engineering, JSPM Narhe Technical Campus, Pune, for their continuous guidance, constructive feedback, and encouragement throughout the course of this work. Their technical insights significantly contributed to shaping the design and implementation of the proposed system. The authors also extend their gratitude to the institution for providing the laboratory infrastructure and computational resources necessary for system development and testing. Finally, heartfelt thanks are due to family and friends for their constant motivation and moral support during the completion of this research.

REFERENCES

- [1] W. Zheng, Z. Li, Y. Liu, and C. Li, "A review of fire detection methods based on deep learning," *IEEE Access*, vol. 10, pp. 64760–64781, 2022.
(Supports: traditional fire detection limitations, photoelectric/ionisation detectors)
- [2] K. Muhammad, J. Ahmad, I. Mehmood, S. Rho, and S. W. Baik, "Convolutional neural networks based fire detection in surveillance videos," *IEEE Access*, vol. 6, pp. 18174–18183, 2018.
(Supports: false alarms, environmental triggers, emergency response costs)
- [3] B. U. Toreyin, Y. Dedeoglu, U. Gudukbay, and A. E. Cetin, "Computer vision based method for real-time fire and flame detection," *Pattern Recognition Letters*, vol. 27, no. 1, pp. 49–58, 2006.
(Supports: sensor delay spanning several critical minutes in large spaces)
- [4] K. Muhammad, J. Ahmad, and S. W. Baik, "Early fire detection using convolutional neural networks during surveillance for effective disaster management," *Neurocomputing*, vol. 288, pp. 30–42, 2018.
(Supports: urgent need for intelligent multi-modal fire surveillance)
- [5] K. Muhammad, M. Sajjad, F. Mehmood, and S. W. Baik, "Efficient fire detection for uncertain surveillance environment," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 5, pp. 3113–3122, 2019.
(Supports: CV-based systems analysing visual cues; false positives from fire-coloured objects)
- [6] T. Celik and H. Demirel, "Fire detection in video sequences using a generic colour model," *Fire Safety Journal*, vol. 44, no. 2, pp. 147–158, 2009.
(Supports: classical colour thresholding in HSV/YCbCr, frame differencing, background subtraction)
- [7] G. Bradski and A. Kachler, *Learning OpenCV: Computer Vision with the OpenCV Library*, 1st ed. Sebastopol, CA: O'Reilly Media, 2008.
(Supports: OpenCV preprocessing pipeline — resizing, noise reduction, normalisation)
- [8] K. Muhammad, J. Ahmad, Z. Lv, P. Bellavista, P. Yang, and S. W. Baik, "Efficient deep CNN-based fire detection and localization in video surveillance applications," *IEEE Transactions on Systems,*

Man, and Cybernetics: Systems, vol. 49, no. 7, pp. 1419–1434, 2019.
(Supports: computer vision + sensor feedback complementary architecture reducing weaknesses of either approach)

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.
(Supports: CNNs learning hierarchical feature representations for classification and localisation)

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, 2016, pp. 779–788.
(Supports: YOLO family widely recognised for speed and accuracy, suitable for embedded deployment)

[11] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
(Supports: YOLOv8 decoupled head, anchor-free strategy, C2f backbone; training and threshold details)

[12] A. Jadon, M. Omama, A. Varshney, M. S. Ansari, and R. Sharma, "FireNet: A Specialized Lightweight Fire & Smoke Detection Model for Real-Time IoT Applications," *arXiv preprint arXiv:1905.11922*, 2019.
(Supports: FIRE dataset used for fine-tuning)

[13] V. Hüttner, C. R. Steffens, and S. S. da Costa Botelho, "First response fire combat: Deep learning based visible fire detection," in *Proc. 2017 Latin American Robotics Symposium (LARS) and 2017 Brazilian Symposium on Robotics (SBR)*, IEEE, 2017, pp. 1–6, doi: 10.1109/SBR-LARS-R.2017.8215312.
(Supports: Furg-Fire-Dataset used for fine-tuning)

[14] Z. Zhang, L. Tan, and T. L. K. Robert, "An improved fire and smoke detection method based on YOLOv8n for smart factories," *Sensors*, vol. 24, no. 15, p. 4786, 2024, doi: 10.3390/s24154786.
(Supports: layered sensing-processing-decision-alert architecture; scalable deployment environments)

[15] N. N. N. Zaid and N. Sazali, "Investigation of IoT devices for fire alarm detection systems using Raspberry Pi," in *Proc. 6th Asia Pacific Conference on Manufacturing Systems and 4th International Manufacturing Engineering Conference (IMEC-APCOMS 2022)*, Lecture Notes in Mechanical Engineering, Springer, Singapore, 2023, pp. 85–93, doi: 10.1007/978-981-99-1245-2_12.
(Supports: Raspberry Pi and ESP8266 NodeMCU hardware platform for IoT fire detection)

[16] S. N. Saydirasulovich, M. Mukhiddinov, O. Djuraev, A. Abdusalomov, and Y.-I. Cho, "An improved wildfire smoke detection based on YOLOv8 and UAV images," *Sensors*, vol. 23, no. 20, p. 8374, 2023, doi: 10.3390/s23208374.
(Supports: hybrid detection pipeline stages; three-tier decision framework reducing false alarms)

[17] D. Zhang, S. Han, and C. Zhao, "Vision-based forest fire detection in aerial images for preventing disasters," *Energies*, vol. 11, no. 8, p. 2013, 2018.
(Supports: experimental evaluation of detection accuracy, response time, and false alarm rate)

APPENDIX B: PUBLISHED PAPER CERTIFICATES



ISSN: 2582-3930



Impact Factor: 8.659

OPEN ACCESS

We are Crossref Sponsoring

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT (IJSREM)

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering and Management is hereby awarding this certificate to

Karan Santosh Gade

In recognition of the scholarly contribution through the publication of the article titled

Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and Yolo-Based Detection

Published in the rigorously peer-reviewed IJSREM Journal, **Volume 10, Issue 6, Jun 2026**



10.55041/IJSREM64992



Editor-in-Chief
On Behalf of the Editorial Board
IJSREM Journal



editor@ijsrem.com
https://ijsrem.com

© Published by IJSREM Journal. Licensed under Creative Commons Attribution 4.0 (CC BY 4.0). Users may share and adapt the work with proper attribution.



ISSN: 2582-3930



Impact Factor: 8.659



We are Crossref Sponsoring

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT (IJSREM)

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases



CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering and Management is hereby awarding this certificate to

Aditya Vinod Gaikwad

In recognition of the scholarly contribution through the publication of the article titled

Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and Yolo-Based Detection

Published in the rigorously peer-reviewed IJSREM Journal, **Volume 10, Issue 6, Jun 2026**

doi 10.55041/IJSREM64992



Aditya Vinod Gaikwad

Editor-in-Chief
On Behalf of the Editorial Board
IJSREM Journal

editor@ijsrem.com

https://ijsrem.com



© Published by IJSREM Journal. Licensed under Creative Commons Attribution 4.0 (CC BY 4.0). Users may share and adapt the work with proper attribution.



ISSN: 2582-3930



Impact Factor: 8.659



We are Crossref Sponsoring

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT (IJSREM)

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering and Management is hereby awarding this certificate to

Avishkar Sandip Gondkar

In recognition of the scholarly contribution through the publication of the article titled

Smart Fire Surveillance and Alarm Generation System Using Raspberry Pi and Yolo-Based Detection

Published in the rigorously peer-reviewed IJSREM Journal, **Volume 10, Issue 6, Jun 2026**



10.55041 / IJSREM64992

Editor-in-Chief
On Behalf of the Editorial Board
IJSREM Journal



editor@ijsrem.com

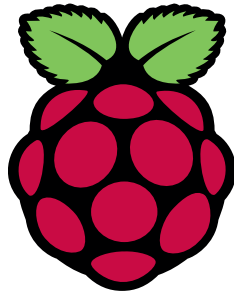
https://ijsrem.com



© Published by IJSREM Journal. Licensed under Creative Commons Attribution 4.0 (CC BY 4.0). Users may share and adapt the work with proper attribution.

ANNEXURE

DATASHEET



Raspberry Pi 4 Model B

Release 1.1

March 2024

Copyright 2024 Raspberry Pi (Trading) Ltd. All rights reserved.



Table 1: Release History

Release	Date	Description
1	21/06/2019	First release
1.1	12/03/2024	Updated obsolescence statement and electrical specification

The latest release of this document can be found at <https://www.raspberrypi.org>



Contents

1 Introduction 5

2 Features 6

2.1 Hardware 6

2.2 Interfaces 6

2.3 Software 6

3 Mechanical Specification 7

4 Electrical Specification 7

4.1 Power Requirements 8

5 Peripherals 9

5.1 GPIO Interface 9

5.1.1 GPIO Pin Assignments 9

5.1.2 GPIO Alternate Functions 10

5.1.3 Display Parallel Interface (DPI) 11

5.1.4 SD/SDIO Interface 11

5.2 Camera and Display Interfaces 11

5.3 USB 11

5.4 HDMI 11

5.5 Audio and Composite (TV Out) 11

5.6 Temperature Range and Thermals 11

6 Availability 11

7 Support 12



List of Figures

1	Mechanical Dimensions	7
2	Digital IO Characteristics	8
3	GPIO Connector Pinout	9



List of Tables

1	Release History	1
2	Absolute Maximum Ratings	7
3	DC Characteristics	8
4	Digital I/O Pin AC Characteristics	8
5	Raspberry Pi 4 GPIO Alternate Functions	10



1 Introduction

The Raspberry Pi 4 Model B (Pi4B) is the first of a new generation of Raspberry Pi computers supporting more RAM and with significantly enhanced CPU, GPU and I/O performance in a similar form factor, power envelope and cost as the previous generation Raspberry Pi 3B+.

The Pi4B is available with 1, 2, 4, or 8 gigabytes of LPDDR4 SDRAM.



2 Features

2.1 Hardware

- Quad core 64-bit ARM-Cortex A72 running at 1.5GHz
- 1, 2 and 4 Gigabyte LPDDR4 RAM options
- H.265 (HEVC) hardware decode (up to 4Kp60)
- H.264 hardware decode (up to 1080p60)
- VideoCore VI 3D Graphics
- Supports dual HDMI display output up to 4Kp60

2.2 Interfaces

- 802.11 b/g/n/ac Wireless LAN
- Bluetooth 5.0 with BLE
- 1x SD Card
- 2x micro-HDMI ports supporting dual displays up to 4Kp60 resolution
- 2x USB2 ports
- 2x USB3 ports
- 1x Gigabit Ethernet port (supports PoE with add-on PoE HAT)
- 1x Raspberry Pi camera port (2-lane MIPI CSI)
- 1x Raspberry Pi display port (2-lane MIPI DSI)
- 28x user GPIO supporting various interface options:
 - Up to 6x UART
 - Up to 6x I2C
 - Up to 5x SPI
 - 1x SDIO interface
 - 1x DPI (Parallel RGB Display)
 - 1x PCM
 - Up to 2x PWM channels
 - Up to 3x GPCLK outputs

2.3 Software

- ARMv8 Instruction Set
- Mature Linux software stack
- Actively developed and maintained



- Recent Linux kernel support
- Many drivers upstreamed
- Stable and well supported userland
- Availability of GPU functions using standard APIs

3 Mechanical Specification

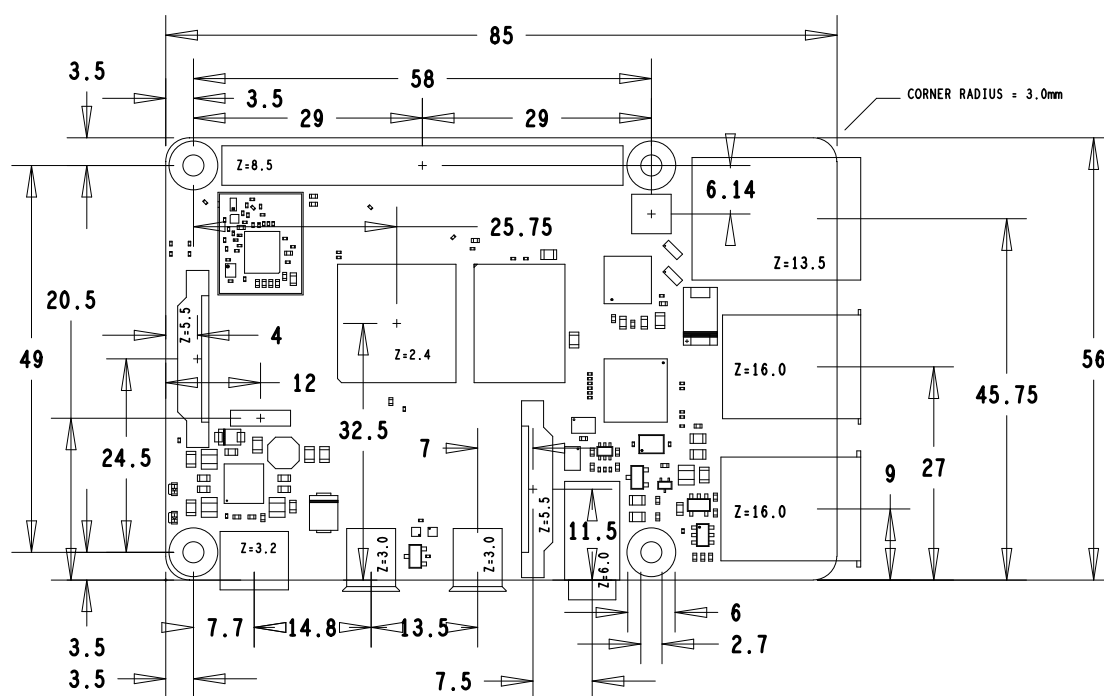


Figure 1: Mechanical Dimensions

4 Electrical Specification

Caution! Stresses above those listed in Table 2 may cause permanent damage to the device. This is a stress rating only; functional operation of the device under these or any other conditions above those listed in the operational sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

Symbol	Parameter	Minimum	Maximum	Unit
VIN	5V Input Voltage	-0.5	6.0	V

Table 2: Absolute Maximum Ratings

Please note that VDD_IO is the GPIO bank voltage which is tied to the on-board 3.3V supply rail.



Symbol	Parameter	Conditions	Minimum	Typical	Maximum	Unit
V_{IL}	Input low voltage ^a	VDD_IO = 3.3V	0	-	0.8	V
V_{IH}	Input high voltage ^a	VDD_IO = 3.3V	2.0	-	VDD_IO	V
I_{IL}	Input leakage current	TA = +85°C	-	-	10	μA
C_{IN}	Input capacitance	-	-	5	-	pF
V_{OL}	Output low voltage ^b	VDD_IO = 3.3V, IOL = -2mA	-	-	0.4	V
V_{OH}	Output high voltage ^b	VDD_IO = 3.3V, IOH = 2mA	VDD_IO - 0.4	-	-	V
I_{OL}	Output low current ^c	VDD_IO = 3.3V, VO = 0.4V	7	-	-	mA
I_{OH}	Output high current ^c	VDD_IO = 3.3V, VO = 2.3V	7	-	-	mA
R_{PU}	Pullup resistor	-	18	47	73	kΩ
R_{PD}	Pulldown resistor	-	18	47	73	kΩ

^a Hysteresis enabled

^b Default drive strength (8mA)

^c Maximum drive strength (16mA)

Table 3: DC Characteristics

Pin Name	Symbol	Parameter	Minimum	Typical	Maximum	Unit
Digital outputs	t_{rise}	10-90% rise time ^a	-	TBD	-	ns
Digital outputs	t_{fall}	90-10% fall time ^a	-	TBD	-	ns

^a Default drive strength, CL = 5pF, VDD_IO = 3.3V

Table 4: Digital I/O Pin AC Characteristics

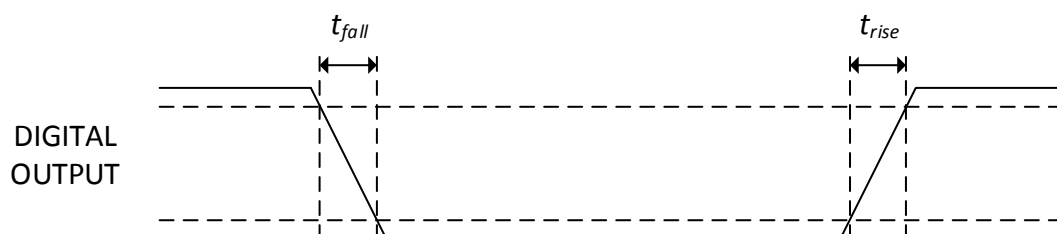


Figure 2: Digital IO Characteristics

4.1 Power Requirements

The Pi4B requires a good quality USB-C power supply capable of delivering 5V at 3A. If attached downstream USB devices consume less than 500mA, a 5V, 2.5A supply may be used.



5 Peripherals

5.1 GPIO Interface

The Pi4B makes 28 BCM2711 GPIOs available via a standard Raspberry Pi 40-pin header. This header is backwards compatible with all previous Raspberry Pi boards with a 40-way header.

5.1.1 GPIO Pin Assignments

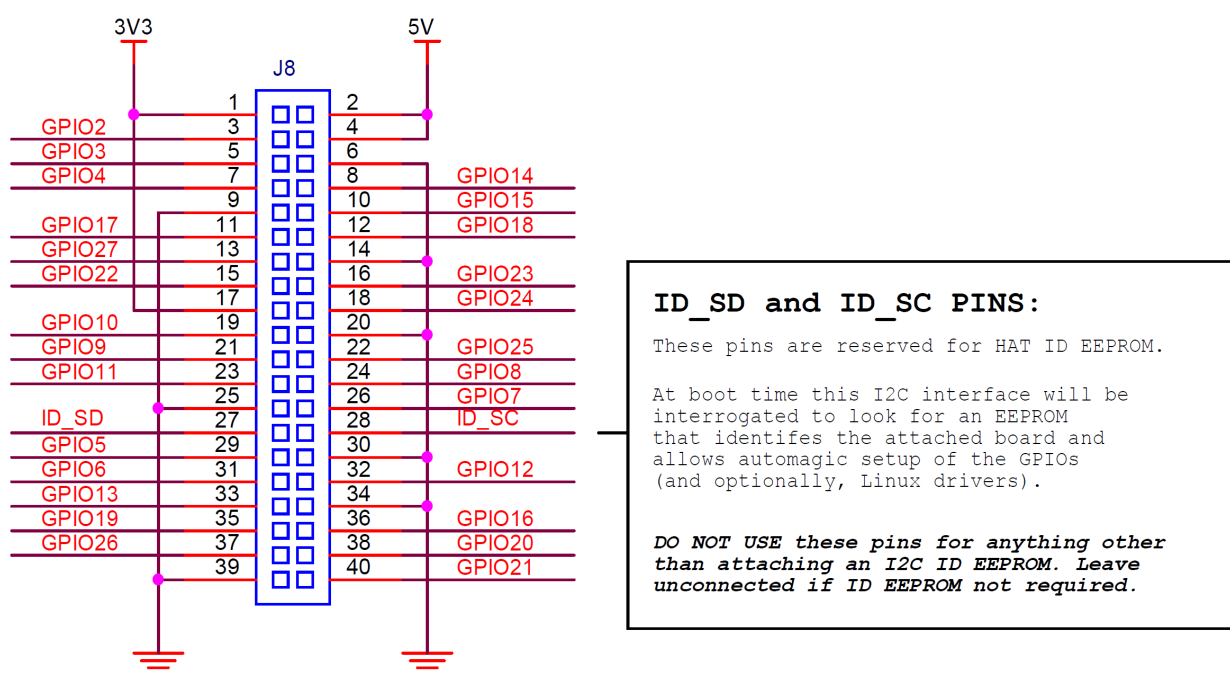


Figure 3: GPIO Connector Pinout

As well as being able to be used as straightforward software controlled input and output (with programmable pulls), GPIO pins can be switched (multiplexed) into various other modes backed by dedicated peripheral blocks such as I2C, UART and SPI.

In addition to the standard peripheral options found on legacy Pis, extra I2C, UART and SPI peripherals have been added to the BCM2711 chip and are available as further mux options on the Pi 4. This gives users much more flexibility when attaching add-on hardware as compared to older models.



5.1.2 GPIO Alternate Functions

GPIO	Default Pull	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
0	High	SDA0	SA5	PCLK	SPI3_CE0_N	TXD2	SDA6
1	High	SCL0	SA4	DE	SPI3_MISO	RXD2	SCL6
2	High	SDA1	SA3	LCD_VSYNC	SPI3_MOSI	CTS2	SDA3
3	High	SCL1	SA2	LCD_HSYNC	SPI3_SCLK	RTS2	SCL3
4	High	GPCLK0	SA1	DPI_D0	SPI4_CE0_N	TXD3	SDA3
5	High	GPCLK1	SA0	DPI_D1	SPI4_MISO	RXD3	SCL3
6	High	GPCLK2	SOE_N	DPI_D2	SPI4_MOSI	CTS3	SDA4
7	High	SPI0_CE1_N	SWE_N	DPI_D3	SPI4_SCLK	RTS3	SCL4
8	High	SPI0_CE0_N	SD0	DPI_D4	-	TXD4	SDA4
9	Low	SPI0_MISO	SD1	DPI_D5	-	RXD4	SCL4
10	Low	SPI0_MOSI	SD2	DPI_D6	-	CTS4	SDA5
11	Low	SPI0_SCLK	SD3	DPI_D7	-	RTS4	SCL5
12	Low	PWM0	SD4	DPI_D8	SPI5_CE0_N	TXD5	SDA5
13	Low	PWM1	SD5	DPI_D9	SPI5_MISO	RXD5	SCL5
14	Low	TXD0	SD6	DPI_D10	SPI5_MOSI	CTS5	TXD1
15	Low	RXD0	SD7	DPI_D11	SPI5_SCLK	RTS5	RXD1
16	Low	FL0	SD8	DPI_D12	CTS0	SPI1_CE2_N	CTS1
17	Low	FL1	SD9	DPI_D13	RTS0	SPI1_CE1_N	RTS1
18	Low	PCM_CLK	SD10	DPI_D14	SPI6_CE0_N	SPI1_CE0_N	PWM0
19	Low	PCM_FS	SD11	DPI_D15	SPI6_MISO	SPI1_MISO	PWM1
20	Low	PCM_DIN	SD12	DPI_D16	SPI6_MOSI	SPI1_MOSI	GPCLK0
21	Low	PCM_DOUT	SD13	DPI_D17	SPI6_SCLK	SPI1_SCLK	GPCLK1
22	Low	SD0_CLK	SD14	DPI_D18	SD1_CLK	ARM_TRST	SDA6
23	Low	SD0_CMD	SD15	DPI_D19	SD1_CMD	ARM_RTCK	SCL6
24	Low	SD0_DAT0	SD16	DPI_D20	SD1_DAT0	ARM_TDO	SPI3_CE1_N
25	Low	SD0_DAT1	SD17	DPI_D21	SD1_DAT1	ARM_TCK	SPI4_CE1_N
26	Low	SD0_DAT2	TE0	DPI_D22	SD1_DAT2	ARM_TDI	SPI5_CE1_N
27	Low	SD0_DAT3	TE1	DPI_D23	SD1_DAT3	ARM_TMS	SPI6_CE1_N

Table 5: Raspberry Pi 4 GPIO Alternate Functions

Table 5 details the default pin pull state and available alternate GPIO functions. Most of these alternate peripheral functions are described in detail in the BCM2711 Peripherals Specification document which can be downloaded from the [hardware documentation](#) section of the website.



5.1.3 Display Parallel Interface (DPI)

A standard parallel RGB (DPI) interface is available the GPIOs. This up-to-24-bit parallel interface can support a secondary display.

5.1.4 SD/SDIO Interface

The Pi4B has a dedicated SD card socket which supports 1.8V, DDR50 mode (at a peak bandwidth of 50 Megabytes / sec). In addition, a legacy SDIO interface is available on the GPIO pins.

5.2 Camera and Display Interfaces

The Pi4B has 1x Raspberry Pi 2-lane MIPI CSI Camera and 1x Raspberry Pi 2-lane MIPI DSI Display connector. These connectors are backwards compatible with legacy Raspberry Pi boards, and support all of the available Raspberry Pi camera and display peripherals.

5.3 USB

The Pi4B has 2x USB2 and 2x USB3 type-A sockets. Downstream USB current is limited to approximately 1.1A in aggregate over the four sockets.

5.4 HDMI

The Pi4B has 2x micro-HDMI ports, both of which support CEC and HDMI 2.0 with resolutions up to 4Kp60.

5.5 Audio and Composite (TV Out)

The Pi4B supports near-CD-quality analogue audio output and composite TV-output via a 4-ring TRS 'A/V' jack.

The analog audio output can drive 32 Ohm headphones directly.

5.6 Temperature Range and Thermals

The recommended ambient operating temperature range is 0 to 50 degrees Celsius.

To reduce thermal output when idling or under light load, the Pi4B reduces the CPU clock speed and voltage. During heavier load the speed and voltage (and hence thermal output) are increased. The internal governor will throttle back both the CPU speed and voltage to make sure the CPU temperature never exceeds 85 degrees C.

The Pi4B will operate perfectly well without any extra cooling and is designed for sprint performance - expecting a light use case on average and ramping up the CPU speed when needed (e.g. when loading a webpage). If a user wishes to load the system continually or operate it at a high temperature at full performance, further cooling may be needed.

6 Availability

Raspberry Pi guarantees availability of the Pi4B until at least January 2031.



7 Support

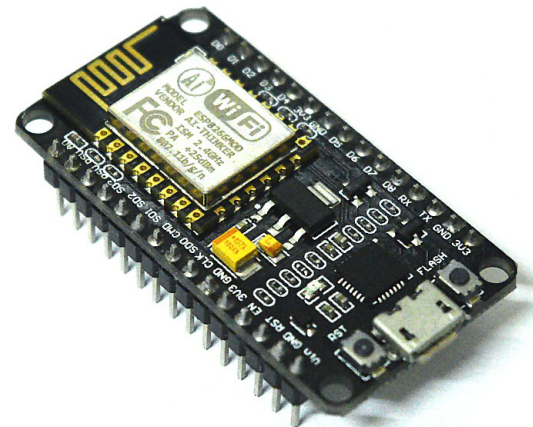
For support please see the hardware documentation section of the [Raspberry Pi website](#) and post questions to the [Raspberry Pi forum](#).

NodeMCU ESP8266 ESP-12E WiFi Development Board

NodeMCU is an open source IoT platform. It includes firmware which runs on the ESP8266 Wi-Fi SoC from Espressif Systems, and hardware which is based on the ESP-12 module. The term “NodeMCU” by default refers to the firmware rather than the DevKit. The firmware uses the Lua scripting language. It is based on the eLua project, and built on the Espressif Non-OS SDK for ESP8266. It uses many open source projects, such as lua-cjson, and spiffs.

Features

- ▶ Version : DevKit v1.0
- ▶ Breadboard Friendly
- ▶ Light Weight and small size.
- ▶ 3.3V operated, can be USB powered.
- ▶ Uses wireless protocol 802.11b/g/n.
- ▶ Built-in wireless connectivity capabilities.
- ▶ Built-in PCB antenna on the ESP-12E chip.
- ▶ Capable of PWM, I2C, SPI, UART, 1-wire, 1 analog pin.
- ▶ Uses CP2102 USB Serial Communication interface module.
- ▶ Arduino IDE compatible (extension board manager required).
- ▶ Supports Lua (alike node.js) and Arduino C programming language.



Wireless Connectivity



Breadboard Friendly



USB Compatible



Lightweight



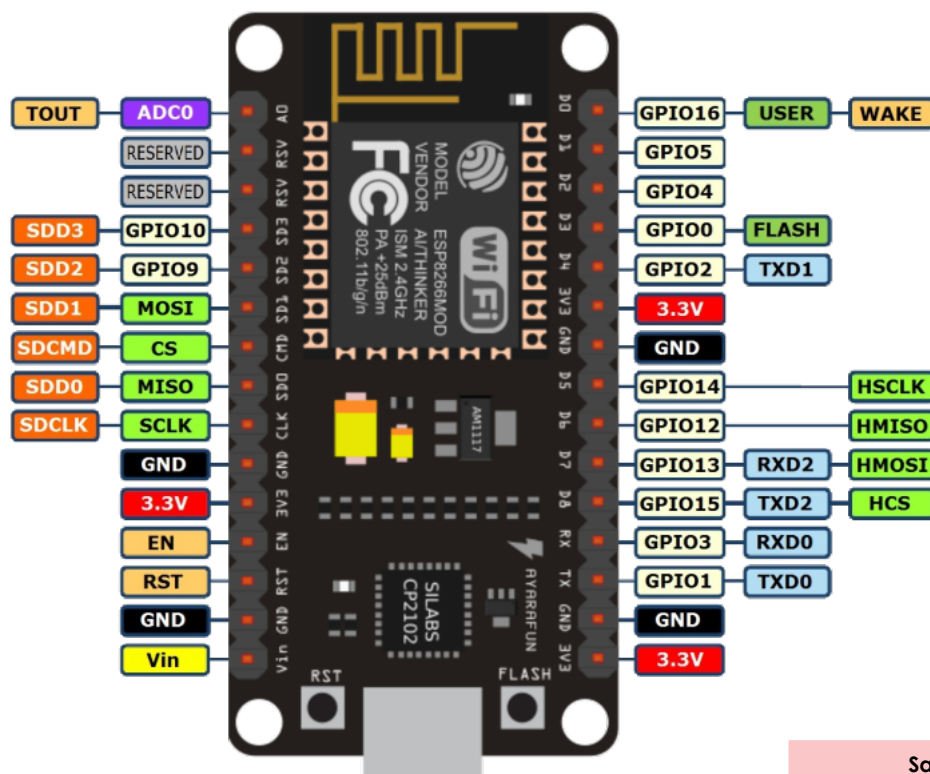
Arduino IDE Compatible



3.3V POWERED
Low Power Consumption

PINOUT DIAGRAM

NodeMCU ESP8266 v1.0

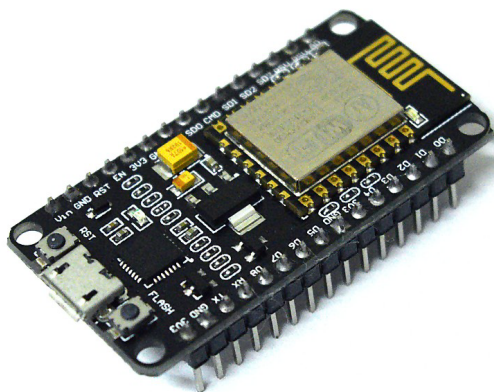


Source

<https://iotbytes.wordpress.com/nodemcu-pinout/>

Safety Precaution

All GPIO runs at 3.3V !!



Front View



Front View

Specifications of ESP-12E WiFi Module

Wireless Standard	IEEE 802.11 b/g/n
Frequency Range	2.412 - 2.484 GHz
Power Transmission	802.11b : +16 ± 2 dBm (at 11 Mbps) 802.11g : +14 ± 2 dBm (at 54 Mbps) 802.11n : +13 ± 2 dBM (at HT20, MCS7)
Receiving Sensitivity	802.11b : -93 dBm (at 11 Mbps, CCK) 802.11g : -85 dBm (at 54 Mbps, OFDM) 802.11n : -82 dBm (at HT20, MCS7)
Wireless Form	On-board PCB Antenna
IO Capability	UART, I2C, PWM, GPIO, 1 ADC
Electrical Characteristic	3.3 V Operated 15 mA output current per GPIO pin 12 - 200 mA working current Less than 200 uA standby current
Operating Temperature	-40 to +125 °C
Serial Transmission	110 - 921600 bps, TCP Client 5
Wireless Network Type	STA / AP / STA + AP
Security Type	WEP / WPA-PSK / WPA2-PSK
Encryption Type	WEP64 / WEP128 / TKIP / AES
Firmware Upgrade	Local Serial Port, OTA Remote Upgrade
Network Protocol	IPv4, TCP / UDP / FTP / HTTP
User Configuration	AT + Order Set, Web Android / iOS, Smart Link APP

Disclaimer

Information provided in this document are compilation from various online resources. Einstronic Enterprise does not ensure the completeness, accuracy and reliability of the information and do not own any rights on any registered trademarks involved. Information provided should be intended for references only.

TECHNICAL DATA**MQ-2 GAS SENSOR****FEATURES**

Wide detecting scope
Stable and long life

Fast response and High sensitivity
Simple drive circuit

APPLICATION

They are used in gas leakage detecting equipments in family and industry, are suitable for detecting of LPG, i-butane, propane, methane ,alcohol, Hydrogen, smoke.

SPECIFICATIONS**A. Standard work condition**

Symbol	Parameter name	Technical condition	Remarks
V _c	Circuit voltage	5V±0.1	AC OR DC
V _H	Heating voltage	5V±0.1	AC OR DC
R _L	Load resistance	can adjust	
R _H	Heater resistance	33 Ω ± 5%	Room Tem
P _H	Heating consumption	less than 800mw	

B. Environment condition

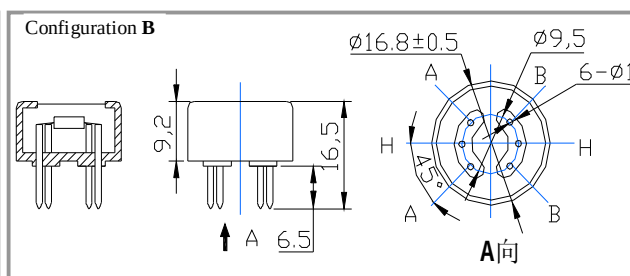
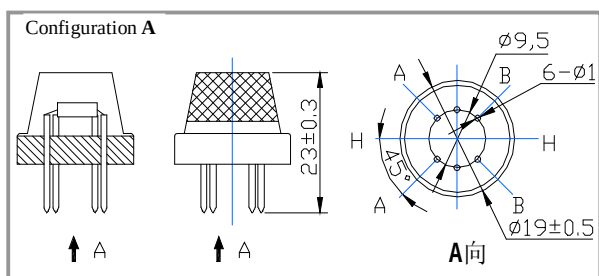
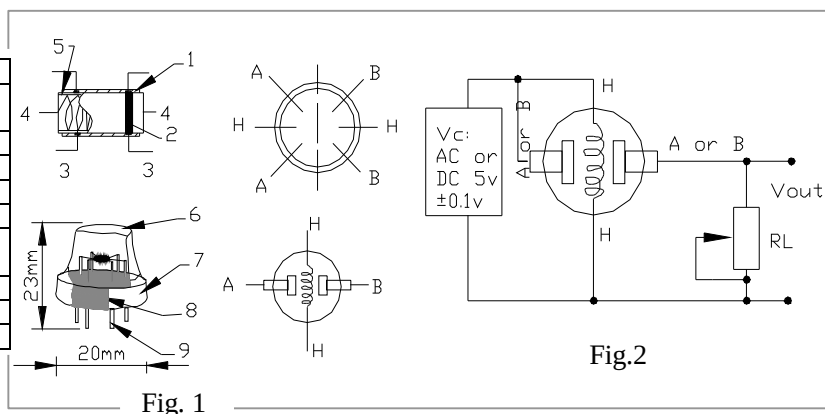
Symbol	Parameter name	Technical condition	Remarks
Tao	Using Tem	-20℃-50℃	
Tas	Storage Tem	-20℃-70℃	
R _H	Related humidity	less than 95%Rh	
O ₂	Oxygen concentration	21%(standard condition)Oxygen concentration can affect sensitivity	minimum value is over 2%

C. Sensitivity characteristic

Symbol	Parameter name	Technical parameter	Remarks
Rs	Sensing Resistance	3K Ω -30K Ω (1000ppm iso-butane)	Detecting concentration scope: 200ppm-5000ppm LPG and propane 300ppm-5000ppm butane 5000ppm-20000ppm methane 300ppm-5000ppm H ₂ 100ppm-2000ppm Alcohol
α (3000/1000) isobutane	Concentration Slope rate	≤0.6	
Standard Detecting Condition	Temp: 20℃ ± 2℃ Vc:5V±0.1 Humidity: 65%±5% Vh: 5V±0.1		
Preheat time	Over 24 hour		

D. Structure and configuration, basic measuring circuit

	Parts	Materials
1	Gas sensing layer	SnO ₂
2	Electrode	Au
3	Electrode line	Pt
4	Heater coil	Ni-Cr alloy
5	Tubular ceramic	Al ₂ O ₃
6	Anti-explosion network	Stainless steel gauze (SUS316 100-mesh)
7	Clamp ring	Copper plating Ni
8	Resin base	Bakelite
9	Tube Pin	Copper plating Ni



Structure and configuration of MQ-2 gas sensor is shown as Fig. 1 (Configuration A or B), sensor composed by micro AL₂O₃ ceramic tube, Tin Dioxide (SnO₂) sensitive layer, measuring electrode and heater are fixed into a

crust made by plastic and stainless steel net. The heater provides necessary work conditions for work of sensitive components. The enveloped MQ-2 have 6 pin ,4 of them are used to fetch signals, and other 2 are used for providing heating current.

Electric parameter measurement circuit is shown as Fig.2

E. Sensitivity characteristic curve

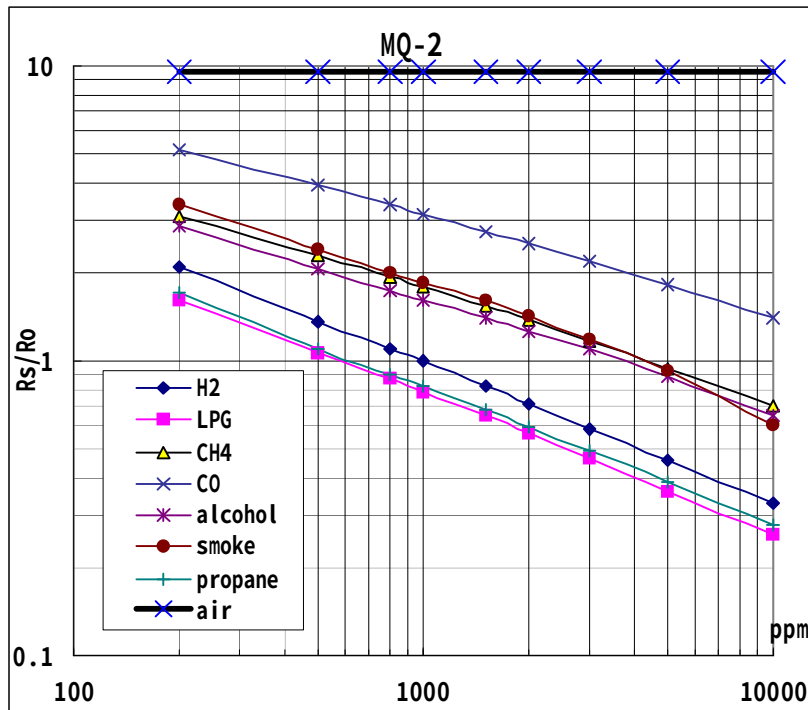


Fig.2 sensitivity characteristics of the MQ-2

Fig.3 is shows the typical sensitivity characteristics of the MQ-2 for several gases.

in their: Temp: 20℃、
Humidity: 65%、
O₂ concentration 21%
RL=5k Ω

Ro: sensor resistance at 1000ppm of H₂ in the clean air.

Rs:sensor resistance at various concentrations of gases.

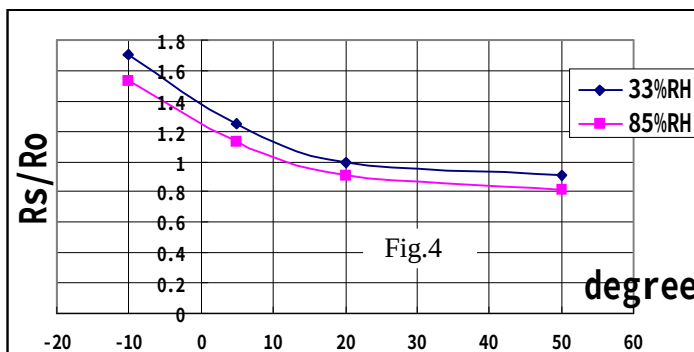


Fig.4 is shows the typical dependence of the MQ-2 on temperature and humidity.

Ro: sensor resistance at 1000ppm of H₂ in air at 33%RH and 20 degree.

Rs: sensor resistance at 1000ppm of H₂ at different temperatures and humidities.

SENSITIVITY ADJUSTMENT

Resistance value of MQ-2 is difference to various kinds and various concentration gases. So, When using this components, sensitivity adjustment is very necessary. we recommend that you calibrate the detector for 1000ppm liquified petroleum gas<LPG>, or 1000ppm iso-butane<i-C₄H₁₀> concentration in air and use value of Load resistance that(R_L) about 20 K Ω (5K Ω to 47 K Ω).

When accurately measuring, the proper alarm point for the gas detector should be determined after considering the temperature and humidity influence.

Mouser Electronics

Authorized Distributor

Click to View Pricing, Inventory, Delivery & Lifecycle Information:

Parallax:

605-00008

LM393, LM393A, LM293, LM2903, LM2903V

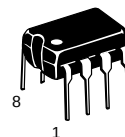
Low Offset Voltage Dual Comparators

The LM393 series are dual independent precision voltage comparators capable of single or split supply operation. These devices are designed to permit a common mode range—to-ground level with single supply operation. Input offset voltage specifications as low as 2.0 mV make this device an excellent selection for many applications in consumer automotive, and industrial electronics.

- Wide Single-Supply Range: 2.0 Vdc to 36 Vdc
- Split-Supply Range: ± 1.0 Vdc to ± 18 Vdc
- Very Low Current Drain Independent of Supply Voltage: 0.4 mA
- Low Input Bias Current: 25 nA
- Low Input Offset Current: 5.0 nA
- Low Input Offset Voltage: 2.0 mV (max) LM393A
5.0 mV (max) LM293/393
- Input Common Mode Range to Ground Level
- Differential Input Voltage Range Equal to Power Supply Voltage
- Output Voltage Compatible with DTL, ECL, TTL, MOS, and CMOS Logic Levels
- ESD Clamps on the Inputs Increase the Ruggedness of the Device without Affecting Performance

SINGLE SUPPLY, LOW POWER DUAL COMPARATORS

SEMICONDUCTOR TECHNICAL DATA

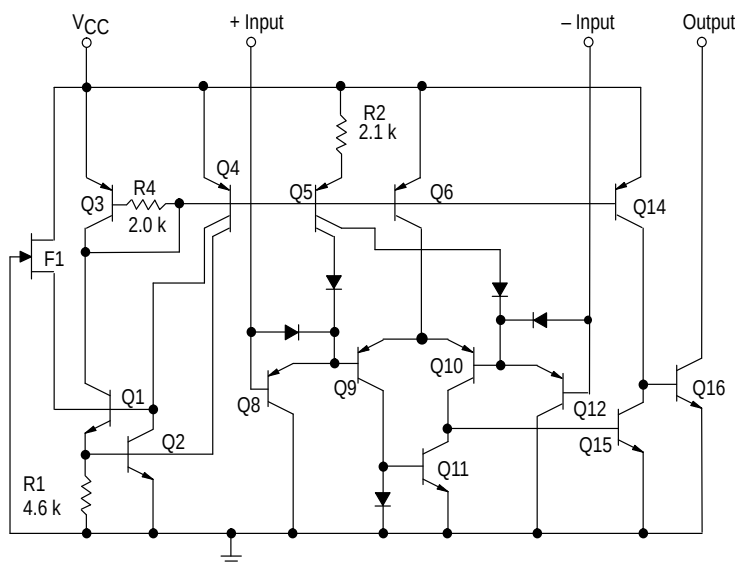


N SUFFIX
PLASTIC PACKAGE
CASE 626

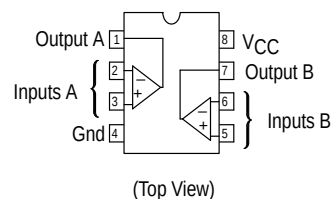


D SUFFIX
PLASTIC PACKAGE
CASE 751
(SO-8)

Representative Schematic Diagram
(Diagram shown is for 1 comparator)



PIN CONNECTIONS



ORDERING INFORMATION

Device	Operating Temperature Range	Package
LM293D	$T_A = -25^\circ$ to $+85^\circ\text{C}$	SO-8
LM393D	$T_A = 0^\circ$ to $+70^\circ\text{C}$	SO-8
LM393AN,N		Plastic DIP
LM2903D	$T_A = -40^\circ$ to $+105^\circ\text{C}$	SO-8
LM2903N		Plastic DIP
LM2903VD	$T_A = -40^\circ$ to $+105^\circ\text{C}$	SO-8
LM2903VN		Plastic DIP

LM393, LM393A, LM293, LM2903, LM2903V

MAXIMUM RATINGS

Rating	Symbol	Value	Unit
Power Supply Voltage	V_{CC}	+36 or ± 18	Vdc
Input Differential Voltage Range	V_{IDR}	36	Vdc
Input Common Mode Voltage Range	V_{ICR}	-0.3 to +36	Vdc
Output Short Circuit-to-Ground Output Sink Current (Note 1)	I_{SC} I_{Sink}	Continuous 20	mA
Power Dissipation @ $T_A = 25^\circ\text{C}$ Derate above 25°C	P_D $1/R_{\theta JA}$	570 5.7	mW mW/ $^\circ\text{C}$
Operating Ambient Temperature Range LM293 LM393, 393A LM2903 LM2903V	T_A	-25 to +85 0 to +70 -40 to +105 -40 to +125	$^\circ\text{C}$
Maximum Operating Junction Temperature LM393, 393A, 2903, LM2903V LM293	$T_{J(max)}$	125 150	$^\circ\text{C}$
Storage Temperature Range	T_{stg}	-65 to +150	$^\circ\text{C}$

ELECTRICAL CHARACTERISTICS ($V_{CC} = 5.0\text{ Vdc}$, $T_{low} \leq T_A \leq T_{high}$, * unless otherwise noted.)

Characteristic	Symbol	LM393A			Unit
		Min	Typ	Max	
Input Offset Voltage (Note 2) $T_A = 25^\circ\text{C}$ $T_{low} \leq T_A \leq T_{high}$	V_{IO}	- -	± 1.0 -	± 2.0 4.0	mV
Input Offset Current $T_A = 25^\circ\text{C}$ $T_{low} \leq T_A \leq T_{high}$	I_{IO}	- -	± 50 -	± 50 ± 150	nA
Input Bias Current (Note 3) $T_A = 25^\circ\text{C}$ $T_{low} \leq T_A \leq T_{high}$	I_{IB}	- -	25 -	250 400	nA
Input Common Mode Voltage Range (Note 4) $T_A = 25^\circ\text{C}$ $T_{low} \leq T_A \leq T_{high}$	V_{ICR}	0 0	- -	$V_{CC} - 1.5$ $V_{CC} - 2.0$	V
Voltage Gain $R_L \geq 15\text{ k}\Omega$, $V_{CC} = 15\text{ Vdc}$, $T_A = 25^\circ\text{C}$	A_{VOL}	50	200	-	V/mV
Large Signal Response Time $V_{in} = \text{TTL Logic Swing}$, $V_{ref} = 1.4\text{ Vdc}$ $V_{RL} = 5.0\text{ Vdc}$, $R_L = 5.1\text{ k}\Omega$, $T_A = 25^\circ\text{C}$	-	-	300	-	ns
Response Time (Note 5) $V_{RL} = 5.0\text{ Vdc}$, $R_L = 5.1\text{ k}\Omega$, $T_A = 25^\circ\text{C}$	t_{TLH}	-	1.3	-	μs
Input Differential Voltage (Note 6) All $V_{in} \geq \text{Gnd}$ or V_- Supply (if used)	V_{ID}	-	-	V_{CC}	V
Output Sink Current $V_{in} \geq 1.0\text{ Vdc}$, $V_{in+} = 0\text{ Vdc}$, $V_O \leq 1.5\text{ Vdc}$, $T_A = 25^\circ\text{C}$	I_{Sink}	6.0	16	-	mA
Output Saturation Voltage $V_{in} \geq 1.0\text{ Vdc}$, $V_{in+} = 0\text{ Vdc}$, $I_{Sink} \leq 4.0\text{ mA}$, $T_A = 25^\circ\text{C}$ $T_{low} \leq T_A \leq T_{high}$	V_{OL}	- -	150 -	400 700	mV

* $T_{low} = 0^\circ\text{C}$, $T_{high} = +70^\circ\text{C}$ for LM393/393A

- NOTES:**
1. The maximum output current may be as high as 20 mA, independent of the magnitude of V_{CC} , output short circuits to V_{CC} can cause excessive heating and eventual destruction.
 2. At output switch point, $V_O = 1.4\text{ Vdc}$, $R_S = 0\ \Omega$ with V_{CC} from 5.0 Vdc to 30 Vdc, and over the full input common mode range (0 V to $V_{CC} = -1.5\text{ V}$).
 3. Due to the PNP transistor inputs, bias current will flow out of the inputs. This current is essentially constant, independent of the output state, therefore, no loading changes will exist on the input lines.
 4. Input common mode of either input should not be permitted to go more than 0.3 V negative of ground or minus supply. The upper limit of common mode range is $V_{CC} - 1.5\text{ V}$.
 5. Response time is specified with a 100 mV step and 5.0 mV of overdrive. With larger magnitudes of overdrive faster response times are obtainable.
 6. The comparator will exhibit proper output state if one of the inputs becomes greater than V_{CC} , the other input must remain within the common mode range. The low input state must not be less than -0.3 V of ground or minus supply.

LM393, LM393A, LM293, LM2903, LM2903V

ELECTRICAL CHARACTERISTICS (V_{CC} = 5.0 Vdc, T_{low} ≤ T_A ≤ T_{high},* unless otherwise noted.)

Characteristic	Symbol	LM393A			Unit
		Min	Typ	Max	
Output Leakage Current V _{in-} = 0 V, V _{in+} ≥ 1.0 Vdc, V _O = 5.0 Vdc, T _A = 25°C V _{in-} = 0 V, V _{in+} ≥ 1.0 Vdc, V _O = 30 Vdc, T _{low} ≤ T _A ≤ T _{high}	I _{OL}	–	0.1	–	μA
		–	–	1.0	
Supply Current R _L = ∞ Both Comparators, T _A = 25°C R _L = ∞ Both Comparators, V _{CC} = 30 V	I _{CC}	–	0.4	1.0	mA
		–	1.0	2.5	

ELECTRICAL CHARACTERISTICS (V_{CC} = 5.0 Vdc, T_{low} ≤ T_A ≤ T_{high}, unless otherwise noted.)

Characteristic	Symbol	LM392, LM393			LM2903, LM2903V			Unit
		Min	Typ	Max	Min	Typ	Max	
Input Offset Voltage (Note 2) T _A = 25°C T _{low} ≤ T _A ≤ T _{high}	V _{IO}	–	±1.0	±5.0	–	±2.0	±7.0	mV
		–	–	9.0	–	9.0	15	
Input Offset Current T _A = 25°C T _{low} ≤ T _A ≤ T _{high}	I _{IO}	–	±5.0	±50	–	±5.0	±50	nA
		–	–	±150	–	±50	±200	
Input Bias Current (Note 3) T _A = 25°C T _{low} ≤ T _A ≤ T _{high}	I _{IB}	–	25	250	–	25	250	nA
		–	–	400	–	200	500	
Input Common Mode Voltage Range (Note 3) T _A = 25°C T _{low} ≤ T _A ≤ T _{high}	V _{ICR}	0	–	V _{CC} –1.5	0	–	V _{CC} –1.5	V
		0	–	V _{CC} –2.0	0	–	V _{CC} –2.0	
Voltage Gain R _L ≥ 15 kΩ, V _{CC} = 15 Vdc, T _A = 25°C	A _{VOL}	50	200	–	25	200	–	V/mV
Large Signal Response Time V _{in} = TTL Logic Swing, V _{ref} = 1.4 Vdc V _{RL} = 5.0 Vdc, R _L = 5.1 kΩ, T _A = 25°C	–	–	300	–	–	300	–	ns
Response Time (Note 5) V _{RL} = 5.0 Vdc, R _L = 5.1 kΩ, T _A = 25°C	t _{TLH}	–	1.3	–	–	1.5	–	μs
Input Differential Voltage (Note 6) All V _{in} ≥ Gnd or V– Supply (if used)	V _{ID}	–	–	V _{CC}	–	–	V _{CC}	V
Output Sink Current V _{in} ≥ 1.0 Vdc, V _{in+} = 0 Vdc, V _O ≤ 1.5 Vdc T _A = 25°C	I _{Sink}	6.0	16	–	6.0	16	–	mA
Output Saturation Voltage V _{in} ≥ 1.0 Vdc, V _{in+} = 0, I _{Sink} ≤ 4.0 mA, T _A = 25°C T _{low} ≤ T _A ≤ T _{high}	V _{OL}	–	150	400	–	–	400	mV
		–	–	700	–	200	700	
Output Leakage Current V _{in-} = 0 V, V _{in+} ≥ 1.0 Vdc, V _O = 5.0 Vdc, T _A = 25°C V _{in-} = 0 V, V _{in+} ≥ 1.0 Vdc, V _O = 30 Vdc, T _{low} ≤ T _A ≤ T _{high}	I _{OL}	–	0.1	–	–	0.1	–	nA
		–	–	1000	–	–	1000	
Supply Current R _L = ∞ Both Comparators, T _A = 25°C R _L = ∞ Both Comparators, V _{CC} = 30 V	I _{CC}	–	0.4	1.0	–	0.4	1.0	mA
		–	–	2.5	–	–	2.5	

* T_{low} = 0°C, T_{high} = +70°C for LM393/393A

LM293 T_{low} = –25°C, T_{high} = +85°C

LM2903 T_{low} = –40°C, T_{high} = +105°C

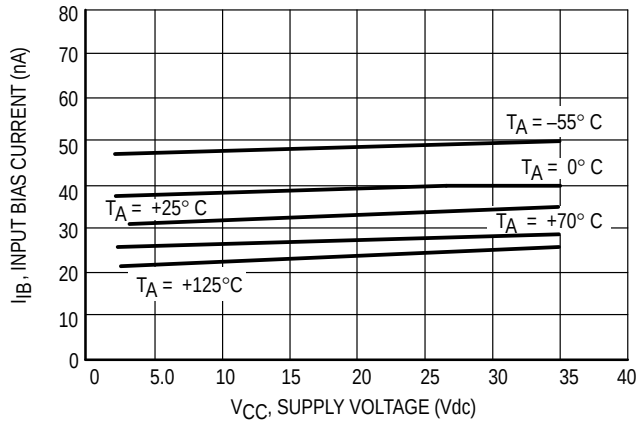
LM2903V T_{low} = –40°C, T_{high} = +125°C

- NOTES:**
- At output switch point, V_O = 1.4 Vdc, R_S = 0 Ω with V_{CC} from 5.0 Vdc to 30 Vdc, and over the full input common mode range (0 V to V_{CC} = –1.5 V).
 - Due to the PNP transistor inputs, bias current will flow out of the inputs. This current is essentially constant, independent of the output state, therefore, no loading changes will exist on the input lines.
 - Response time is specified with a 100 mV step and 5.0 mV of overdrive. With larger magnitudes of overdrive faster response times are obtainable.
 - The comparator will exhibit proper output state if one of the inputs becomes greater than V_{CC}, the other input must remain within the common mode range. The low input state must not be less than –0.3 V of ground or minus supply.

LM393, LM393A, LM293, LM2903, LM2903V

LM293/393,A

Figure 1. Input Bias Current versus Power Supply Voltage



LM2903

Figure 2. Input Bias Current versus Power Supply Voltage

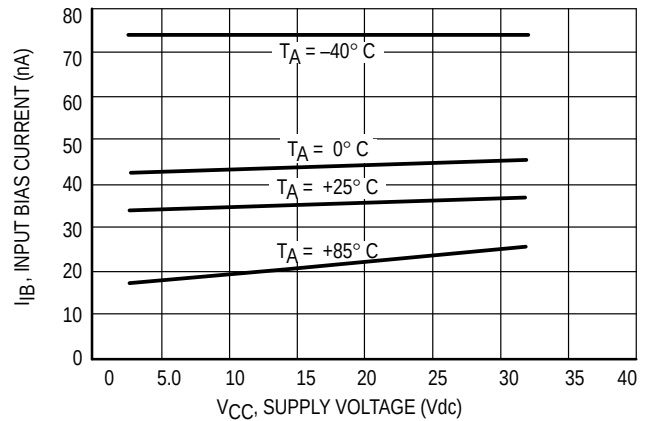


Figure 3. Output Saturation Voltage versus Output Sink Current

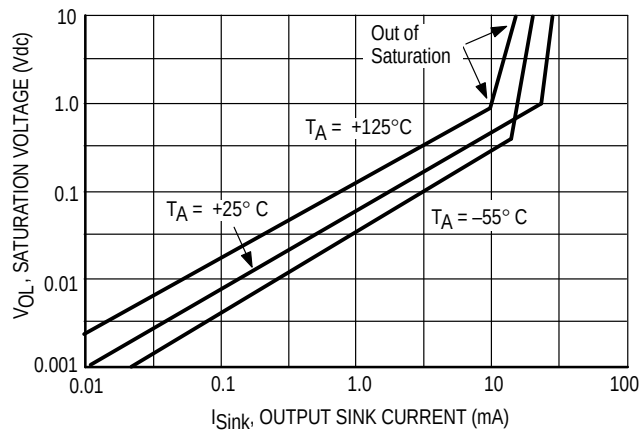


Figure 4. Output Saturation Voltage versus Output Sink Current

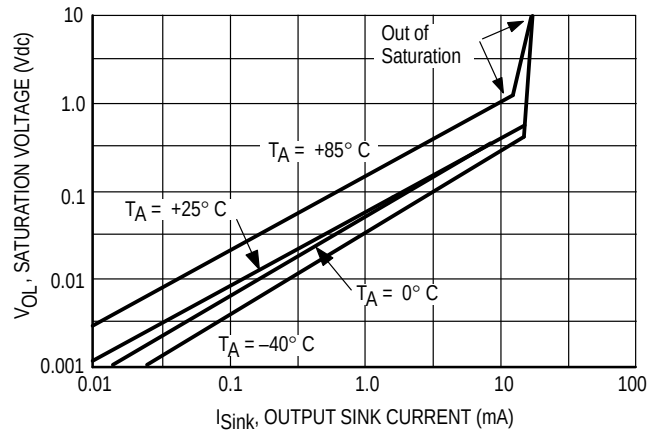


Figure 5. Power Supply Current versus Power Supply Voltage

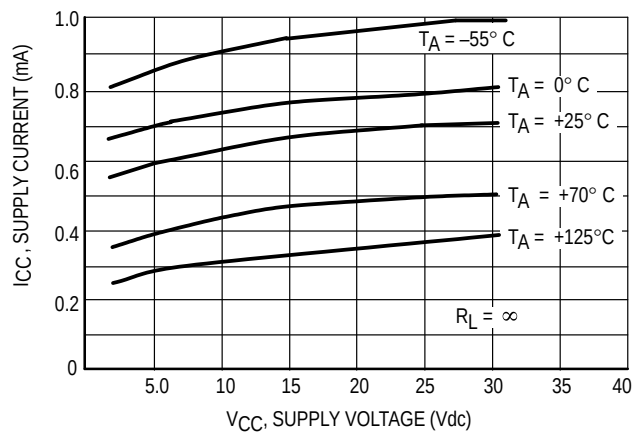
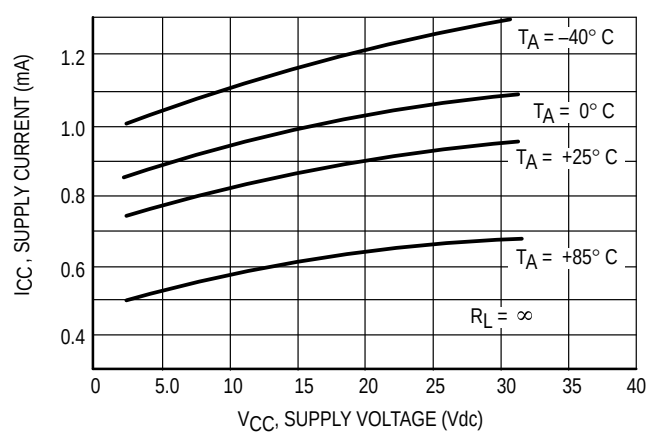


Figure 6. Power Supply Current versus Power Supply Voltage



LM393, LM393A, LM293, LM2903, LM2903V

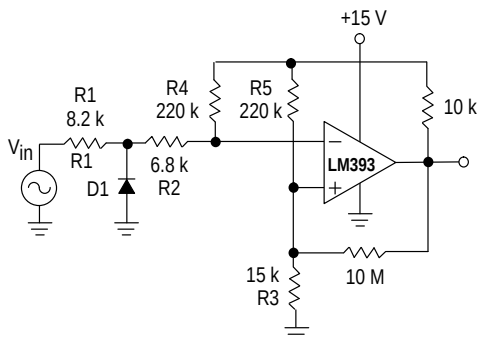
APPLICATIONS INFORMATION

These dual comparators feature high gain, wide bandwidth characteristics. This gives the device oscillation tendencies if the outputs are capacitively coupled to the inputs via stray capacitance. This oscillation manifests itself during output transitions (V_{OL} to V_{OH}). To alleviate this situation, input resistors $<10\text{ k}\Omega$ should be used.

The addition of positive feedback ($<10\text{ mV}$) is also recommended. It is good design practice to ground all unused pins.

Differential input voltages may be larger than supply voltage without damaging the comparator's inputs. Voltages more negative than -0.3 V should not be used.

Figure 7. Zero Crossing Detector (Single Supply)

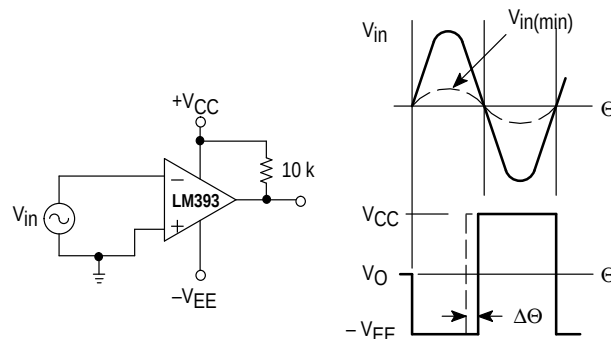


D1 prevents input from going negative by more than 0.6 V .

$$R1 + R2 = R3$$

$$R3 \leq \frac{R5}{10} \text{ for small error in zero crossing.}$$

Figure 8. Zero Crossing Detector (Split Supply)



$$V_{in(min)} \approx 0.4\text{ V peak for } 1\% \text{ phase distortion } (\Delta\theta).$$

Figure 9. Free-Running Square-Wave Oscillator

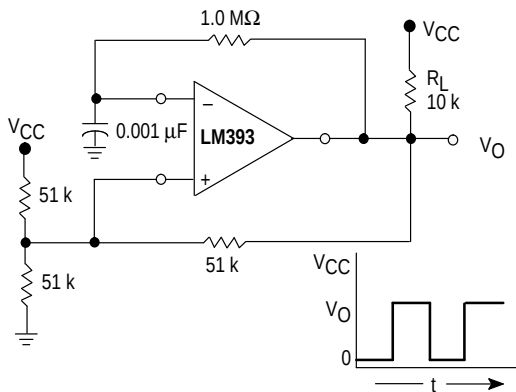
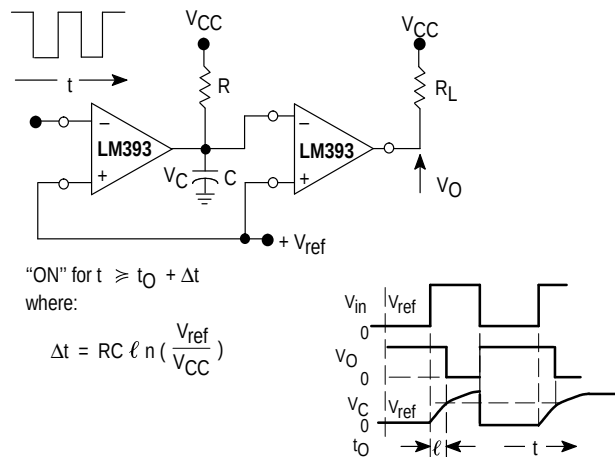


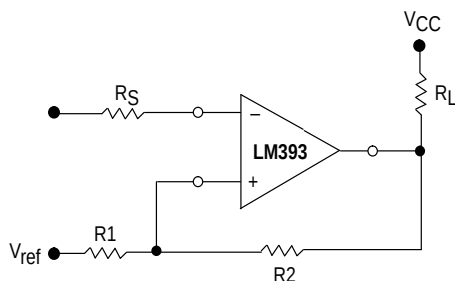
Figure 10. Time Delay Generator



"ON" for $t \geq t_0 + \Delta t$
where:

$$\Delta t = RC \ln \left(\frac{V_{ref}}{V_{CC}} \right)$$

Figure 11. Comparator with Hysteresis



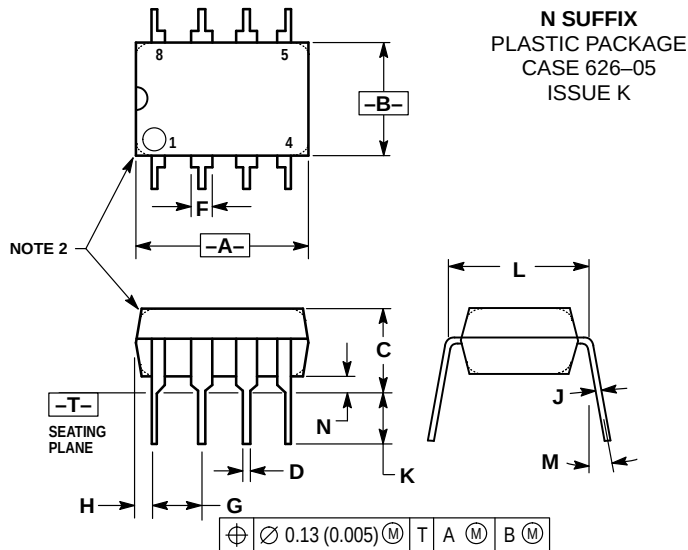
$$R_S = R1 \parallel R2$$

$$V_{th1} = V_{ref} + \frac{(V_{CC} - V_{ref}) R1}{R1 + R2 + R_L}$$

$$V_{th2} = V_{ref} - \frac{(V_{ref} - V_{OL}) R1}{R1 + R2}$$

LM393, LM393A, LM293, LM2903, LM2903V

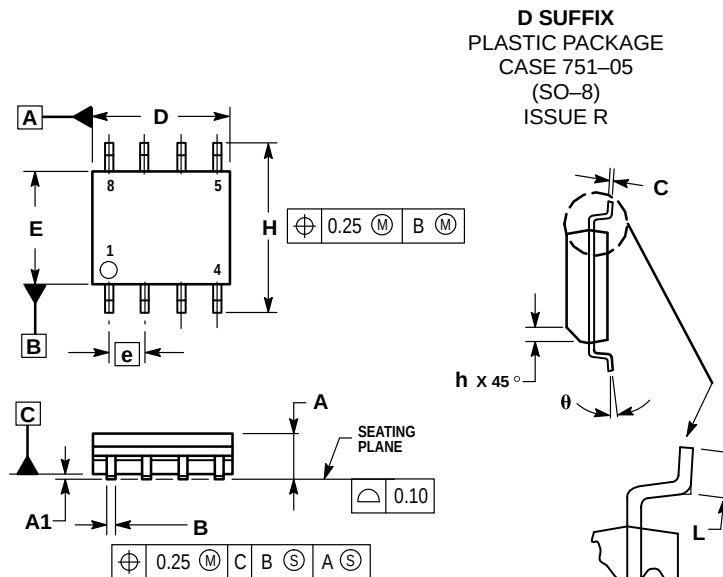
OUTLINE DIMENSIONS



NOTES:

1. DIMENSION L TO CENTER OF LEAD WHEN FORMED PARALLEL.
2. PACKAGE CONTOUR OPTIONAL (ROUND OR SQUARE CORNERS).
3. DIMENSIONING AND TOLERANCING PER ANSI Y14.5M, 1982.

DIM	MILLIMETERS		INCHES	
	MIN	MAX	MIN	MAX
A	9.40	10.16	0.370	0.400
B	6.10	6.60	0.240	0.260
C	3.94	4.45	0.155	0.175
D	0.38	0.51	0.015	0.020
F	1.02	1.78	0.040	0.070
G	2.54 BSC		0.100 BSC	
H	0.76	1.27	0.030	0.050
J	0.20	0.30	0.008	0.012
K	2.92	3.43	0.115	0.135
L	7.62 BSC		0.300 BSC	
M	10°		10°	
N	0.76	1.01	0.030	0.040



NOTES:

1. DIMENSIONING AND TOLERANCING PER ASME Y14.5M, 1994.
2. DIMENSIONS ARE IN MILLIMETERS.
3. DIMENSION D AND E DO NOT INCLUDE MOLD PROTRUSION.
4. MAXIMUM MOLD PROTRUSION 0.15 PER SIDE.
5. DIMENSION B DOES NOT INCLUDE MOLD PROTRUSION. ALLOWABLE DAMBAR PROTRUSION SHALL BE 0.127 TOTAL IN EXCESS OF THE B DIMENSION AT MAXIMUM MATERIAL CONDITION.

DIM	MILLIMETERS	
	MIN	MAX
A	1.35	1.75
A1	0.10	0.25
B	0.35	0.49
C	0.18	0.25
D	4.80	5.00
E	3.80	4.00
e	1.27 BSC	
H	5.80	6.20
h	0.25	0.50
L	0.40	1.25
θ	0°	7°

Motorola reserves the right to make changes without further notice to any products herein. Motorola makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does Motorola assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters which may be provided in Motorola data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. Motorola does not convey any license under its patent rights nor the rights of others. Motorola products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the Motorola product could create a situation where personal injury or death may occur. Should Buyer purchase or use Motorola products for any such unintended or unauthorized application, Buyer shall indemnify and hold Motorola and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that Motorola was negligent regarding the design or manufacture of the part. Motorola and are registered trademarks of Motorola, Inc. Motorola, Inc. is an Equal Opportunity/Affirmative Action Employer.

How to reach us:

USA/EUROPE/Locations Not Listed: Motorola Literature Distribution;
P.O. Box 20912; Phoenix, Arizona 85036. 1-800-441-2447 or 602-303-5454

MFAX: RMFAX0@email.sps.mot.com - TOUCHTONE 602-244-6609
INTERNET: <http://Design-NET.com>

JAPAN: Nippon Motorola Ltd.; Tatsumi-SPD-JLDC, 6F Seibu-Butsuryu-Center,
3-14-2 Tatsumi Koto-Ku, Tokyo 135, Japan. 03-81-3521-8315

ASIA/PACIFIC: Motorola Semiconductors H.K. Ltd.; 8B Tai Ping Industrial Park,
51 Ting Kok Road, Tai Po, N.T., Hong Kong. 852-26629298





Features

- Black in colour
- With internal drive circuit
- Sealed structure
- Wave solderable and washable
- Housing material: Noryl

**RoHS
Compliant**

Applications

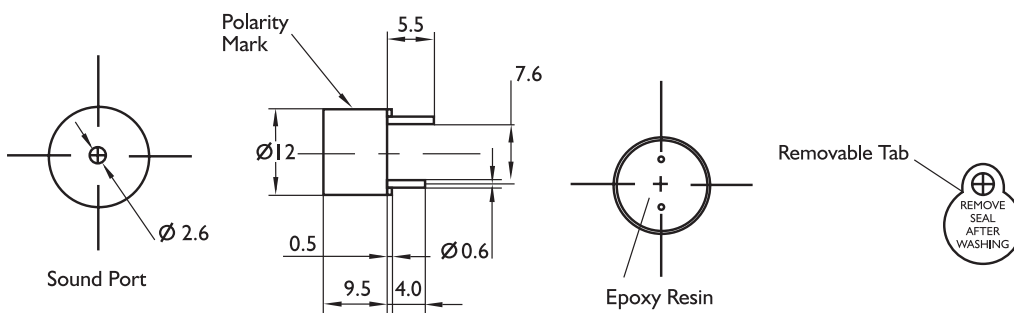
- Computer and peripherals
- Communications equipment
- Portable equipment
- Automobile electronics
- POS system
- Electronic cash register

Specifications:

Rated Voltage	: 6V DC
Operating Voltage	: 4 to 8V DC
Rated Current*	: ≤30mA
Sound Output at 10cm*	: ≥85dB
Resonant Frequency	: 2300 ±300Hz
Tone	: Continuous
Operating Temperature	: -25°C to +80°C
Storage Temperature	: -30°C to +85°C
Weight	: 2g

*Value applying at rated voltage (DC)

Diagram



Dimensions : Millimetres
Tolerance : ±0.5mm

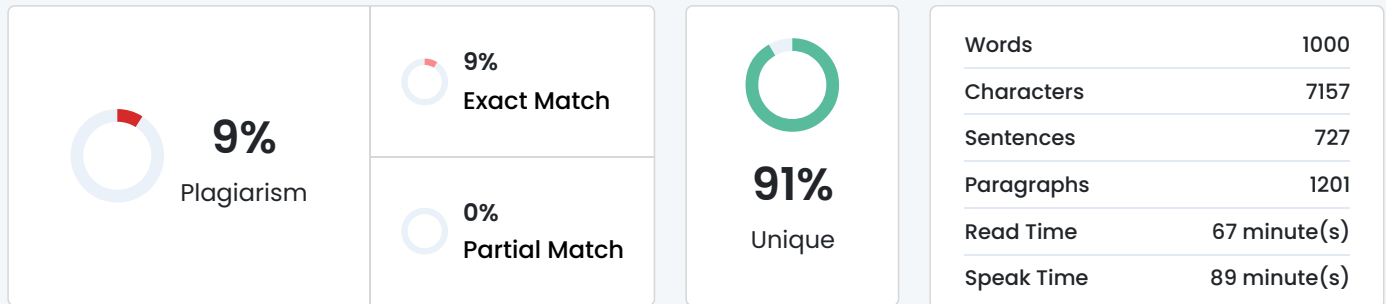
Part Number Table

Description	Part Number
Buzzer, Electromech, 6V DC	ABI-009-RC

Important Notice : This data sheet and its contents (the "Information") belong to the members of the Premier Farnell group of companies (the "Group") or are licensed to it. No licence is granted for the use of it other than for information purposes in connection with the products to which it relates. No licence of any intellectual property rights is granted. The Information is subject to change without notice and replaces all data sheets previously supplied. The Information supplied is believed to be accurate but the Group assumes no responsibility for its accuracy or completeness, any error in or omission from it or for any use made of it. Users of this data sheet should check for themselves the Information and the suitability of the products for their purpose and not make any assumptions based on information included or omitted. Liability for loss or damage resulting from any reliance on the Information or use of it (including liability resulting from negligence or where the Group was aware of the possibility of such loss or damage arising) is excluded. This will not operate to limit or restrict the Group's liability for death or personal injury resulting from its negligence. pro-SIGNAL is the registered trademark of the Group. © Premier Farnell plc 2012.

PLAGIARISM REPORT

Plagiarism Scan Report



Content Checked For Plagiarism

1

Department of E & TC, JSPM NTC, Pune

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Fire-related incidents remain a persistent and severe safety concern across industrial, commercial, and residential environments in India and globally. **Data from the National Crime Records Bureau (NCRB)** consistently places fire accidents among the leading causes of accidental deaths and property destruction, with thousands of incidents reported each year in urban and semi-urban settings. The human and economic toll of these events highlights a critical gap between the fire safety infrastructure currently deployed and the level of responsiveness that modern environments demand.

Conventional fire detection systems, which have remained largely unchanged in their fundamental design for several decades, depend on physical threshold-based sensors — ionization smoke detectors, photoelectric smoke sensors, and fixed-temperature heat elements. While these devices constitute the foundational layer of fire safety infrastructure, they exhibit a well-documented architectural limitation: their operation is predicated entirely on physical proximity. An alarm is **triggered only after smoke particles or thermal energy accumulate beyond a threshold concentration** within the sensor's immediate detection chamber. In large, open, or mechanically ventilated environments such as warehouses, server rooms, or manufacturing floors, the physical dispersion of combustion byproducts can be significantly delayed by air movement and structural barriers, allowing a containable fire to develop into a major emergency before any alarm is raised [2][4].

Furthermore, most installed conventional systems provide only a binary output — either an alarm is triggered or it is not — with no visual context made available to responders or administrators. In practice, this means that emergency personnel can only assess the severity and location of an incident upon physical arrival at the scene, contributing to inefficient resource deployment and delayed intervention.

The evolution of Edge Artificial Intelligence and convolutional neural network-based object detection offers a fundamentally different approach to fire detection. The YOLO (You Only Look Once) family of single-pass detection architectures, which frames detection as a unified regression problem processed in a single forward pass [1][2], has demonstrated real-time detection performance suitable for safety-critical applications.

Unlike earlier multi-stage detectors, YOLO-based models can analyze a complete video frame and
2

Department of E & TC, JSPM NTC, Pune

return detection outputs at frame rates sufficient for live surveillance systems [3][5].

To make such AI capability practically deployable without specialized hardware, this project employs the Raspberry Pi 4 Model B as the central processing unit in combination with the ONNX (Open Neural Network Exchange) Runtime, which is used to execute an optimized YOLOv8 model with improved inference throughput and reduced memory consumption compared to native framework execution [3][4]. This design choice makes high-accuracy, camera-based fire detection viable on a low-cost embedded platform. Parallel to the visual detection pipeline, an ESP8266 NodeMCU microcontroller handles dedicated sensor acquisition from an MQ-2 Smoke/Gas Sensor and an LM393 Infrared Flame Sensor, providing a hardware-level detection layer that operates independently of the camera and network.

The resulting system — a Smart Fire Detection and Surveillance System — represents a hybrid architecture that unifies AI-based visual analysis, hardware sensor monitoring, a multi-level alert pipeline, and a web-based remote monitoring dashboard into a single, cohesive safety framework. This approach addresses both the detection accuracy limitations and the remote monitoring deficiencies of conventional fire alarm systems

1.2 Problem Statement

Despite widespread deployment of conventional fire safety systems, three fundamental technical deficiencies continue to limit their effectiveness in modern environments and motivate the development of the proposed system:

1. Proximity-Dependent Detection with Delayed Response: Standard smoke and gas sensors require physical atmospheric changes — smoke particles or thermal energy — to reach and exceed a threshold at the sensor's detection element before an alarm is triggered. In environments with active ventilation, elevated ceilings, or structural partitions, this physical travel time introduces a detection delay during which a fire may grow beyond the window where suppression is straightforward [2][4]. Vision-based detection, by contrast, can identify the visual signature of flames or smoke plumes the moment they appear within the camera's field of view, regardless of atmospheric dispersion condition
 2. Absence of Real-Time Remote Monitoring and Visual Verification: Conventional alarm systems typically provide only a local audible alert. When facility administrators or safety personnel are off-site, there is no mechanism to remotely assess the nature or severity
- 3

Department of E & TC, JSPM NTC, Pune

of a detected event. This delays the appropriate escalation of response and can result in either under-response (treating a real fire as a false alarm) or over-response (evacuating unnecessarily for a nuisance alarm). A web-accessible live video feed integrated with sensor data addresses this gap by enabling remote visual verification before and during emergency response.

3. Computational and Hardware Constraints in Edge AI Deployment: Deploying deep learning detection models on resource-constrained single-board computers introduces known challenges. Standard YOLO model implementations in native training frameworks consume significant RAM and computational cycles, which can produce inference latency

that is unacceptable for life-safety applications on platforms such as the Raspberry Pi 4 [3][5]. Additionally, the high-frequency write cycles associated with continuous event logging on SD card storage — the standard storage medium for such embedded systems — accelerate media wear and can lead to storage failure in long-duration surveillance deployments, undermining system reliability.

This project directly addresses all three deficiencies by designing and implementing a system that integrates ONNX-optimized YOLOv8-based visual fire detection, dual hardware sensor monitoring (MQ-2 and LM393 via ESP8266), an intelligent four-case decision engine, and a Flask-React.js remote monitoring dashboard with cloud-backed data persistence

1.3 Objectives of the Project

The primary objective of this project is to design, develop, and validate a Smart Fire Detection and Surveillance System on the Raspberry Pi 4 platform that provides more reliable and faster fire detection than conventional sensor-only alarm systems. The specific objectives are as follows:

1. Real-Time Visual Fire Detection: — To implement the YOLOv8 object detection model, converted to and executed in the ONNX format, for real-time identification of fire and smoke in live video streams captured by a Pi-compatible camera module

Matched Source

Similarity 4%

Title:[Home | U.S. Department of Education](#)

Home | U.S. Department of Education U.S. Department of Education (.gov)<http://www.ed.gov>U.S. Department of Education (.gov)<http://www.ed.gov>
<http://www.ed.gov>

Similarity 1%

Title:[AN ANALYSIS ON THE NCRB DATA: THE CRIME STORY](#)

Dec 6, 2023 · Mains Focus THE CONTEXT: The latest edition of the National Crime Records Bureau (NCRB) report reveals disturbing trends. The recorded crimes against women, children, senior citizens, Scheduled Castes, and Scheduled Tribes show a steep increase, 468 people took their lives every day in 2022 and cybercrimes...
<https://blog.lukmaanias.com/2023/12/06/express-view-on-latest-ncrb-data-the-crime-story>

Similarity 1%

Title:[Part 2: Smoke concentration and detector characteristics in fires](#)

Part 2: Smoke concentration and detector characteristics in fires
ScienceDirect.com<https://www.sciencedirect.com/science/article/pii/S2214157X25005520>
piiScienceDirect.com<https://www.sciencedirect.com/science/article/pii/S2214157X25005520>

Similarity 1%

Title:[Scalable edge computing architecture for multi-site ...](#)

Scalable edge computing architecture for multi-site ... ScienceDirect.com<https://www.sciencedirect.com/science/article/pii/S2214157X25005520>

Similarity 1%

Title: [What Are the Differences Between Conventional and ...](#)

What Are the Differences Between Conventional and ... Control Fire

Systems<https://www.controlfiresystems.com> › news › what-are-t...Control Fire

Systems<https://www.controlfiresystems.com> › news › what-are-t...

<https://www.controlfiresystems.com/news/what-are-the-differences-between-conventional-and-analogue-fire-alarm-systems>
